

IF3070 Dasar Intelegensi Artifisial

Tugas Besar 1



Disusun oleh: Kelompok 50

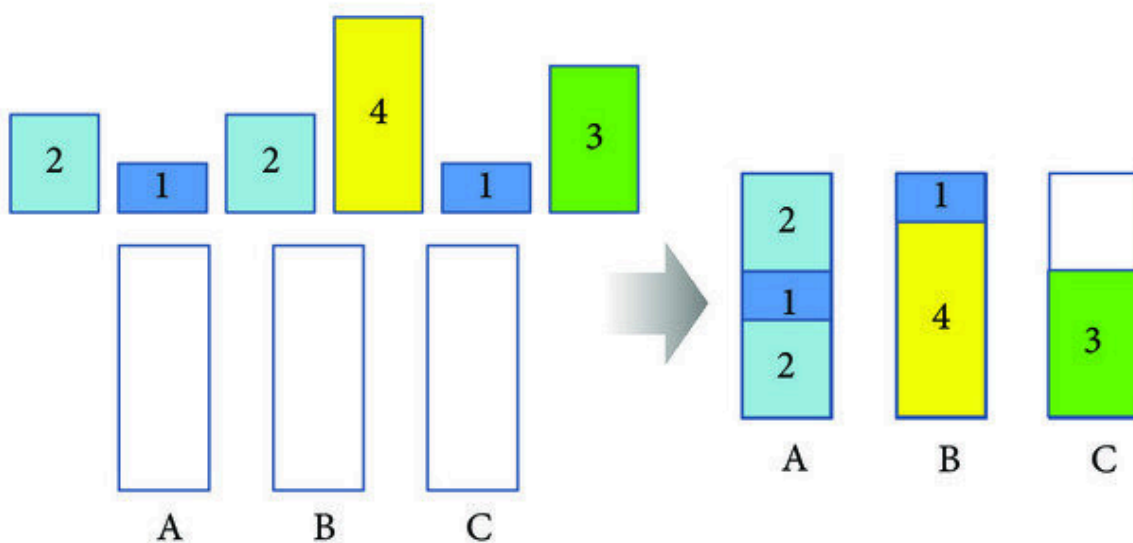
18223109 Fadhil Rifqi Rabbani Pane
18223122 Anggita Najmi Layali

**Program Studi Sistem dan Teknologi Informasi Sekolah Teknik
Elektro dan Informatika- Institut Teknologi Bandung
Jl. Ganesha 10, Bandung 40132**

Bab 1

Deskripsi Persoalan

Bin Packing Problem adalah permasalahan di mana suatu himpunan objek harus ditempatkan di sejumlah kontainer atau wadah, yang biasanya disebut “bin”, sedemikian rupa sehingga meminimalkan jumlah wadah yang digunakan. Setiap objek memiliki ukuran tertentu, sedangkan setiap bin memiliki kapasitas tertentu. Namun, sejumlah bin yang digunakan memiliki kapasitas yang sama.



Gambar 1. Visualisasi Bin Packing Problem

Dalam persoalan tugas besar ini, terdapat 3 aspek utama yang ingin dioptimasi sehingga solusi harus memenuhi *rules* yang memiliki nilai penalti, di antaranya:

- Objek-objek yang diletakkan dalam 1 bin TIDAK BOLEH melebihi kapasitas bin tersebut, atau akan dikenakan penalti. Solusi yang seperti ini akan dianggap solusi tidak valid.
- Setiap bin harus diisi SEPADAT MUNGKIN, mendekati nilai kapasitasnya. Semakin sedikit/kosong bin tersebut, semakin tinggi penaltinya.
- Penggunaan bin harus SEMINIMAL MUNGKIN. Semakin banyak jumlah bin yang dipakai, semakin tinggi penaltinya.

Oleh karena itu, algoritma local search digunakan untuk menyusun solusi sedemikian rupa sehingga ditemukan cara yang paling murah dan efisien untuk menempatkan objek-objek tersebut ke dalam tempat yang tepat.

Bab 2

Pembahasan

2.1 Pemilihan Objective Function

Objective function adalah fungsi-fungsi yang digunakan untuk menyelesaikan permasalahan optimasi dan memenuhi *rules* yang telah disebutkan pada deskripsi persoalan.

Nilai dari *objective function* akan berpengaruh pada efektivitas solusi. Semakin banyak penalti, maka semakin tinggi nilai *objective function* yang berarti semakin buruk pula kualitas solusi tersebut. Oleh karena itu, apabila suatu algoritma menemukan solusi paling ideal dari suatu *bin packing problem*, idealnya *objective function* pada *final state* lebih kecil dari *initial state*.

Berikut implementasi objective function yang digunakan pada program ini.

```
def objective_function(state, kapasitas):
    total_penalty = 0

    for current_kontainer in state.kontainer_list:
        total_isi = sum(barang.ukuran for barang in current_kontainer.isi)

        if total_isi > kapasitas:
            total_penalty += 1000 * (total_isi - kapasitas) # penalti over capacity
        else:
            total_penalty += (kapasitas - total_isi) # penalti ruang kosong

    total_penalty += len(state.kontainer_list) * 10 # penalti jumlah kontainer

    return total_penalty
```

- **Penalti over capacity:** jika total barang > kapasitas bin, diberikan penalti sangat besar (1000 x selisih kapasitas) sehingga algoritma menghindari solusi ini.
- **Penalti ruang kosong:** jika masih ada sisa kapasitas dalam bin yang terisi, penalti ditambahkan sebesar (kapasitas - total isi) untuk mendorong algoritma memilih bin yang padat.
- **Penalti jumlah kontainer:** setiap bin menambah biaya 10 agar algoritma memilih mengurangi jumlah bin yang digunakan .

2.2 Pemilihan Heuristic

Terdapat dua heuristik umum yang sering digunakan dalam *bin packing problem*, yaitu First-Fit Decreasing (FFD) yang bersifat greedy, dan Best-Fit Decreasing (BFD) yang bersifat dinamis. Secara teoretis, algoritma BFD mampu menghasilkan solusi yang lebih optimal karena selalu memilih kontainer dengan sisa ruang terkecil setelah penempatan barang. Namun, eksekusi BFD lebih mahal secara komputasi karena setiap penempatan barang membutuhkan pencarian ke seluruh kontainer yang tersedia.

Oleh karena itu, pada program ini dipilih heuristik FFD karena lebih efisien dari sisi waktu komputasi, stabil, dan cukup baik meskipun tidak selalu menghasilkan solusi paling optimal.

Pada tahap pembentukan *initial state*, heuristik FFD bekerja dengan mengurutkan seluruh barang berdasarkan ukuran dari terbesar ke terkecil. Selanjutnya, setiap barang ditempatkan pada kontainer pertama yang masih memiliki kapasitas untuk menampungnya. Apabila tidak ada kontainer yang valid, maka dibuat kontainer baru. Dengan cara ini, kualitas solusi awal menjadi cukup baik, deterministik (tidak random), dan tidak terlalu buruk untuk dijadikan titik awal algoritma optimasi selanjutnya.

Berikut adalah implementasi heuristik FFD pada program ini.

```
# Generate initial state dengan First Fit Decreasing (FFD) heuristic.
def generate_initial_state(barang_list, kapasitas):
    kontainer_list = [Kontainer(kapasitas)]
    for barang in barang_list:
        placed = False
        for k in kontainer_list:
            if k.tambah_barang(barang):
                placed = True
                break
        if not placed:
            new_k = Kontainer(kapasitas)
            new_k.tambah_barang(barang)
            kontainer_list.append(new_k)

    return State(kontainer_list)
```

2.3 Implementasi Entity

Sebelum mengimplementasikan algoritma local search, diperlukan algoritma untuk membuat entitas persoalan. Entity dalam program ini yaitu item (barang) dan bin (kontainer).

Berikut source code dari implementasi entity tersebut.

```
class Barang:
    def __init__(self, id_barang, ukuran):
        self.id = id_barang
        self.ukuran = ukuran

    def __repr__(self):
        return f"{self.id} ({self.ukuran})"

from .barang import Barang

class Kontainer:
    def __init__(self, kapasitas):
        self.kapasitas = kapasitas
        self.isi = []

    def tambah_barang(self, barang):
        if not isinstance(barang, Barang):
            raise TypeError("Hanya objek dari class Barang yang bisa dimasukkan ke dalam Kontainer.")

        if self.total_isi() + barang.ukuran <= self.kapasitas:
            self.isi.append(barang)
            return True
        return False

    def hapus_barang(self, barang):
        if barang in self.isi:
            self.isi.remove(barang)

    def total_isi(self):
        return sum(b.ukuran for b in self.isi)

    def __repr__(self):
        return f"Kontainer(total: {self.total_isi()}/{self.kapasitas}, isi: {self.isi})"
```

2.4 Main program

Sebelum masuk ke algoritma, diperlukan juga main program dari persoalan optimasi ini sebagai berikut.

```
import random
from initializer import generate_initial_state
from algorithm.hill_climbing.steepest_ascent.SteepestAscent import SteepestAscent
from algorithm.hill_climbing.sideways_move.HillClimbingSideways import
HillClimbingSideways
from algorithm.hill_climbing.random_restart.RandomRestartHillClimbing import
RandomRestartHillClimbing
from algorithm.hill_climbing.stochastic.StochasticHillClimbing import
StochasticHillClimbing
from algorithm.simulated_annealing.SimulatedAnnealing import SimulatedAnnealing
from algorithm.genetic.Genetic import GeneticAlgorithm
from utils import load_input
```

```
def main_menu():
```

```
    print(r"""
```

```
    _____

    IRASSHAIMASE! Selamat Datang di Bin Packing Optimizer!
```



```
    """)
```

```
    while True:
```

```
        print("""
```

```
            +-----+
            |   PILIH ALGORITMA YANG   |
            |         INGIN DICoba         |
            +-----+
```

```

"""
print("1. Steepest Ascent Hill-Climbing")
print("2. Hill-Climbing with Sideways Move")
print("3. Random Restart Hill-Climbing")
print("4. Stochastic Hill-Climbing")
print("5. Simulated Annealing")
print("6. Genetic Algorithm")
print("7. Keluar dari program")

pilihan = input("Masukkan nomor opsi (1-7): ")

if pilihan == '1':
    print("\nAnda memilih Steepest Ascent Hill-Climbing.")

    filename = input("Masukkan nama file input (misal: input.json): ")
    kapasitas, barang_list = load_input(filename)

    random.shuffle(barang_list)
    initial_state = generate_initial_state(barang_list, kapasitas)

    algo = SteepestAscent(initial_state, kapasitas)
    algo.run()

elif pilihan == '2':
    print("\nAnda memilih Hill-Climbing with Sideways Move.")
    filename = input("Masukkan nama file input (misal: input.json): ")
    kapasitas, barang_list = load_input(filename)

    random.shuffle(barang_list)
    initial_state = generate_initial_state(barang_list, kapasitas)

    algo = HillClimbingSideways(initial_state, kapasitas)
    algo.run()

elif pilihan == '3':
    print("\nAnda memilih Random Restart Hill-Climbing.")

    filename = input("Masukkan nama file input (misal: input.json): ")
    kapasitas, barang_list = load_input(filename)

```

```

        algo = RandomRestartHillClimbing(barang_list, kapasitas)
        algo.run()
    elif pilihan == '4':
        print("\nAnda memilih Stochastic Hill-Climbing.")

        filename = input("Masukkan nama file input (misal: input.json): ")
        kapasitas, barang_list = load_input(filename)

        random.shuffle(barang_list)
        initial_state = generate_initial_state(barang_list, kapasitas)

        algo = StochasticHillClimbing(initial_state, kapasitas)
        algo.run()

    elif pilihan == '5':
        print("\nAnda memilih Simulated Annealing.")

        filename = input("Masukkan nama file input (misal: input.json): ")
        kapasitas, barang_list = load_input(filename)

        random.shuffle(barang_list)
        initial_state = generate_initial_state(barang_list, kapasitas)

        algo = SimulatedAnnealing(initial_state, kapasitas)
        algo.run()

    elif pilihan == '6':
        print("\nAnda memilih Genetic Algorithm.")

        filename = input("Masukkan nama file input (misal: input.json): ")
        kapasitas, barang_list = load_input(filename)

        initial_state = generate_initial_state(barang_list, kapasitas)
        ga = GeneticAlgorithm(initial_state, kapasitas)
        ga.run()

    elif pilihan == '7':
        print("\nTerima kasih telah menggunakan program ini. Keluar...")
        break
    else:
        print("\nOpsi tidak valid. Silakan coba lagi.")

```



```

if __name__ == "__main__":
    main_menu()
    print(r"""

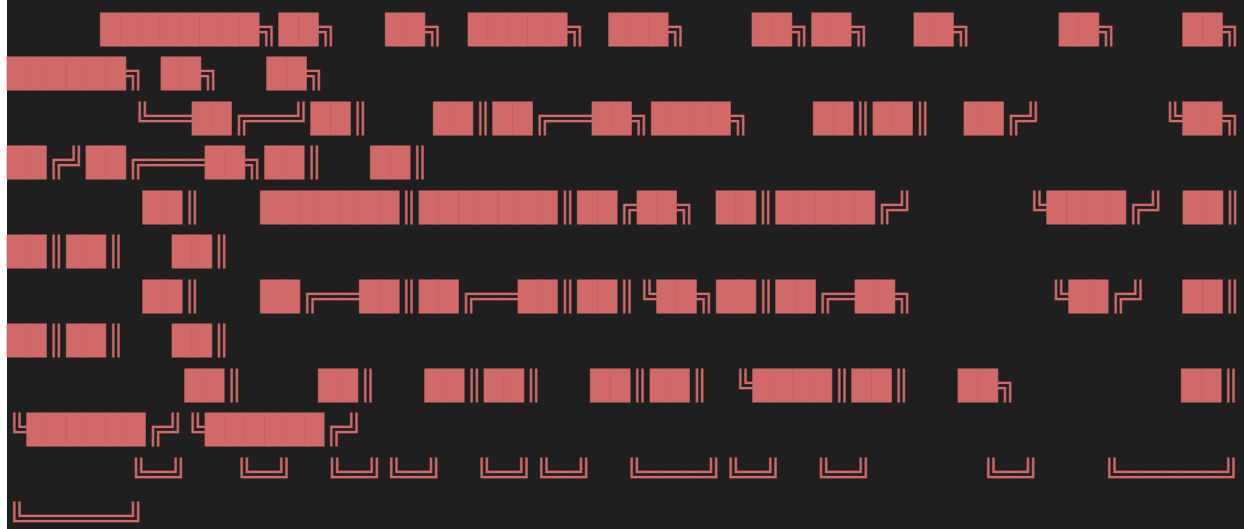
    _____

    SAYONARA! Terima kasih telah menggunakan Bin Packing Optimizer!

    _____

    """)

```



Selain itu, terdapat source code lain yang digunakan untuk menginisialisasi state dan neighbour yang dapat dilihat dalam [repository](#).

2.5 Implementasi Algoritma Local Search

2.5.1 Steepest Ascent Hill Climbing

Tidak seperti varian hill-climbing lainnya yang langsung berpindah ke tetangga pertama yang memberikan peningkatan value, Steepest Ascent melakukan langkah tambahan yakni: menghasilkan semua state neighbour, menghitung objective value masing-masing neighbour, dan bergerak ke neighbour yang memiliki peningkatan terbaik dan mengulangi proses sampai tidak ada solusi terbaik.

a. Source code Steepest Ascent

```

import matplotlib.pyplot as plt
import time
from bin.objective_function import objective_function
from bin.neighbor_state import generate_neighbors

class SteepestAscent:

```

```

def __init__(self, initial_state, kapasitas):
    self.current_state = initial_state
    self.kapasitas = kapasitas
    self.current_value = objective_function(initial_state, kapasitas)
    # Simpan nilai objective awal sebagai iterasi ke-0
    self.objective_values = [self.current_value]

def search_best_neighbor(self):
    best_neighbor = None
    best_value = self.current_value

    neighbors = generate_neighbors(self.current_state)
    for neighbor in neighbors:
        value = objective_function(neighbor, self.kapasitas)
        if value < best_value:
            best_value = value
            best_neighbor = neighbor
    return best_neighbor, best_value

def run(self):
    total_iteration = 0

    print("\n=====")
    print("STATE AWAL STEEPEST ASCENT")
    print("=====")
    print(self.current_state)
    print(f"Objective awal: {self.current_value}")
    print(f"Total Kontainer: {len(self.current_state.kontainer_list)}\n")

    start_time = time.time()

    while True:
        best_neighbor, best_value = self.search_best_neighbor()
        if best_neighbor is not None and best_value < self.current_value:
            self.current_state = best_neighbor
            self.current_value = best_value
            total_iteration += 1

            # Hanya log saat ada perbaikan
            self.objective_values.append(self.current_value)

            print(
                f"Iterasi {total_iteration} → Objective: {self.current_value}
                f"(Kontainer: {len(self.current_state.kontainer_list)})"
            )
        else:
            # Tidak ada perbaikan lagi (local optimum)
            break

```

```

        elapsed_time = time.time() - start_time

        print("\n=====")
        print("STATE AKHIR STEEPEST ASCENT")
        print("=====")
        print(self.current_state)
        print(f"Nilai Objective Akhir : {self.current_value}")
        print(f"Total Kontainer      : {len(self.current_state.kontainer_list)}")
        print(f"Total Iterasi       : {total_iteration}")
        print(f"Waktu Eksekusi        : {elapsed_time:.3f} detik\n")

        self.show_plot(self.objective_values,
                        title=f"Perkembangan Nilai Objective Function\n(Waktu:
{elapsed_time:.2f} detik)"
                        )
        return self.current_state, self.current_value, elapsed_time

    @staticmethod
    def show_plot(objective_values, title="Perkembangan Nilai Objective Function"):
        if len(objective_values) < 2:
            print("Tidak cukup data untuk menampilkan grafik (solusi tidak
membaik).")
            return

        plt.figure(figsize=(10, 6))
        plt.plot(
            range(len(objective_values)),
            objective_values,
            label="Objective Value",
            color="blue",
            linewidth=2,
            marker='o'
        )
        plt.title(title)
        plt.xlabel("Iteration")
        plt.ylabel("Objective Function")
        plt.grid(True, linestyle="--", linewidth=0.5)
        plt.legend()
        plt.tight_layout()
        plt.show()

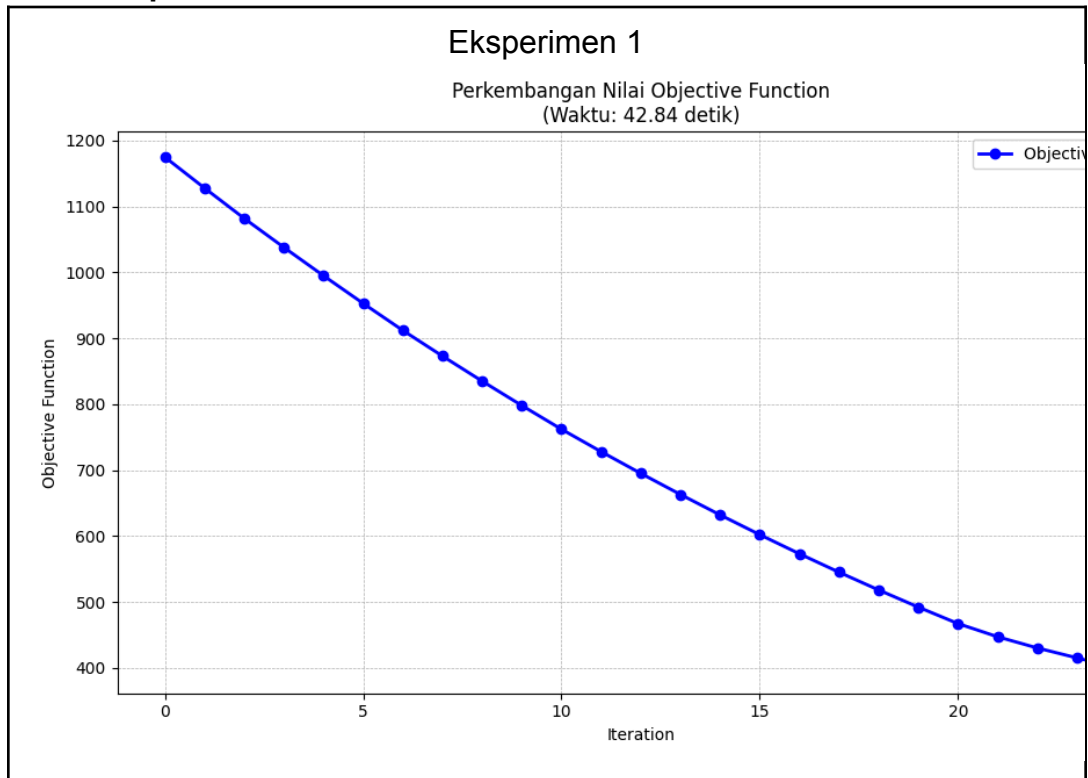
```

b. Fungsi dan Prosedur

Nama fungsi	Search_best_neighbour(self)
Deskripsi	Menghasilkan semua neighbor dari current state, menghitung objective-nya, kemudian memilih neighbor dengan nilai objective paling kecil (terbaik).

Output	Best_value dan best_neighbour
Source code	Bagian bertanda kuning pada source code Steepest Ascent.

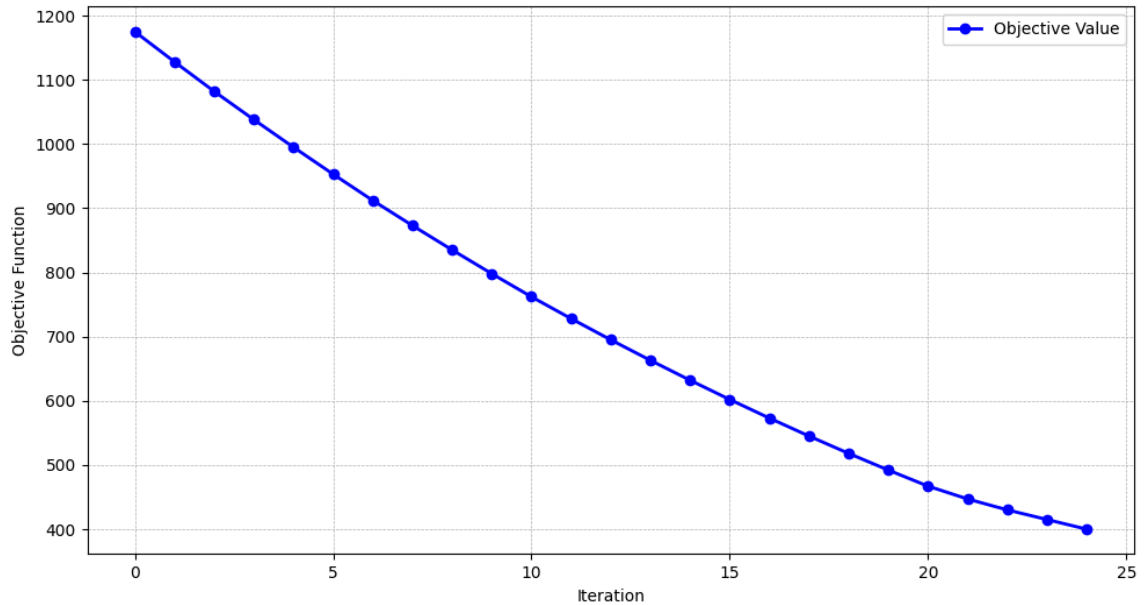
c. Hasil Eksperimen dan Analisis



State Awal	<pre>STATE AWAL STEEPEST ASCENT Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>		
State Akhir	<pre>STATE AKHIR STEEPEST ASCENT Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27), BRG020 (15)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 100/100, isi: [BRG028 (46), BRG029 (39), BRG020 (15)]) Kontainer 13: Kontainer(total: 100/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32), BRG016 (25)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG034 (34), BRG035 (36), BRG014 (30)]) Kontainer 15: Kontainer(total: 100/100, isi: [BRG036 (40), BRG037 (43), BRG031 (17)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 100/100, isi: [BRG042 (53), BRG039 (47)]) Kontainer 19: Kontainer(total: 100/100, isi: [BRG043 (54), BRG028 (46)]) Kontainer 20: Kontainer(total: 100/100, isi: [BRG044 (56), BRG038 (44)]) Kontainer 21: Kontainer(total: 100/100, isi: [BRG045 (57), BRG037 (43)]) Kontainer 22: Kontainer(total: 100/100, isi: [BRG046 (58), BRG010 (42)]) Kontainer 23: Kontainer(total: 100/100, isi: [BRG047 (59), BRG024 (41)]) Kontainer 24: Kontainer(total: 100/100, isi: [BRG048 (61), BRG029 (39)]) Kontainer 25: Kontainer(total: 100/100, isi: [BRG049 (62), BRG011 (38)]) Kontainer 26: Kontainer(total: 100/100, isi: [BRG050 (63), BRG025 (37)]) Kontainer 27: Kontainer(total: 100/100, isi: [BRG051 (64), BRG035 (36)]) Kontainer 28: Kontainer(total: 100/100, isi: [BRG052 (66), BRG034 (34)]) Kontainer 29: Kontainer(total: 100/100, isi: [BRG053 (67), BRG013 (33)]) Kontainer 30: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)]) Kontainer 31: Kontainer(total: 100/100, isi: [BRG055 (69), BRG023 (31)]) Kontainer 32: Kontainer(total: 100/100, isi: [BRG056 (71), BRG026 (29)]) Kontainer 33: Kontainer(total: 100/100, isi: [BRG057 (72), BRG015 (28)]) Kontainer 34: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Kontainer 35: Kontainer(total: 100/100, isi: [BRG059 (74), BRG032 (26)]) Kontainer 36: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)])</pre>		
Objective awal	1175	Objective akhir	400
Jml kontainer awal	36	Jml kontainer akhir	36
Jml iterasi	24		
Durasi	42.84 s		

Eksperimen 2

Perkembangan Nilai Objective Function
(Waktu: 41.84 detik)



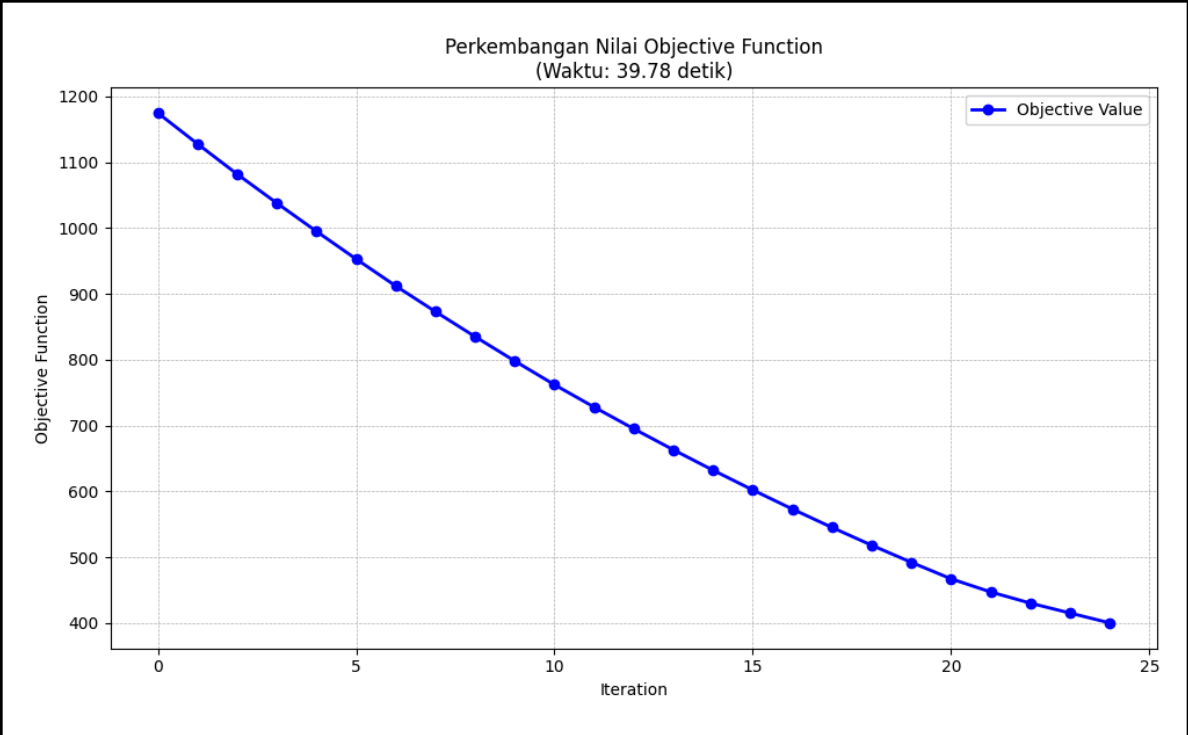
State Awal

STATE AWAL STEEPEST ASCENT

```
=====
Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)])
Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)])
Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)])
Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)])
Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)])
Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)])
Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)])
Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)])
Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)])
Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)])
Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)])
Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)])
Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)])
Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)])
Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)])
Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)])
Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)])
Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)])
Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)])
Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)])
Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)])
Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)])
Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)])
Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)])
Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)])
Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)])
Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)])
Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)])
Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)])
Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)])
Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)])
Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)])
Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)])
Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)])
Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)])
Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])
```

State Akhir	<pre>STATE AKHIR STEEPEST ASCENT Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27), BRG020 (15)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 100/100, isi: [BRG028 (46), BRG029 (39), BRG020 (15)]) Kontainer 13: Kontainer(total: 100/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32), BRG016 (25)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG034 (34), BRG035 (36), BRG014 (30)]) Kontainer 15: Kontainer(total: 100/100, isi: [BRG036 (40), BRG037 (43), BRG031 (17)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 100/100, isi: [BRG042 (53), BRG039 (47)]) Kontainer 19: Kontainer(total: 100/100, isi: [BRG043 (54), BRG028 (46)]) Kontainer 20: Kontainer(total: 100/100, isi: [BRG044 (56), BRG038 (44)]) Kontainer 21: Kontainer(total: 100/100, isi: [BRG045 (57), BRG037 (43)]) Kontainer 22: Kontainer(total: 100/100, isi: [BRG046 (58), BRG010 (42)]) Kontainer 23: Kontainer(total: 100/100, isi: [BRG047 (59), BRG024 (41)]) Kontainer 24: Kontainer(total: 100/100, isi: [BRG048 (61), BRG029 (39)]) Kontainer 25: Kontainer(total: 100/100, isi: [BRG049 (62), BRG011 (38)]) Kontainer 26: Kontainer(total: 100/100, isi: [BRG050 (63), BRG025 (37)]) Kontainer 27: Kontainer(total: 100/100, isi: [BRG051 (64), BRG035 (36)]) Kontainer 28: Kontainer(total: 100/100, isi: [BRG052 (66), BRG034 (34)]) Kontainer 29: Kontainer(total: 100/100, isi: [BRG053 (67), BRG013 (33)]) Kontainer 30: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)]) Kontainer 31: Kontainer(total: 100/100, isi: [BRG055 (69), BRG023 (31)]) Kontainer 32: Kontainer(total: 100/100, isi: [BRG056 (71), BRG026 (29)]) Kontainer 33: Kontainer(total: 100/100, isi: [BRG057 (72), BRG015 (28)]) Kontainer 34: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Kontainer 35: Kontainer(total: 100/100, isi: [BRG059 (74), BRG032 (26)]) Kontainer 36: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)])</pre>		
Objective awal	1175	Objective akhir	400
Jml kontainer awal	36	Jml kontainer akhir	36
Jml iterasi	24		
Durasi	41.842 s		

Eksperimen 3



State Awal

```
STATE AWAL STEEPEST ASCENT
Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)])
Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)])
Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)])
Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)])
Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)])
Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)])
Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)])
Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)])
Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)])
Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)])
Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)])
Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)])
Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)])
Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)])
Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)])
Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)])
Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)])
Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)])
Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)])
Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)])
Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)])
Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)])
Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)])
Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)])
Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)])
Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)])
Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)])
Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)])
Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)])
Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)])
Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)])
Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)])
Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)])
Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)])
Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)])
Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])
```


State Akhir	<pre> STATE AKHIR STEEPEST ASCENT Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27), BRG020 (15)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 100/100, isi: [BRG028 (46), BRG029 (39), BRG020 (15)]) Kontainer 13: Kontainer(total: 100/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32), BRG016 (25)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG034 (34), BRG035 (36), BRG014 (30)]) Kontainer 15: Kontainer(total: 100/100, isi: [BRG036 (40), BRG037 (43), BRG031 (17)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 100/100, isi: [BRG042 (53), BRG039 (47)]) Kontainer 19: Kontainer(total: 100/100, isi: [BRG043 (54), BRG028 (46)]) Kontainer 20: Kontainer(total: 100/100, isi: [BRG044 (56), BRG038 (44)]) Kontainer 21: Kontainer(total: 100/100, isi: [BRG045 (57), BRG037 (43)]) Kontainer 22: Kontainer(total: 100/100, isi: [BRG046 (58), BRG010 (42)]) Kontainer 23: Kontainer(total: 100/100, isi: [BRG047 (59), BRG024 (41)]) Kontainer 24: Kontainer(total: 100/100, isi: [BRG048 (61), BRG029 (39)]) Kontainer 25: Kontainer(total: 100/100, isi: [BRG049 (62), BRG011 (38)]) Kontainer 26: Kontainer(total: 100/100, isi: [BRG050 (63), BRG025 (37)]) Kontainer 27: Kontainer(total: 100/100, isi: [BRG051 (64), BRG035 (36)]) Kontainer 28: Kontainer(total: 100/100, isi: [BRG052 (66), BRG034 (34)]) Kontainer 29: Kontainer(total: 100/100, isi: [BRG053 (67), BRG013 (33)]) Kontainer 30: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)]) Kontainer 31: Kontainer(total: 100/100, isi: [BRG055 (69), BRG023 (31)]) Kontainer 32: Kontainer(total: 100/100, isi: [BRG056 (71), BRG026 (29)]) Kontainer 33: Kontainer(total: 100/100, isi: [BRG057 (72), BRG015 (28)]) Kontainer 34: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Kontainer 35: Kontainer(total: 100/100, isi: [BRG059 (74), BRG032 (26)]) Kontainer 36: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)]) </pre>		
Objective awal	1175	Objective akhir	400
Jml kontainer awal	36	Jml kontainer akhir	36
Jml iterasi	24		
Durasi	39.78 s		

Analisis

Steepest Ascent Hill Climbing menunjukkan performa sangat stabil dan deterministik. Pada tiga eksperimen, algoritma selalu mencapai solusi yang sama (objective 400, 36 kontainer, 24 iterasi). Meskipun waktu eksekusi tidak identik, rentangnya tetap konsisten. Namun, hasil berhenti di local optimum karena algoritma hanya bergerak ke tetangga terbaik tanpa eksplorasi solusi di luar jalur lokal. Jika ingin hasil di bawah 400, diperlukan teknik tambahan seperti random restart atau meta-heuristic lain (misalnya Simulated Annealing atau Genetic Algorithm).

2.5.2 Stochastic Hill Climbing

Stochastic Hill Climbing merupakan varian hill climbing yang tidak mengecek dan mengevaluasi semua tetangga, melainkan memilih satu tetangga secara random di setiap iterasinya. Jika tetangga yang dipilih secara random tersebut lebih baik daripada current state, maka algoritma akan pindah ke state tetangga tersebut. Stochastic Hill Climbing akan berhenti setelah algoritma gagal menemukan tetangga yang memiliki state yang lebih baik.

a. Source code Stochastic

```
import matplotlib.pyplot as plt
import time
import random
from bin.objective_function import objective_function
from bin.neighbor_state import generate_neighbors

class StochasticHillClimbing:
    def __init__(self, initial_state, kapasitas, max_iter=100, max_no_improve=10):
        self.current_state = initial_state
        self.kapasitas = kapasitas
        self.current_value = objective_function(initial_state, kapasitas)
        self.objective_values = [self.current_value]
        self.max_iter = max_iter
        self.max_no_improve = max_no_improve

    def run(self):
        print("\n=====")
        print("    STATE AWAL STOCHASTIC    ")
        print("=====")
        print(self.current_state)
        print(f"Objective awal : {self.current_value}")
        print(f"Jumlah kontainer: {len(self.current_state.kontainer_list)}\n")

        start_time = time.time()
        no_improve_count = 0

        for iterasi in range(1, self.max_iter + 1):
            neighbors = generate_neighbors(self.current_state)

            if not neighbors:
                print(f"Iterasi {iterasi}: tidak ada tetangga, berhenti.")
                break

            if len(neighbors) > 50:
                neighbors = random.sample(neighbors, 50)

            neighbor = random.choice(neighbors)
            value = objective_function(neighbor, self.kapasitas)

            if value < self.current_value:
                self.current_state = neighbor
                self.current_value = value
                no_improve_count = 0
                print(f"Iterasi {iterasi}: perbaikan → {value}")
            else:
                no_improve_count += 1

            self.objective_values.append(self.current_value)

            if no_improve_count >= self.max_no_improve:
                print(f"Iterasi {iterasi}: tidak ada perbaikan dalam {self.max_no_improve} iterasi, berhenti.")
                break

            if iterasi % 10 == 0:
                print(f"Iterasi {iterasi}: Objective = {self.current_value}")

        elapsed = time.time() - start_time
```

```

print("\n=====")
print("      STATE AKHIR STOCHASTIC      ")
print("=====")
print(self.current_state)
print(f"Objective akhir : {self.current_value}")
print(f"Total iterasi   : {iterasi}")
print(f"Jumlah kontainer: {len(self.current_state.kontainer_list)}")
print(f"Durasi eksekusi   : {elapsed:.3f} detik\n")

self.show_plot()
return self.current_state, self.current_value, elapsed

def show_plot(self):
    if len(self.objective_values) < 2:
        print("Tidak cukup data untuk plot.")
        return

    plt.figure(figsize=(10, 6))
    plt.plot(self.objective_values, marker='o', linewidth=2, markersize=4)
    plt.title("Perkembangan Objective Value per Iterasi")
    plt.xlabel("Iterasi")
    plt.ylabel("Objective Value")
    plt.grid(True, linestyle="--", linewidth=0.5, alpha=0.7)
    plt.tight_layout()
    plt.show()

```

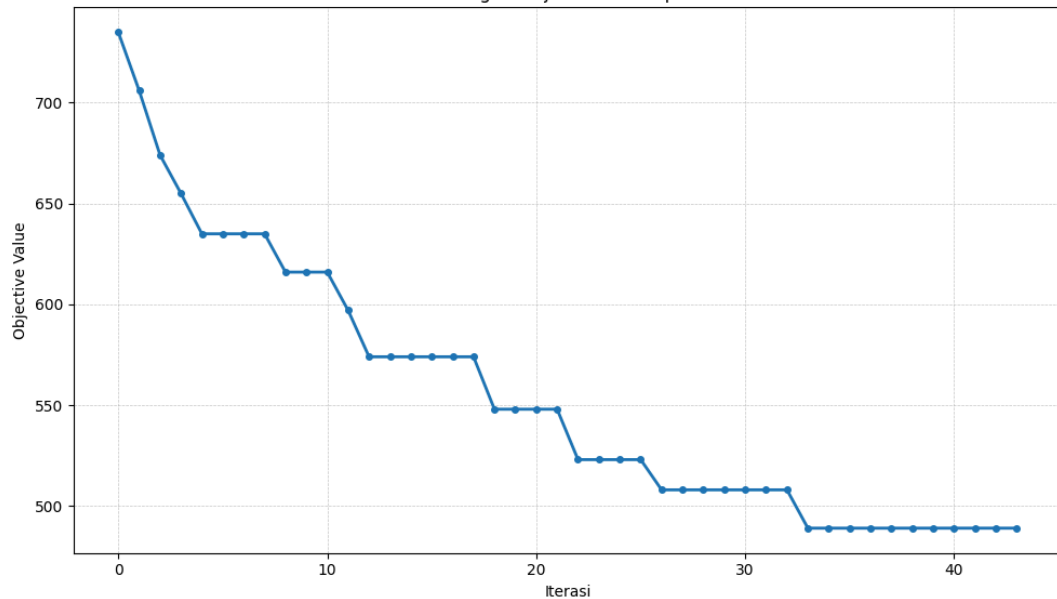
b. Fungsi dan Prosedur

Nama fungsi	run()
Deskripsi	Pada proses utama ini,
Output	Mengembalikan state terbaik
Source code	

c. Hasil Eksperimen

Eksperimen 1

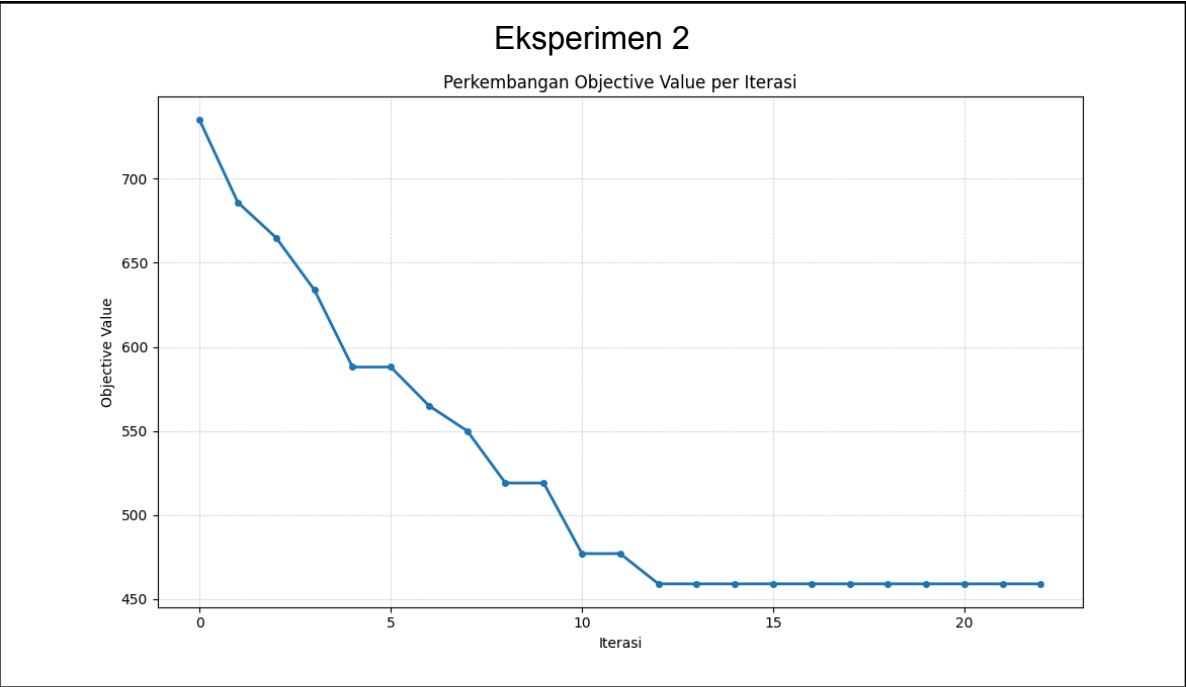
Perkembangan Objective Value per Iterasi



State Awal

```
=====
STATE AWAL STOCHASTIC
=====
Kontainer 1: Kontainer(total: 99/100, isi: [BRG032 (26), BRG037 (43), BRG014 (30)])
Kontainer 2: Kontainer(total: 100/100, isi: [BRG033 (32), BRG007 (50), BRG019 (18)])
Kontainer 3: Kontainer(total: 92/100, isi: [BRG046 (58), BRG034 (34)])
Kontainer 4: Kontainer(total: 98/100, isi: [BRG045 (57), BRG017 (22), BRG021 (19)])
Kontainer 5: Kontainer(total: 97/100, isi: [BRG005 (55), BRG010 (42)])
Kontainer 6: Kontainer(total: 95/100, isi: [BRG004 (60), BRG012 (35)])
Kontainer 7: Kontainer(total: 88/100, isi: [BRG003 (65), BRG027 (23)])
Kontainer 8: Kontainer(total: 94/100, isi: [BRG042 (53), BRG024 (41)])
Kontainer 9: Kontainer(total: 95/100, isi: [BRG002 (70), BRG016 (25)])
Kontainer 10: Kontainer(total: 95/100, isi: [BRG049 (62), BRG013 (33)])
Kontainer 11: Kontainer(total: 95/100, isi: [BRG053 (67), BRG015 (28)])
Kontainer 12: Kontainer(total: 93/100, isi: [BRG039 (47), BRG028 (46)])
Kontainer 13: Kontainer(total: 98/100, isi: [BRG043 (54), BRG038 (44)])
Kontainer 14: Kontainer(total: 95/100, isi: [BRG001 (75), BRG018 (20)])
Kontainer 15: Kontainer(total: 99/100, isi: [BRG041 (51), BRG008 (48)])
Kontainer 16: Kontainer(total: 99/100, isi: [BRG047 (59), BRG036 (40)])
Kontainer 17: Kontainer(total: 98/100, isi: [BRG006 (52), BRG026 (29), BRG031 (17)])
Kontainer 18: Kontainer(total: 96/100, isi: [BRG055 (69), BRG022 (27)])
Kontainer 19: Kontainer(total: 89/100, isi: [BRG059 (74), BRG020 (15)])
Kontainer 20: Kontainer(total: 99/100, isi: [BRG054 (68), BRG023 (31)])
Kontainer 21: Kontainer(total: 99/100, isi: [BRG050 (63), BRG035 (36)])
Kontainer 22: Kontainer(total: 92/100, isi: [BRG056 (71), BRG030 (21)])
Kontainer 23: Kontainer(total: 64/100, isi: [BRG051 (64)])
Kontainer 24: Kontainer(total: 80/100, isi: [BRG060 (80)])
Kontainer 25: Kontainer(total: 73/100, isi: [BRG058 (73)])
Kontainer 26: Kontainer(total: 75/100, isi: [BRG011 (38), BRG025 (37)])
Kontainer 27: Kontainer(total: 88/100, isi: [BRG040 (49), BRG029 (39)])
Kontainer 28: Kontainer(total: 56/100, isi: [BRG044 (56)])
Kontainer 29: Kontainer(total: 61/100, isi: [BRG048 (61)])
Kontainer 30: Kontainer(total: 72/100, isi: [BRG057 (72)])
Kontainer 31: Kontainer(total: 66/100, isi: [BRG052 (66)])
Kontainer 32: Kontainer(total: 45/100, isi: [BRG009 (45)])
```

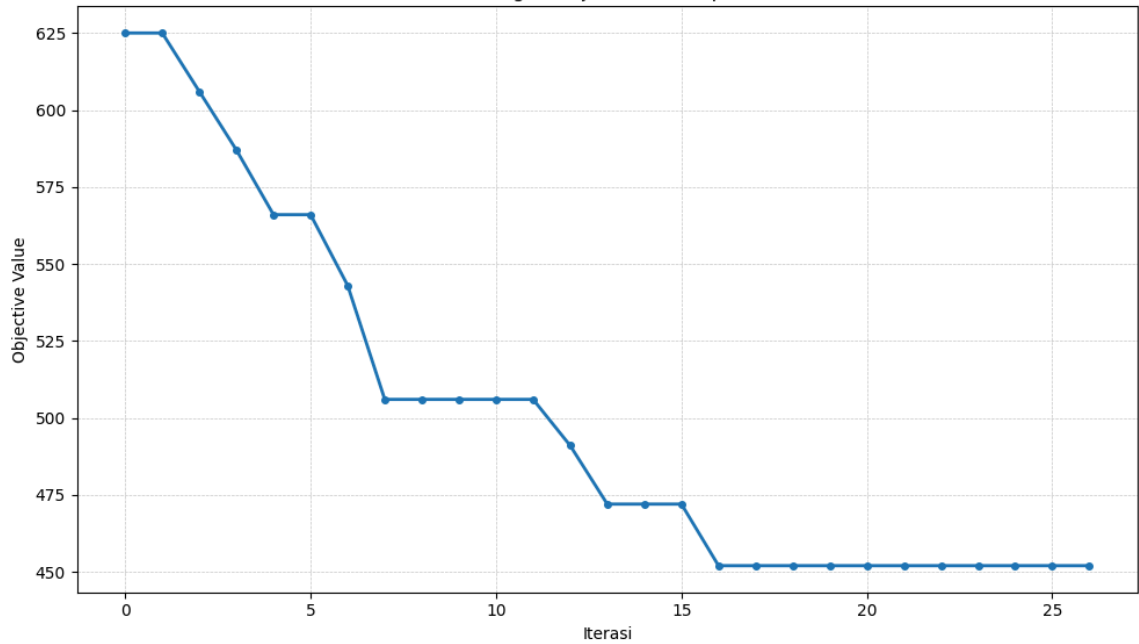
State Akhir	<pre>===== STATE AKHIR STOCHASTIC ===== Kontainer 1: Kontainer(total: 90/100, isi: [BRG032 (26), BRG037 (43), BRG014 (30)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG033 (32), BRG007 (50), BRG019 (18)]) Kontainer 3: Kontainer(total: 92/100, isi: [BRG046 (58), BRG034 (34)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG045 (57), BRG017 (22), BRG021 (19)]) Kontainer 5: Kontainer(total: 97/100, isi: [BRG005 (55), BRG010 (42)]) Kontainer 6: Kontainer(total: 95/100, isi: [BRG004 (60), BRG012 (35)]) Kontainer 7: Kontainer(total: 88/100, isi: [BRG003 (65), BRG027 (23)]) Kontainer 8: Kontainer(total: 94/100, isi: [BRG042 (53), BRG024 (41)]) Kontainer 9: Kontainer(total: 95/100, isi: [BRG002 (70), BRG016 (25)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG049 (62), BRG013 (33)]) Kontainer 11: Kontainer(total: 95/100, isi: [BRG053 (67), BRG015 (28)]) Kontainer 12: Kontainer(total: 93/100, isi: [BRG039 (47), BRG028 (46)]) Kontainer 13: Kontainer(total: 98/100, isi: [BRG043 (54), BRG038 (44)]) Kontainer 14: Kontainer(total: 95/100, isi: [BRG001 (75), BRG018 (20)]) Kontainer 15: Kontainer(total: 99/100, isi: [BRG041 (51), BRG008 (48)]) Kontainer 16: Kontainer(total: 99/100, isi: [BRG047 (59), BRG036 (40)]) Kontainer 17: Kontainer(total: 98/100, isi: [BRG006 (52), BRG026 (29), BRG031 (17)]) Kontainer 18: Kontainer(total: 96/100, isi: [BRG055 (69), BRG022 (27)]) Kontainer 19: Kontainer(total: 89/100, isi: [BRG059 (74), BRG020 (15)]) Kontainer 20: Kontainer(total: 99/100, isi: [BRG054 (68), BRG023 (31)]) Kontainer 21: Kontainer(total: 99/100, isi: [BRG050 (63), BRG035 (36)]) Kontainer 22: Kontainer(total: 92/100, isi: [BRG056 (71), BRG030 (21)]) Kontainer 23: Kontainer(total: 96/100, isi: [BRG051 (64), BRG033 (32)]) Kontainer 24: Kontainer(total: 95/100, isi: [BRG000 (80), BRG020 (15)]) Kontainer 25: Kontainer(total: 92/100, isi: [BRG058 (73), BRG021 (19)]) Kontainer 26: Kontainer(total: 98/100, isi: [BRG011 (38), BRG025 (37), BRG027 (23)]) Kontainer 27: Kontainer(total: 88/100, isi: [BRG040 (49), BRG029 (39)]) Kontainer 28: Kontainer(total: 95/100, isi: [BRG044 (56), BRG018 (20), BRG021 (19)]) Kontainer 29: Kontainer(total: 86/100, isi: [BRG048 (61), BRG016 (25)]) Kontainer 30: Kontainer(total: 98/100, isi: [BRG057 (72), BRG032 (26)]) Kontainer 31: Kontainer(total: 85/100, isi: [BRG052 (66), BRG021 (19)]) Kontainer 32: Kontainer(total: 93/100, isi: [BRG009 (45), BRG026 (29), BRG021 (19)])</pre>		
Objective awal	735	Objective akhir	489
Jml kontainer awal	32	Jml kontainer akhir	32
Jml iterasi	43		
Durasi	54.646 s		



State Awal	<pre>===== STATE AWAL STOCHASTIC ===== Kontainer 1: Kontainer(total: 99/100, isi: [BRG032 (26), BRG037 (43), BRG014 (30)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG033 (32), BRG007 (50), BRG019 (18)]) Kontainer 3: Kontainer(total: 92/100, isi: [BRG046 (58), BRG034 (34)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG045 (57), BRG017 (22), BRG021 (19)]) Kontainer 5: Kontainer(total: 97/100, isi: [BRG005 (55), BRG010 (42)]) Kontainer 6: Kontainer(total: 95/100, isi: [BRG004 (60), BRG012 (35)]) Kontainer 7: Kontainer(total: 88/100, isi: [BRG003 (65), BRG027 (23)]) Kontainer 8: Kontainer(total: 94/100, isi: [BRG042 (53), BRG024 (41)]) Kontainer 9: Kontainer(total: 95/100, isi: [BRG002 (70), BRG016 (25)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG049 (62), BRG013 (33)]) Kontainer 11: Kontainer(total: 95/100, isi: [BRG053 (67), BRG015 (28)]) Kontainer 12: Kontainer(total: 93/100, isi: [BRG039 (47), BRG028 (46)]) Kontainer 13: Kontainer(total: 98/100, isi: [BRG043 (54), BRG038 (44)]) Kontainer 14: Kontainer(total: 95/100, isi: [BRG001 (75), BRG018 (20)]) Kontainer 15: Kontainer(total: 99/100, isi: [BRG041 (51), BRG008 (48)]) Kontainer 16: Kontainer(total: 99/100, isi: [BRG047 (59), BRG036 (40)]) Kontainer 17: Kontainer(total: 98/100, isi: [BRG006 (52), BRG026 (29), BRG031 (17)]) Kontainer 18: Kontainer(total: 96/100, isi: [BRG055 (69), BRG022 (27)]) Kontainer 19: Kontainer(total: 89/100, isi: [BRG059 (74), BRG020 (15)]) Kontainer 20: Kontainer(total: 99/100, isi: [BRG054 (68), BRG023 (31)]) Kontainer 21: Kontainer(total: 99/100, isi: [BRG050 (63), BRG035 (36)]) Kontainer 22: Kontainer(total: 92/100, isi: [BRG056 (71), BRG030 (21)]) Kontainer 23: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 24: Kontainer(total: 80/100, isi: [BRG060 (80)]) Kontainer 25: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 26: Kontainer(total: 75/100, isi: [BRG011 (38), BRG025 (37)]) Kontainer 27: Kontainer(total: 88/100, isi: [BRG040 (49), BRG029 (39)]) Kontainer 28: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 29: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 30: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 31: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 32: Kontainer(total: 45/100, isi: [BRG009 (45)])</pre>		
State Akhir	<pre>===== STATE AKHIR STOCHASTIC ===== Kontainer 1: Kontainer(total: 94/100, isi: [BRG029 (39), BRG031 (17), BRG011 (38)]) Kontainer 2: Kontainer(total: 94/100, isi: [BRG008 (48), BRG018 (20), BRG032 (26)]) Kontainer 3: Kontainer(total: 90/100, isi: [BRG048 (61), BRG026 (29)]) Kontainer 4: Kontainer(total: 88/100, isi: [BRG024 (41), BRG039 (47)]) Kontainer 5: Kontainer(total: 95/100, isi: [BRG057 (72), BRG027 (23)]) Kontainer 6: Kontainer(total: 96/100, isi: [BRG013 (33), BRG050 (63)]) Kontainer 7: Kontainer(total: 95/100, isi: [BRG014 (30), BRG003 (65)]) Kontainer 8: Kontainer(total: 88/100, isi: [BRG045 (57), BRG023 (31)]) Kontainer 9: Kontainer(total: 94/100, isi: [BRG043 (54), BRG021 (19), BRG030 (21)]) Kontainer 10: Kontainer(total: 98/100, isi: [BRG051 (64), BRG034 (34)]) Kontainer 11: Kontainer(total: 86/100, isi: [BRG056 (71), BRG020 (15)]) Kontainer 12: Kontainer(total: 97/100, isi: [BRG001 (75), BRG017 (22)]) Kontainer 13: Kontainer(total: 100/100, isi: [BRG009 (45), BRG025 (37), BRG019 (18)]) Kontainer 14: Kontainer(total: 94/100, isi: [BRG052 (66), BRG015 (28)]) Kontainer 15: Kontainer(total: 99/100, isi: [BRG005 (55), BRG038 (44)]) Kontainer 16: Kontainer(total: 100/100, isi: [BRG004 (60), BRG036 (40)]) Kontainer 17: Kontainer(total: 95/100, isi: [BRG047 (59), BRG035 (36)]) Kontainer 18: Kontainer(total: 100/100, isi: [BRG046 (58), BRG010 (42)]) Kontainer 19: Kontainer(total: 96/100, isi: [BRG028 (46), BRG033 (32), BRG019 (18)]) Kontainer 20: Kontainer(total: 97/100, isi: [BRG049 (62), BRG012 (35)]) Kontainer 21: Kontainer(total: 99/100, isi: [BRG044 (56), BRG037 (43)]) Kontainer 22: Kontainer(total: 95/100, isi: [BRG002 (70), BRG016 (25)]) Kontainer 23: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Kontainer 24: Kontainer(total: 99/100, isi: [BRG054 (68), BRG023 (31)]) Kontainer 25: Kontainer(total: 99/100, isi: [BRG007 (50), BRG040 (49)]) Kontainer 26: Kontainer(total: 100/100, isi: [BRG041 (51), BRG040 (49)]) Kontainer 27: Kontainer(total: 90/100, isi: [BRG055 (69), BRG030 (21)]) Kontainer 28: Kontainer(total: 95/100, isi: [BRG060 (80), BRG020 (15)]) Kontainer 29: Kontainer(total: 98/100, isi: [BRG053 (67), BRG023 (31)]) Kontainer 30: Kontainer(total: 99/100, isi: [BRG042 (53), BRG028 (46)]) Kontainer 31: Kontainer(total: 94/100, isi: [BRG006 (52), BRG010 (42)]) Kontainer 32: Kontainer(total: 97/100, isi: [BRG059 (74), BRG027 (23)])</pre>		
Objective awal	735	Objective akhir	459
Jml kontainer awal	32	Jml kontainer akhir	32
Jml iterasi	22		
Durasi	26.265 s		

Eksperimen 3

Perkembangan Objective Value per Iterasi



State Awal

```
=====
STATE AWAL STOCHASTIC
=====
Kontainer 1: Kontainer(total: 99/100, isi: [BRG032 (26), BRG037 (43), BRG014 (30)])
Kontainer 2: Kontainer(total: 100/100, isi: [BRG033 (32), BRG007 (50), BRG019 (18)])
Kontainer 3: Kontainer(total: 92/100, isi: [BRG046 (58), BRG034 (34)])
Kontainer 4: Kontainer(total: 98/100, isi: [BRG045 (57), BRG017 (22), BRG021 (19)])
Kontainer 5: Kontainer(total: 97/100, isi: [BRG005 (55), BRG010 (42)])
Kontainer 6: Kontainer(total: 95/100, isi: [BRG004 (60), BRG012 (35)])
Kontainer 7: Kontainer(total: 88/100, isi: [BRG003 (65), BRG027 (23)])
Kontainer 8: Kontainer(total: 94/100, isi: [BRG042 (53), BRG024 (41)])
Kontainer 9: Kontainer(total: 95/100, isi: [BRG002 (70), BRG016 (25)])
Kontainer 10: Kontainer(total: 95/100, isi: [BRG049 (62), BRG013 (33)])
Kontainer 11: Kontainer(total: 95/100, isi: [BRG053 (67), BRG015 (28)])
Kontainer 12: Kontainer(total: 93/100, isi: [BRG039 (47), BRG028 (46)])
Kontainer 13: Kontainer(total: 98/100, isi: [BRG043 (54), BRG038 (44)])
Kontainer 14: Kontainer(total: 95/100, isi: [BRG001 (75), BRG018 (20)])
Kontainer 15: Kontainer(total: 99/100, isi: [BRG041 (51), BRG008 (48)])
Kontainer 16: Kontainer(total: 99/100, isi: [BRG047 (59), BRG036 (40)])
Kontainer 17: Kontainer(total: 98/100, isi: [BRG006 (52), BRG026 (29), BRG031 (17)])
Kontainer 18: Kontainer(total: 96/100, isi: [BRG055 (69), BRG022 (27)])
Kontainer 19: Kontainer(total: 89/100, isi: [BRG059 (74), BRG020 (15)])
Kontainer 20: Kontainer(total: 99/100, isi: [BRG054 (68), BRG023 (31)])
Kontainer 21: Kontainer(total: 99/100, isi: [BRG050 (63), BRG035 (36)])
Kontainer 22: Kontainer(total: 92/100, isi: [BRG056 (71), BRG030 (21)])
Kontainer 23: Kontainer(total: 64/100, isi: [BRG051 (64)])
Kontainer 24: Kontainer(total: 80/100, isi: [BRG060 (80)])
Kontainer 25: Kontainer(total: 73/100, isi: [BRG058 (73)])
Kontainer 26: Kontainer(total: 75/100, isi: [BRG011 (38), BRG025 (37)])
Kontainer 27: Kontainer(total: 88/100, isi: [BRG040 (49), BRG029 (39)])
Kontainer 28: Kontainer(total: 56/100, isi: [BRG044 (56)])
Kontainer 29: Kontainer(total: 61/100, isi: [BRG048 (61)])
Kontainer 30: Kontainer(total: 72/100, isi: [BRG057 (72)])
Kontainer 31: Kontainer(total: 66/100, isi: [BRG052 (66)])
Kontainer 32: Kontainer(total: 45/100, isi: [BRG009 (45)])
```

State Akhir	<pre> ===== STATE AKHIR STOCHASTIC ===== Kontainer 1: Kontainer(total: 86/100, isi: [BRG004 (60), BRG032 (26)]) Kontainer 2: Kontainer(total: 98/100, isi: [BRG048 (61), BRG025 (37)]) Kontainer 3: Kontainer(total: 96/100, isi: [BRG003 (65), BRG023 (31)]) Kontainer 4: Kontainer(total: 97/100, isi: [BRG006 (52), BRG009 (45)]) Kontainer 5: Kontainer(total: 93/100, isi: [BRG051 (64), BRG026 (29)]) Kontainer 6: Kontainer(total: 99/100, isi: [BRG046 (58), BRG024 (41)]) Kontainer 7: Kontainer(total: 97/100, isi: [BRG053 (67), BRG014 (30)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG052 (66), BRG033 (32)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG056 (71), BRG015 (28)]) Kontainer 10: Kontainer(total: 99/100, isi: [BRG050 (63), BRG035 (36)]) Kontainer 11: Kontainer(total: 98/100, isi: [BRG047 (59), BRG029 (39)]) Kontainer 12: Kontainer(total: 97/100, isi: [BRG038 (44), BRG042 (53)]) Kontainer 13: Kontainer(total: 97/100, isi: [BRG059 (74), BRG027 (23)]) Kontainer 14: Kontainer(total: 97/100, isi: [BRG012 (35), BRG049 (62)]) Kontainer 15: Kontainer(total: 87/100, isi: [BRG055 (69), BRG019 (18)]) Kontainer 16: Kontainer(total: 87/100, isi: [BRG057 (72), BRG020 (15)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG028 (46), BRG034 (34), BRG018 (20)]) Kontainer 18: Kontainer(total: 94/100, isi: [BRG058 (73), BRG030 (21)]) Kontainer 19: Kontainer(total: 97/100, isi: [BRG044 (56), BRG017 (22), BRG021 (19)]) Kontainer 20: Kontainer(total: 99/100, isi: [BRG005 (55), BRG022 (27), BRG031 (17)]) Kontainer 21: Kontainer(total: 98/100, isi: [BRG043 (54), BRG016 (25), BRG021 (19)]) Kontainer 22: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)]) Kontainer 23: Kontainer(total: 98/100, isi: [BRG041 (51), BRG039 (47)]) Kontainer 24: Kontainer(total: 98/100, isi: [BRG008 (48), BRG007 (50)]) Kontainer 25: Kontainer(total: 99/100, isi: [BRG045 (57), BRG010 (42)]) Kontainer 26: Kontainer(total: 88/100, isi: [BRG036 (40), BRG013 (33), BRG020 (15)]) Kontainer 27: Kontainer(total: 92/100, isi: [BRG037 (43), BRG040 (49)]) Kontainer 28: Kontainer(total: 94/100, isi: [BRG001 (75), BRG021 (19)]) Kontainer 29: Kontainer(total: 87/100, isi: [BRG054 (68), BRG021 (19)]) Kontainer 30: Kontainer(total: 91/100, isi: [BRG002 (70), BRG030 (21)]) Kontainer 31: Kontainer(total: 98/100, isi: [BRG011 (38), BRG027 (23), BRG025 (37)]) </pre>		
Objective awal	735	Objective akhir	452
Jml kontainer awal	32	Jml kontainer akhir	31
Jml iterasi	26		
Durasi	27.627 s		

d. Analisis

Berdasarkan tiga pengujian, stochastic hill-climbing menunjukkan perilaku yang tidak deterministik akibat pemilihan neighbour secara acak. Ketiga eksperimen selalu mampu menurunkan nilai objective secara signifikan dari 735 menjadi rentang 452–489. Pada eksperimen ke-3, algoritma bahkan berhasil mengurangi jumlah kontainer dari 32 menjadi 31, yang tidak dicapai oleh steepest ascent. Hal ini menunjukkan bahwa strategi eksplorasi acak membuat algoritma mampu keluar dari plateau/shoulder atau local optimum yang tidak dapat dilewati oleh steepest ascent.

Namun, kualitas solusi bervariasi antar percobaan dan ketidakpastian pemilihan tetangga menyebabkan performanya tidak selalu stabil, serta membutuhkan waktu eksekusi yang lebih besar pada beberapa skenario.

2.5.3 Hill Climbing with Sideways Move

Hill Climbing with Sideways Move merupakan algoritma yang dimodifikasi dari Steepest Ascent Hill Climbing. Dalam algoritma ini, dia tidak hanya menerima state tetangga yang lebih bagus, namun juga state yang memiliki skor yang sama. Hal ini berguna agar algoritma tidak stuck di dataran. Algoritma ini juga memiliki parameter *max_sideways_moves* untuk membatasi jumlah langkah sideways yang dapat dilakukan oleh algoritma.

a. Source code Sideways Hill Climbing

```
import time
import matplotlib.pyplot as plt
from bin.objective_function import objective_function as evaluate
from bin.neighbor_state import generate_neighbors

class HillClimbingSideways:
    def __init__(self, initial_state, kapasitas):
        self.kapasitas = kapasitas

        self.current_state = initial_state
        self.current_eval = evaluate(self.current_state, self.kapasitas)

        self.best_state = initial_state
        self.best_eval = self.current_eval
        self.scores_history = [self.best_eval]

    def _plot_scores(self):
        plt.figure(figsize=(12, 6))
        plt.plot(self.scores_history)
        plt.title("Score vs Iteration (Hill Climbing w/ Sideways Move)")
        plt.xlabel("Iteration")
        plt.ylabel("Best Score")
        plt.grid(True)
        print("Menampilkan Plot Skor...")
        plt.show()

    def run(self):
        print("\n--- STATE AWAL ---")
        print(self.current_state)
        print(f"Skor Awal: {self.current_eval}")
        print("-----\n")

        max_sideways_moves = int(input("Masukkan Max Sideways Moves (misal: 10): "))

        print("--- Memulai Hill Climbing w/ Sideways Move ---")
        start_time = time.time()
        sideways_move_count = 0
        iteration = 0

        while True:
            neighbors = generate_neighbors(self.current_state)

            if not neighbors:
```

```

        print("Tidak ada tetangga yang bisa digenerate. Berhenti.")
        break

    best_neighbor = None
    best_neighbor_eval = float("inf")

    for neighbor in neighbors:
        eval_n = evaluate(neighbor, self.kapasitas)
        if eval_n < best_neighbor_eval:
            best_neighbor_eval = eval_n
            best_neighbor = neighbor

    self.current_eval = evaluate(self.current_state, self.kapasitas)

    if best_neighbor_eval < self.current_eval:
        self.current_state = best_neighbor
        sideways_move_count = 0

        if best_neighbor_eval < self.best_eval:
            self.best_eval = best_neighbor_eval
            self.best_state = best_neighbor

    elif best_neighbor_eval == self.current_eval:
        if sideways_move_count >= max_sideways_moves:
            print(f"\nBatas maksimum {max_sideways_moves} sideways move tercapai.
Berhenti.")
            break
        else:
            self.current_state = best_neighbor
            sideways_move_count += 1

    else:
        print("\nTidak ada langkah yang lebih baik atau sideways. Terjebak di local
optimum.")
        break

    self.scores_history.append(self.best_eval)
    print(f"Iter: {iteration}, Best: {self.best_eval:.2f}, Current:
{evaluate(self.current_state, self.kapasitas):.2f}, Sideways:
{sideways_move_count}/{max_sideways_moves}")
    iteration += 1

end_time = time.time()
duration = end_time - start_time

print("--- Hill Climbing w/ Sideways Move Selesai ---")
print(f"Durasi: {duration:.4f} detik")
print(f"Berhenti setelah {iteration} iterasi")
print(f"Best Score Ditemukan: {self.best_eval}")

print("\n--- STATE AKHIR (HASIL) ---")
print(self.best_state)
print(f"Total Kontainer: {self.best_state.total_kontainer()}")
print(f"Skor Akhir Terbaik: {self.best_eval}")

```

```

print("-----\n")

print("Tutup window plot untuk melanjutkan program")
self._plot_scores()

return

```

b. Fungsi dan Prosedur

Nama fungsi	<code>_plot_scores(self)</code>
Deskripsi	Fungsi untuk membuat visualisasi hasil menggunakan matplotlib, menunjukkan skor terbaik (scores).
Output	Menampilkan window plot.
Source code	Bagian bertanda kuning pada source code Sideways Hill Climbing.

c. Hasil Eksperimen

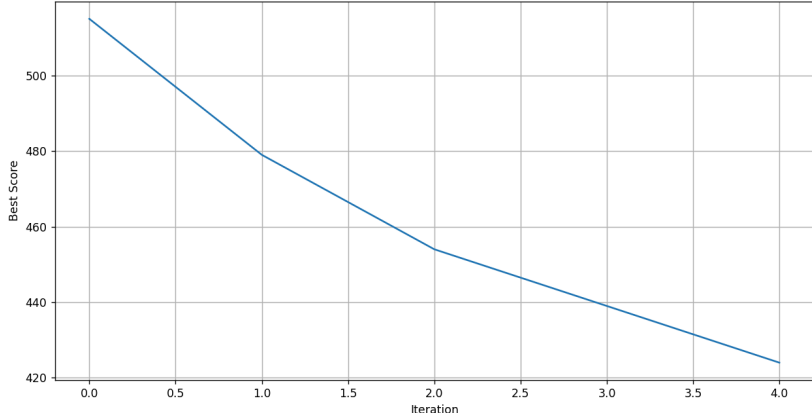
Eksperimen 1	
State Awal	<pre> --- STATE AWAL --- Kontainer 1: Kontainer(total: 100/100, isi: [BRG059 (74), BRG032 (26)]) Kontainer 2: Kontainer(total: 98/100, isi: [BRG009 (45), BRG042 (53)]) Kontainer 3: Kontainer(total: 98/100, isi: [BRG005 (55), BRG015 (28), BRG020 (15)]) Kontainer 4: Kontainer(total: 95/100, isi: [BRG045 (57), BRG011 (38)]) Kontainer 5: Kontainer(total: 84/100, isi: [BRG048 (61), BRG027 (23)]) Kontainer 6: Kontainer(total: 93/100, isi: [BRG007 (50), BRG037 (43)]) Kontainer 7: Kontainer(total: 99/100, isi: [BRG057 (72), BRG022 (27)]) Kontainer 8: Kontainer(total: 94/100, isi: [BRG006 (52), BRG010 (42)]) Kontainer 9: Kontainer(total: 89/100, isi: [BRG055 (69), BRG018 (20)]) Kontainer 10: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)]) Kontainer 11: Kontainer(total: 92/100, isi: [BRG024 (41), BRG041 (51)]) Kontainer 12: Kontainer(total: 96/100, isi: [BRG049 (62), BRG034 (34)]) Kontainer 13: Kontainer(total: 99/100, isi: [BRG052 (66), BRG013 (33)]) Kontainer 14: Kontainer(total: 93/100, isi: [BRG028 (46), BRG039 (47)]) Kontainer 15: Kontainer(total: 99/100, isi: [BRG060 (80), BRG021 (19)]) Kontainer 16: Kontainer(total: 90/100, isi: [BRG003 (65), BRG016 (25)]) Kontainer 17: Kontainer(total: 99/100, isi: [BRG002 (70), BRG026 (29)]) Kontainer 18: Kontainer(total: 97/100, isi: [BRG040 (49), BRG008 (48)]) Kontainer 19: Kontainer(total: 92/100, isi: [BRG044 (56), BRG035 (36)]) Kontainer 20: Kontainer(total: 90/100, isi: [BRG058 (73), BRG031 (17)]) Kontainer 21: Kontainer(total: 89/100, isi: [BRG056 (71), BRG019 (18)]) Kontainer 22: Kontainer(total: 97/100, isi: [BRG053 (67), BRG014 (30)]) Kontainer 23: Kontainer(total: 97/100, isi: [BRG001 (75), BRG017 (22)]) Kontainer 24: Kontainer(total: 100/100, isi: [BRG050 (63), BRG025 (37)]) Kontainer 25: Kontainer(total: 97/100, isi: [BRG046 (58), BRG029 (39)]) Kontainer 26: Kontainer(total: 84/100, isi: [BRG036 (40), BRG038 (44)]) Kontainer 27: Kontainer(total: 91/100, isi: [BRG004 (60), BRG023 (31)]) Kontainer 28: Kontainer(total: 94/100, isi: [BRG047 (59), BRG012 (35)]) Kontainer 29: Kontainer(total: 75/100, isi: [BRG043 (54), BRG030 (21)]) Kontainer 30: Kontainer(total: 64/100, isi: [BRG051 (64)]) Skor Awal: 515 ----- </pre>

State Akhir

```
--- STATE AKHIR (HASIL) ---
Kontainer 1: Kontainer(total: 100/100, isi: [BRG059 (74), BRG032 (26)])
Kontainer 2: Kontainer(total: 98/100, isi: [BRG009 (45), BRG042 (53)])
Kontainer 3: Kontainer(total: 98/100, isi: [BRG005 (55), BRG015 (28), BRG020 (15)])
Kontainer 4: Kontainer(total: 95/100, isi: [BRG045 (57), BRG011 (38)])
Kontainer 5: Kontainer(total: 99/100, isi: [BRG048 (61), BRG027 (23), BRG020 (15)])
Kontainer 6: Kontainer(total: 93/100, isi: [BRG007 (50), BRG037 (43)])
Kontainer 7: Kontainer(total: 99/100, isi: [BRG057 (72), BRG022 (27)])
Kontainer 8: Kontainer(total: 94/100, isi: [BRG006 (52), BRG010 (42)])
Kontainer 9: Kontainer(total: 89/100, isi: [BRG055 (69), BRG018 (20)])
Kontainer 10: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)])
Kontainer 11: Kontainer(total: 92/100, isi: [BRG024 (41), BRG041 (51)])
Kontainer 12: Kontainer(total: 96/100, isi: [BRG049 (62), BRG034 (34)])
Kontainer 13: Kontainer(total: 99/100, isi: [BRG052 (66), BRG013 (33)])
Kontainer 14: Kontainer(total: 93/100, isi: [BRG028 (46), BRG039 (47)])
Kontainer 15: Kontainer(total: 99/100, isi: [BRG060 (80), BRG021 (19)])
Kontainer 16: Kontainer(total: 90/100, isi: [BRG003 (65), BRG016 (25)])
Kontainer 17: Kontainer(total: 99/100, isi: [BRG002 (70), BRG026 (29)])
Kontainer 18: Kontainer(total: 97/100, isi: [BRG040 (49), BRG008 (48)])
Kontainer 19: Kontainer(total: 92/100, isi: [BRG044 (56), BRG035 (36)])
Kontainer 20: Kontainer(total: 90/100, isi: [BRG058 (73), BRG031 (17)])
Kontainer 21: Kontainer(total: 89/100, isi: [BRG056 (71), BRG019 (18)])
Kontainer 22: Kontainer(total: 97/100, isi: [BRG053 (67), BRG014 (30)])
Kontainer 23: Kontainer(total: 97/100, isi: [BRG001 (75), BRG017 (22)])
Kontainer 24: Kontainer(total: 100/100, isi: [BRG050 (63), BRG025 (37)])
Kontainer 25: Kontainer(total: 97/100, isi: [BRG046 (58), BRG029 (39)])
Kontainer 26: Kontainer(total: 99/100, isi: [BRG036 (40), BRG038 (44), BRG020 (15)])
Kontainer 27: Kontainer(total: 91/100, isi: [BRG004 (60), BRG023 (31)])
Kontainer 28: Kontainer(total: 94/100, isi: [BRG047 (59), BRG012 (35)])
Kontainer 29: Kontainer(total: 100/100, isi: [BRG043 (54), BRG030 (21), BRG016 (25)])
Kontainer 30: Kontainer(total: 100/100, isi: [BRG051 (64), BRG035 (36)])

Total Kontainer: 30
Skor Akhir Terbaik: 424
-----
```

Score vs Iteration (Hill Climbing w/ Sideways Move)

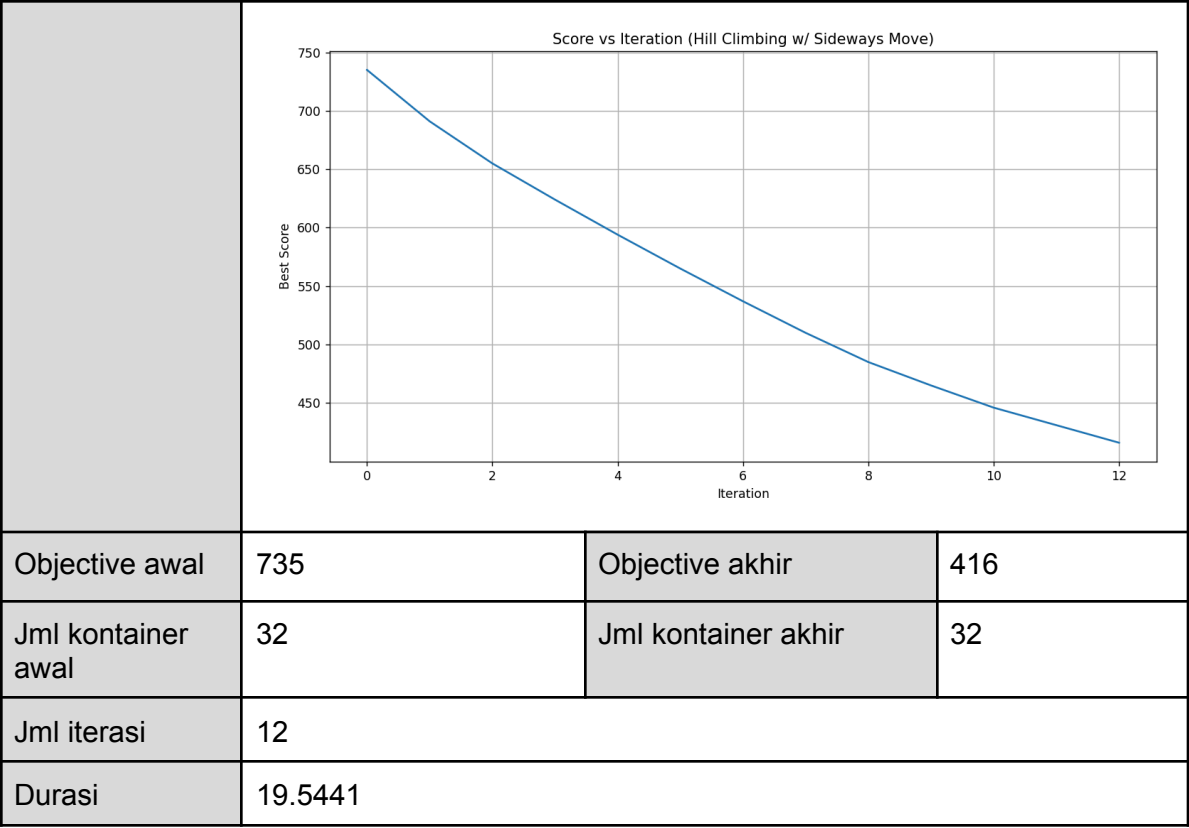


Iteration	Best Score
0.0	515
1.0	478
2.0	455
3.0	440
4.0	424

Objective awal	515	Objective akhir	474
Jml kontainer awal	30	Jml kontainer akhir	30
Jml iterasi	4		
Durasi	5.8498 detik		

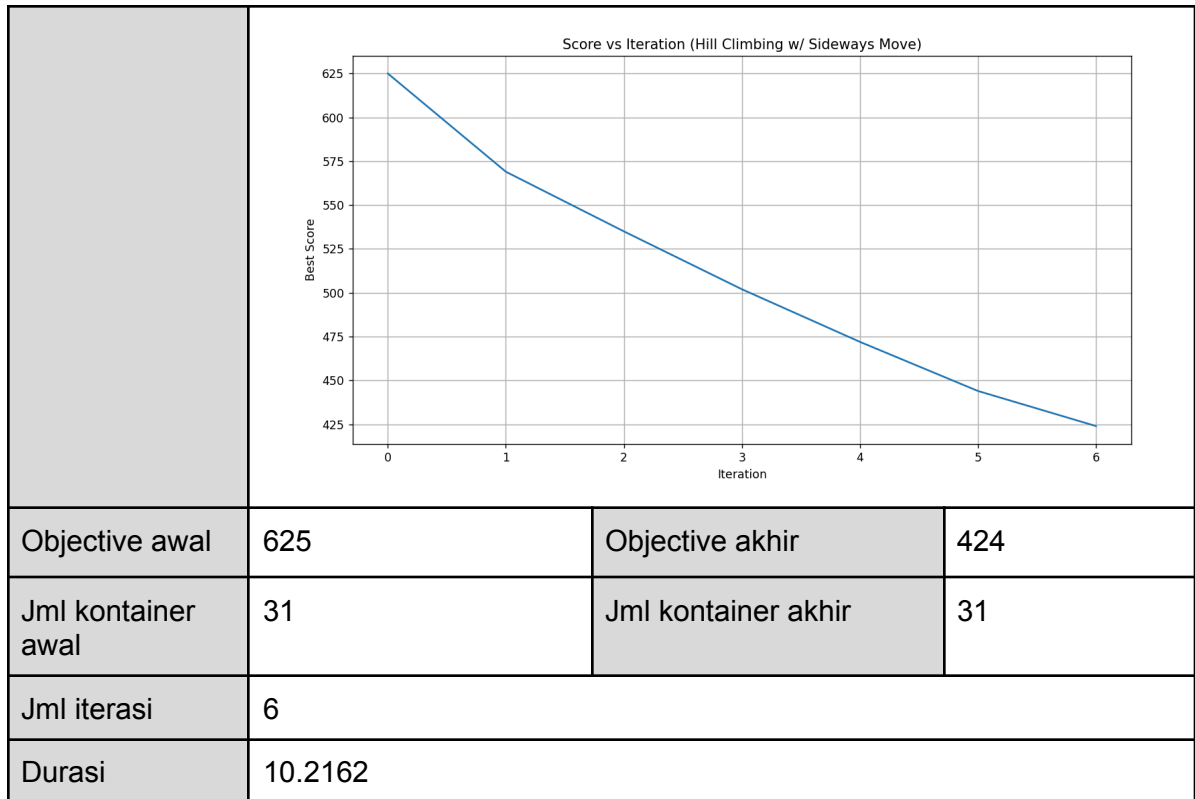
Eksperimen 2

State Awal	<pre>--- STATE AWAL --- Kontainer 1: Kontainer(total: 96/100, isi: [BRG021 (19), BRG049 (62), BRG020 (15)]) Kontainer 2: Kontainer(total: 85/100, isi: [BRG041 (51), BRG034 (34)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG004 (60), BRG036 (40)]) Kontainer 4: Kontainer(total: 94/100, isi: [BRG052 (66), BRG015 (28)]) Kontainer 5: Kontainer(total: 99/100, isi: [BRG045 (57), BRG010 (42)]) Kontainer 6: Kontainer(total: 99/100, isi: [BRG006 (52), BRG039 (47)]) Kontainer 7: Kontainer(total: 97/100, isi: [BRG003 (65), BRG033 (32)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG038 (44), BRG016 (25), BRG026 (29)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG028 (46), BRG042 (53)]) Kontainer 10: Kontainer(total: 91/100, isi: [BRG007 (50), BRG024 (41)]) Kontainer 11: Kontainer(total: 93/100, isi: [BRG043 (54), BRG029 (39)]) Kontainer 12: Kontainer(total: 86/100, isi: [BRG050 (63), BRG027 (23)]) Kontainer 13: Kontainer(total: 97/100, isi: [BRG053 (67), BRG014 (30)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG008 (48), BRG012 (35), BRG031 (17)]) Kontainer 15: Kontainer(total: 92/100, isi: [BRG059 (74), BRG019 (18)]) Kontainer 16: Kontainer(total: 81/100, isi: [BRG011 (38), BRG037 (43)]) Kontainer 17: Kontainer(total: 85/100, isi: [BRG046 (58), BRG022 (27)]) Kontainer 18: Kontainer(total: 88/100, isi: [BRG054 (68), BRG018 (20)]) Kontainer 19: Kontainer(total: 95/100, isi: [BRG035 (36), BRG047 (59)]) Kontainer 20: Kontainer(total: 92/100, isi: [BRG025 (37), BRG005 (55)]) Kontainer 21: Kontainer(total: 100/100, isi: [BRG017 (22), BRG032 (26), BRG030 (21), BRG023 (31)]) Kontainer 22: Kontainer(total: 94/100, isi: [BRG048 (61), BRG013 (33)]) Kontainer 23: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 24: Kontainer(total: 80/100, isi: [BRG060 (80)]) Kontainer 25: Kontainer(total: 75/100, isi: [BRG001 (75)]) Kontainer 26: Kontainer(total: 70/100, isi: [BRG002 (70)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 94/100, isi: [BRG040 (49), BRG009 (45)]) Kontainer 29: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 30: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 31: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 32: Kontainer(total: 69/100, isi: [BRG055 (69)]) Skor Awal: 735 -----</pre>
State Akhir	<pre>--- STATE AKHIR (HASIL) --- Kontainer 1: Kontainer(total: 96/100, isi: [BRG021 (19), BRG049 (62), BRG020 (15)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG041 (51), BRG034 (34), BRG020 (15)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG004 (60), BRG036 (40)]) Kontainer 4: Kontainer(total: 94/100, isi: [BRG052 (66), BRG015 (28)]) Kontainer 5: Kontainer(total: 99/100, isi: [BRG045 (57), BRG010 (42)]) Kontainer 6: Kontainer(total: 99/100, isi: [BRG006 (52), BRG039 (47)]) Kontainer 7: Kontainer(total: 97/100, isi: [BRG003 (65), BRG033 (32)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG038 (44), BRG016 (25), BRG026 (29)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG028 (46), BRG042 (53)]) Kontainer 10: Kontainer(total: 91/100, isi: [BRG007 (50), BRG024 (41)]) Kontainer 11: Kontainer(total: 93/100, isi: [BRG043 (54), BRG029 (39)]) Kontainer 12: Kontainer(total: 86/100, isi: [BRG050 (63), BRG027 (23)]) Kontainer 13: Kontainer(total: 97/100, isi: [BRG053 (67), BRG014 (30)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG008 (48), BRG012 (35), BRG031 (17)]) Kontainer 15: Kontainer(total: 92/100, isi: [BRG059 (74), BRG019 (18)]) Kontainer 16: Kontainer(total: 100/100, isi: [BRG011 (38), BRG037 (43), BRG021 (19)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG046 (58), BRG022 (27), BRG020 (15)]) Kontainer 18: Kontainer(total: 88/100, isi: [BRG054 (68), BRG018 (20)]) Kontainer 19: Kontainer(total: 95/100, isi: [BRG035 (36), BRG047 (59)]) Kontainer 20: Kontainer(total: 92/100, isi: [BRG025 (37), BRG005 (55)]) Kontainer 21: Kontainer(total: 100/100, isi: [BRG017 (22), BRG032 (26), BRG030 (21), BRG023 (31)]) Kontainer 22: Kontainer(total: 94/100, isi: [BRG048 (61), BRG013 (33)]) Kontainer 23: Kontainer(total: 100/100, isi: [BRG044 (56), BRG038 (44)]) Kontainer 24: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)]) Kontainer 25: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 26: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 27: Kontainer(total: 100/100, isi: [BRG051 (64), BRG035 (36)]) Kontainer 28: Kontainer(total: 94/100, isi: [BRG040 (49), BRG009 (45)]) Kontainer 29: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Kontainer 30: Kontainer(total: 100/100, isi: [BRG057 (72), BRG015 (28)]) Kontainer 31: Kontainer(total: 100/100, isi: [BRG056 (71), BRG026 (29)]) Kontainer 32: Kontainer(total: 100/100, isi: [BRG055 (69), BRG023 (31)]) Total Kontainer: 32 Skor Akhir Terbaik: 416 -----</pre>



Eksperimen 3

State Awal	<pre> --- STATE AWAL --- Kontainer 1: Kontainer(total: 97/100, isi: [BRG028 (46), BRG031 (17), BRG034 (34)]) Kontainer 2: Kontainer(total: 88/100, isi: [BRG047 (59), BRG026 (29)]) Kontainer 3: Kontainer(total: 92/100, isi: [BRG049 (62), BRG014 (30)]) Kontainer 4: Kontainer(total: 94/100, isi: [BRG005 (55), BRG018 (20), BRG021 (19)]) Kontainer 5: Kontainer(total: 86/100, isi: [BRG008 (48), BRG011 (38)]) Kontainer 6: Kontainer(total: 89/100, isi: [BRG044 (56), BRG013 (33)]) Kontainer 7: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 8: Kontainer(total: 99/100, isi: [BRG043 (54), BRG009 (45)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG058 (73), BRG032 (26)]) Kontainer 10: Kontainer(total: 97/100, isi: [BRG048 (61), BRG035 (36)]) Kontainer 11: Kontainer(total: 98/100, isi: [BRG045 (57), BRG024 (41)]) Kontainer 12: Kontainer(total: 97/100, isi: [BRG055 (69), BRG015 (28)]) Kontainer 13: Kontainer(total: 98/100, isi: [BRG050 (63), BRG012 (35)]) Kontainer 14: Kontainer(total: 95/100, isi: [BRG051 (64), BRG023 (31)]) Kontainer 15: Kontainer(total: 97/100, isi: [BRG003 (65), BRG033 (32)]) Kontainer 16: Kontainer(total: 97/100, isi: [BRG010 (42), BRG025 (37), BRG019 (18)]) Kontainer 17: Kontainer(total: 96/100, isi: [BRG004 (60), BRG030 (21), BRG020 (15)]) Kontainer 18: Kontainer(total: 99/100, isi: [BRG039 (47), BRG006 (52)]) Kontainer 19: Kontainer(total: 98/100, isi: [BRG056 (71), BRG022 (27)]) Kontainer 20: Kontainer(total: 91/100, isi: [BRG054 (68), BRG027 (23)]) Kontainer 21: Kontainer(total: 90/100, isi: [BRG041 (51), BRG029 (39)]) Kontainer 22: Kontainer(total: 98/100, isi: [BRG046 (58), BRG036 (40)]) Kontainer 23: Kontainer(total: 96/100, isi: [BRG059 (74), BRG017 (22)]) Kontainer 24: Kontainer(total: 80/100, isi: [BRG060 (80)]) Kontainer 25: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 26: Kontainer(total: 96/100, isi: [BRG042 (53), BRG037 (43)]) Kontainer 27: Kontainer(total: 70/100, isi: [BRG002 (70)]) Kontainer 28: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 29: Kontainer(total: 99/100, isi: [BRG007 (50), BRG040 (49)]) Kontainer 30: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 31: Kontainer(total: 44/100, isi: [BRG038 (44)]) Skor Awal: 625 ----- </pre>
State Akhir	<pre> --- STATE AKHIR (HASIL) --- Kontainer 1: Kontainer(total: 97/100, isi: [BRG028 (46), BRG031 (17), BRG034 (34)]) Kontainer 2: Kontainer(total: 88/100, isi: [BRG047 (59), BRG026 (29)]) Kontainer 3: Kontainer(total: 92/100, isi: [BRG049 (62), BRG014 (30)]) Kontainer 4: Kontainer(total: 94/100, isi: [BRG005 (55), BRG018 (20), BRG021 (19)]) Kontainer 5: Kontainer(total: 86/100, isi: [BRG008 (48), BRG011 (38)]) Kontainer 6: Kontainer(total: 89/100, isi: [BRG044 (56), BRG013 (33)]) Kontainer 7: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 8: Kontainer(total: 99/100, isi: [BRG043 (54), BRG009 (45)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG058 (73), BRG032 (26)]) Kontainer 10: Kontainer(total: 97/100, isi: [BRG048 (61), BRG035 (36)]) Kontainer 11: Kontainer(total: 98/100, isi: [BRG045 (57), BRG024 (41)]) Kontainer 12: Kontainer(total: 97/100, isi: [BRG055 (69), BRG015 (28)]) Kontainer 13: Kontainer(total: 98/100, isi: [BRG050 (63), BRG012 (35)]) Kontainer 14: Kontainer(total: 95/100, isi: [BRG051 (64), BRG023 (31)]) Kontainer 15: Kontainer(total: 97/100, isi: [BRG003 (65), BRG033 (32)]) Kontainer 16: Kontainer(total: 97/100, isi: [BRG010 (42), BRG025 (37), BRG019 (18)]) Kontainer 17: Kontainer(total: 96/100, isi: [BRG004 (60), BRG030 (21), BRG020 (15)]) Kontainer 18: Kontainer(total: 99/100, isi: [BRG039 (47), BRG006 (52)]) Kontainer 19: Kontainer(total: 98/100, isi: [BRG056 (71), BRG022 (27)]) Kontainer 20: Kontainer(total: 91/100, isi: [BRG054 (68), BRG027 (23)]) Kontainer 21: Kontainer(total: 90/100, isi: [BRG041 (51), BRG029 (39)]) Kontainer 22: Kontainer(total: 98/100, isi: [BRG046 (58), BRG036 (40)]) Kontainer 23: Kontainer(total: 96/100, isi: [BRG059 (74), BRG017 (22)]) Kontainer 24: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)]) Kontainer 25: Kontainer(total: 100/100, isi: [BRG053 (67), BRG013 (33)]) Kontainer 26: Kontainer(total: 96/100, isi: [BRG042 (53), BRG037 (43)]) Kontainer 27: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 28: Kontainer(total: 100/100, isi: [BRG057 (72), BRG015 (28)]) Kontainer 29: Kontainer(total: 99/100, isi: [BRG007 (50), BRG040 (49)]) Kontainer 30: Kontainer(total: 100/100, isi: [BRG052 (66), BRG034 (34)]) Kontainer 31: Kontainer(total: 100/100, isi: [BRG038 (44), BRG044 (56)]) Total Kontainer: 31 Skor Akhir Terbaik: 424 ----- </pre>



d. Analisis

Berdasarkan ketiga hasil eksperimen, hasil akhir menunjukkan bahwa algoritma kerap terjebak di local optimum, diindikasikan pada ketiga hasil akhir yang berbeda-beda. Di setiap eksperimen, algoritma selalu berhasil mencari dan mendapatkan solusi yang lebih baik dari state awalnya. Namun juga algoritma ini berhenti dengan cepat yang menunjukkan bahwa algoritma ini sangat cepat mencapai *plateau* atau local optimum.

2.5.4 Random-Restart Hill Climbing

Random-Restart Hill Climbing adalah salah satu variant algoritma hill climbing dimana algoritma akan menjalankan Steepest Ascent Hill Climbing secara terus menerus, dimana setiap kali dia menemukan neighbour yang tidak ideal, maka dia akan mengulangi hill climbing dari state awal yang baru dan secara acak. State awal acak ini dibuat dengan memilih daftar barang acak sebelum memanggil initializer. Algoritma ini akan menyimpan solusi terbaik (*global best*) yang ditemukan dari semua restart.

a. Source code Random-Restart

```
import time
import random
import matplotlib.pyplot as plt
from bin.objective_function import objective_function as evaluate
from bin.neighbor_state import generate_neighbors
from initializer import generate_initial_state
```



```

class RandomRestartHillClimbing:
    def __init__(self, barang_list, kapasitas):
        self.barang_list_master = barang_list
        self.kapasitas = kapasitas
        self.max_restarts = 0

        self.global_best_state = None
        self.global_best_eval = float("inf")

        self.scores_history = []
        self.iterations_per_restart = []

    def _plot_scores(self):
        plt.figure(figsize=(12, 6))
        plt.plot(self.scores_history)
        plt.title("Best Score vs. Total Iterations (Random Restart)")
        plt.xlabel("Total Iterations (in all restarts)")
        plt.ylabel("Global Best Score")
        plt.grid(True)
        print("Menampilkan Plot Skor...")
        plt.show()

    def _run_local_search(self, start_state):
        current_state = start_state
        current_eval = evaluate(current_state, self.kapasitas)
        iter_count = 0

        print(f" ... Memulai local search dari skor: {current_eval:.2f}")

        while True:
            neighbors = generate_neighbors(current_state)
            if not neighbors:
                break

            best_neighbor = None
            best_neighbor_eval = current_eval

            for neighbor in neighbors:
                eval_n = evaluate(neighbor, self.kapasitas)
                if eval_n < best_neighbor_eval:
                    best_neighbor_eval = eval_n
                    best_neighbor = neighbor

            if best_neighbor_eval < current_eval:
                current_state = best_neighbor
                current_eval = best_neighbor_eval
                iter_count += 1

            if current_eval < self.global_best_eval:
                self.global_best_eval = current_eval
                self.global_best_state = current_state
                print(f" ... Iterasi {iter_count}: Current {current_eval:.2f}, Global Best {self.global_best_eval:.2f} ", end='\r')

```

```

        else:
            break

        self.scores_history.append(self.global_best_eval)

    print()
    return current_state, current_eval, iter_count

def run(self):
    try:
        max_restarts_input = int(input("Masukkan Max Restarts (misal: 10): "))
    except ValueError:
        print("Error: Input harus angka. Menggunakan default 10.")
        max_restarts_input = 10

    self.max_restarts = max_restarts_input

    print(f"--- Memulai Random Restart Hill Climbing (Max Restarts: {self.max_restarts}) ---")
    start_time = time.time()

    self.scores_history = []

    for r in range(self.max_restarts):
        print(f"\n--- Restart {r + 1}/{self.max_restarts} ---")

        shuffled_list = self.barang_list_master[:]
        random.shuffle(shuffled_list)
        initial_state = generate_initial_state(shuffled_list, self.kapasitas)

        initial_eval = evaluate(initial_state, self.kapasitas)

        if initial_eval < self.global_best_eval:
            self.global_best_eval = initial_eval
            self.global_best_state = initial_state

        self.scores_history.append(self.global_best_eval)

        if r == 0:
            print("State Awal (dari Restart 1):")
            print(initial_state)
            print(f"Skor Awal: {initial_eval:.2f}")

        local_best_state, local_best_eval, iter_count = self._run_local_search(initial_state)

        print(f"Restart {r + 1} selesai setelah {iter_count} iterasi. Local Best: {local_best_eval:.2f}, Global Best: {self.global_best_eval:.2f}")
        self.iterations_per_restart.append(iter_count)

    end_time = time.time()
    duration = end_time - start_time

```

```

print("\n--- Random Restart Hill Climbing Selesai ---")
print(f"Durasi Total: {duration:.4f} detik")
print(f"Total Iterasi (di semua restart): {len(self.scores_history) - self.max_restarts}")
print(f"Best Score Ditemukan: {self.global_best_eval}")
if self.iterations_per_restart:
    print(f"Rata-rata iterasi per restart: {sum(self.iterations_per_restart) / len(self.iterations_per_restart):.2f}")

print("\n--- STATE AKHIR (HASIL GLOBAL TERBAIK) ---")
if self.global_best_state:
    print(self.global_best_state)
    print(f"Total Kontainer: {self.global_best_state.total_kontainer()}")
    print(f"Skor Akhir Terbaik: {self.global_best_eval}")
else:
    print("Tidak ada solusi yang ditemukan")
print("-----\n")

print("Menyiapkan plot...")
print("(Tutup window plot untuk melanjutkan program)")
self._plot_scores()

return

```

b. Fungsi dan Prosedur

Nama fungsi	<code>_plot_scores(self)</code>
Deskripsi	Fungsi untuk membuat visualisasi hasil menggunakan matplotlib, menunjukkan skor terbaik (scores).
Output	Menampilkan window plot
Source code	Bagian bertanda cyan pada source code Random-Restart.

Nama fungsi	<code>_run_local_search(self, start_state)</code>
Deskripsi	Fungsi yang menjalankan steepest ascent. Fungsi ini menerima <i>start_state</i> dan mencari local optimum dengan mengevaluasi semua tetangga. Selama pencarian juga mengupdate <i>global best</i> jika menemukan solusi yang lebih baik.
Output	<i>Local optimum (current_state, current_eval)</i> dan <i>iter_count</i>
Source code	Bagian bertanda kuning pada source code Random-Restart.

c. Hasil Eksperimen

Eksperimen 1

State Awal

```
--- Restart 1/7 ---
State Awal (dari Restart 1):
Kontainer 1: Kontainer(total: 93/100, isi: [BRG048 (61), BRG033 (32)])
Kontainer 2: Kontainer(total: 99/100, isi: [BRG036 (40), BRG038 (44), BRG020 (15)])
Kontainer 3: Kontainer(total: 87/100, isi: [BRG025 (37), BRG007 (50)])
Kontainer 4: Kontainer(total: 99/100, isi: [BRG041 (51), BRG023 (31), BRG031 (17)])
Kontainer 5: Kontainer(total: 94/100, isi: [BRG028 (46), BRG015 (28), BRG018 (20)])
Kontainer 6: Kontainer(total: 93/100, isi: [BRG051 (64), BRG026 (29)])
Kontainer 7: Kontainer(total: 93/100, isi: [BRG045 (57), BRG035 (36)])
Kontainer 8: Kontainer(total: 90/100, isi: [BRG053 (67), BRG027 (23)])
Kontainer 9: Kontainer(total: 98/100, isi: [BRG042 (53), BRG022 (27), BRG019 (18)])
Kontainer 10: Kontainer(total: 99/100, isi: [BRG052 (66), BRG013 (33)])
Kontainer 11: Kontainer(total: 99/100, isi: [BRG060 (80), BRG021 (19)])
Kontainer 12: Kontainer(total: 89/100, isi: [BRG054 (68), BRG030 (21)])
Kontainer 13: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)])
Kontainer 14: Kontainer(total: 97/100, isi: [BRG057 (72), BRG016 (25)])
Kontainer 15: Kontainer(total: 97/100, isi: [BRG056 (71), BRG032 (26)])
Kontainer 16: Kontainer(total: 97/100, isi: [BRG008 (48), BRG040 (49)])
Kontainer 17: Kontainer(total: 93/100, isi: [BRG047 (59), BRG034 (34)])
Kontainer 18: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)])
Kontainer 19: Kontainer(total: 87/100, isi: [BRG003 (65), BRG017 (22)])
Kontainer 20: Kontainer(total: 94/100, isi: [BRG010 (42), BRG006 (52)])
Kontainer 21: Kontainer(total: 98/100, isi: [BRG037 (43), BRG005 (55)])
Kontainer 22: Kontainer(total: 73/100, isi: [BRG058 (73)])
Kontainer 23: Kontainer(total: 97/100, isi: [BRG046 (58), BRG029 (39)])
Kontainer 24: Kontainer(total: 69/100, isi: [BRG055 (69)])
Kontainer 25: Kontainer(total: 99/100, isi: [BRG043 (54), BRG009 (45)])
Kontainer 26: Kontainer(total: 75/100, isi: [BRG001 (75)])
Kontainer 27: Kontainer(total: 97/100, isi: [BRG044 (56), BRG024 (41)])
Kontainer 28: Kontainer(total: 74/100, isi: [BRG059 (74)])
Kontainer 29: Kontainer(total: 82/100, isi: [BRG039 (47), BRG012 (35)])
Kontainer 30: Kontainer(total: 62/100, isi: [BRG049 (62)])
Kontainer 31: Kontainer(total: 63/100, isi: [BRG050 (63)])

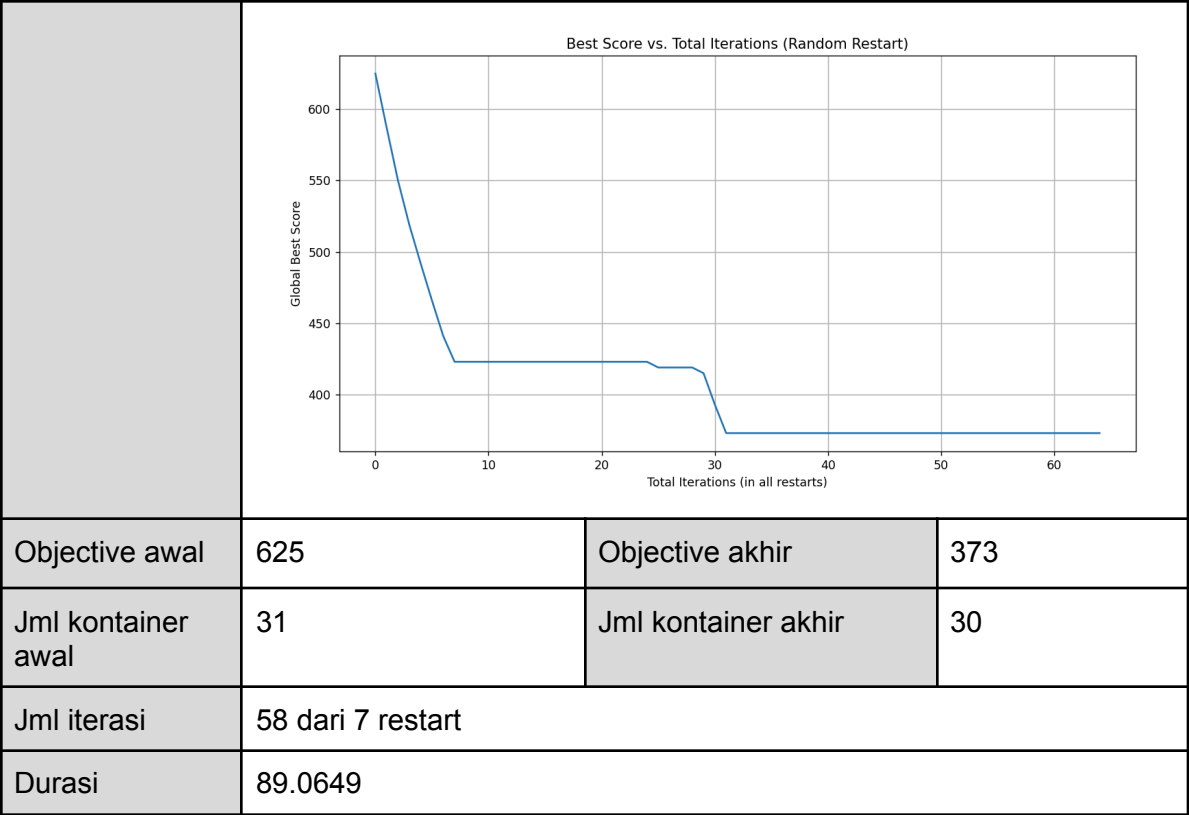
Skor Awal: 625.00
```

State Akhir

```
--- Random Restart Hill Climbing Selesai ---
Durasi Total: 89.0649 detik
Total Iterasi (di semua restart): 58
Best Score Ditemukan: 373
Rata-rata iterasi per restart: 8.29

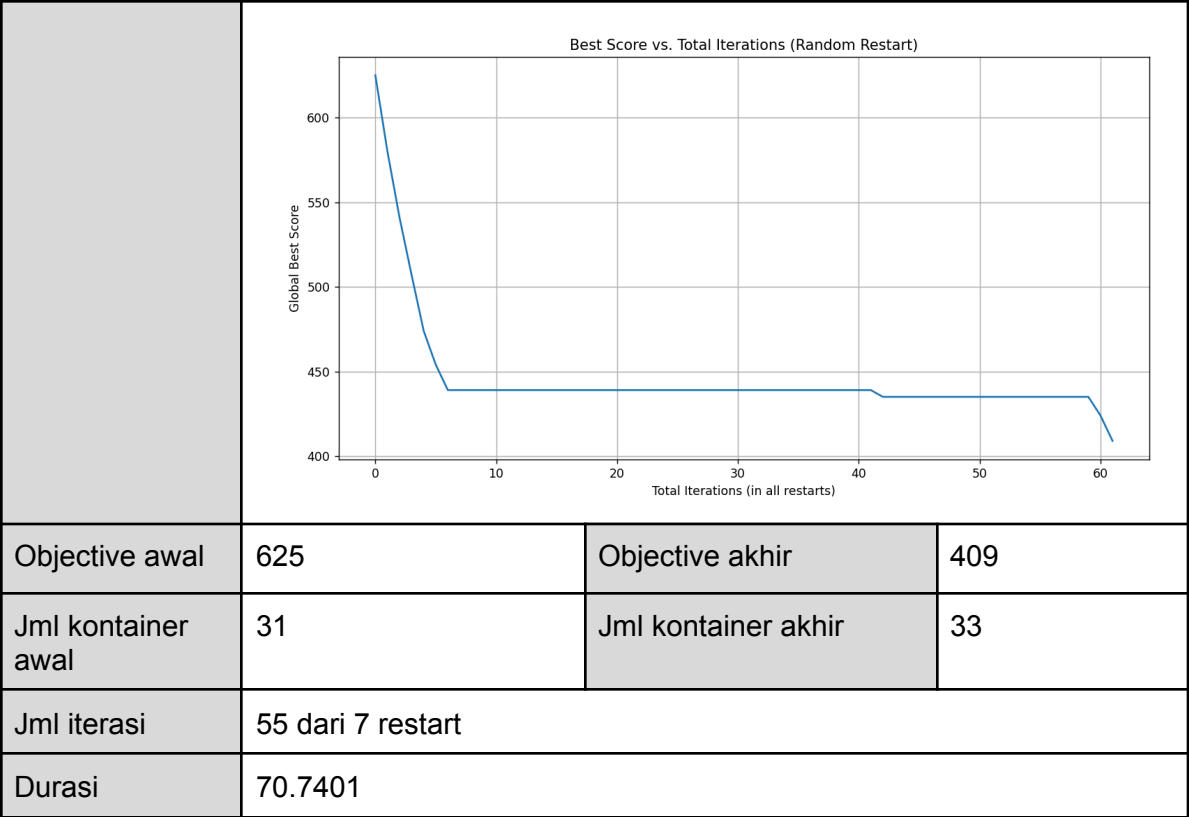
--- STATE AKHIR (HASIL GLOBAL TERBAIK) ---
Kontainer 1: Kontainer(total: 100/100, isi: [BRG036 (40), BRG037 (43), BRG031 (17)])
Kontainer 2: Kontainer(total: 99/100, isi: [BRG030 (21), BRG035 (36), BRG018 (20), BRG017 (22)])
Kontainer 3: Kontainer(total: 90/100, isi: [BRG026 (29), BRG048 (61)])
Kontainer 4: Kontainer(total: 93/100, isi: [BRG059 (74), BRG021 (19)])
Kontainer 5: Kontainer(total: 93/100, isi: [BRG001 (75), BRG019 (18)])
Kontainer 6: Kontainer(total: 99/100, isi: [BRG056 (71), BRG015 (28)])
Kontainer 7: Kontainer(total: 100/100, isi: [BRG046 (58), BRG022 (27), BRG020 (15)])
Kontainer 8: Kontainer(total: 100/100, isi: [BRG038 (44), BRG044 (56)])
Kontainer 9: Kontainer(total: 94/100, isi: [BRG004 (60), BRG034 (34)])
Kontainer 10: Kontainer(total: 100/100, isi: [BRG047 (59), BRG024 (41)])
Kontainer 11: Kontainer(total: 95/100, isi: [BRG042 (53), BRG010 (42)])
Kontainer 12: Kontainer(total: 98/100, isi: [BRG053 (67), BRG023 (31)])
Kontainer 13: Kontainer(total: 96/100, isi: [BRG058 (73), BRG027 (23)])
Kontainer 14: Kontainer(total: 98/100, isi: [BRG050 (63), BRG012 (35)])
Kontainer 15: Kontainer(total: 96/100, isi: [BRG028 (46), BRG007 (50)])
Kontainer 16: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)])
Kontainer 17: Kontainer(total: 99/100, isi: [BRG049 (62), BRG025 (37)])
Kontainer 18: Kontainer(total: 95/100, isi: [BRG008 (48), BRG039 (47)])
Kontainer 19: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)])
Kontainer 20: Kontainer(total: 97/100, isi: [BRG016 (25), BRG057 (72)])
Kontainer 21: Kontainer(total: 100/100, isi: [BRG006 (52), BRG032 (26), BRG017 (22)])
Kontainer 22: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)])
Kontainer 23: Kontainer(total: 100/100, isi: [BRG041 (51), BRG040 (49)])
Kontainer 24: Kontainer(total: 100/100, isi: [BRG055 (69), BRG023 (31)])
Kontainer 25: Kontainer(total: 97/100, isi: [BRG051 (64), BRG013 (33)])
Kontainer 26: Kontainer(total: 99/100, isi: [BRG043 (54), BRG009 (45)])
Kontainer 27: Kontainer(total: 94/100, isi: [BRG005 (55), BRG029 (39)])
Kontainer 28: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)])
Kontainer 29: Kontainer(total: 95/100, isi: [BRG045 (57), BRG011 (38)])
Kontainer 30: Kontainer(total: 100/100, isi: [BRG052 (66), BRG034 (34)])

Total Kontainer: 30
Skor Akhir Terbaik: 373
```



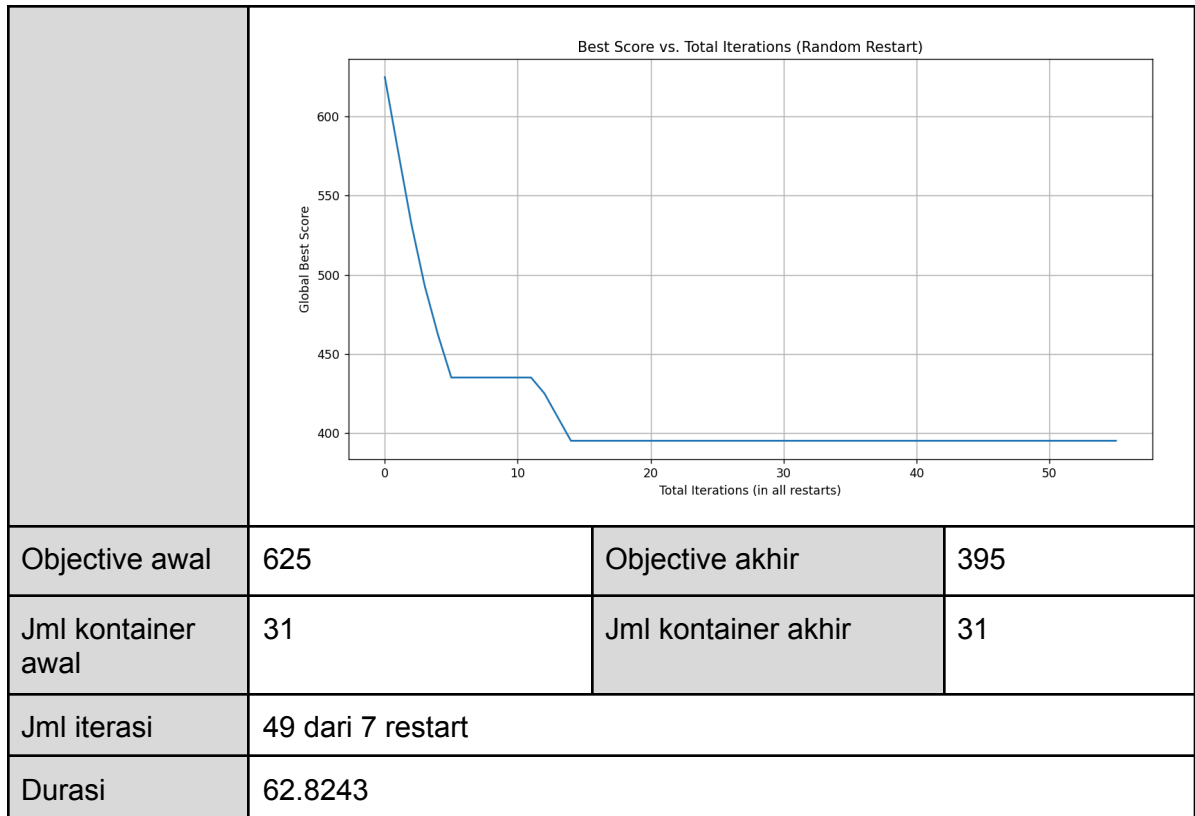
Eksperimen 2

State Awal	<pre>--- Restart 1/7 --- State Awal (dari Restart 1): Kontainer 1: Kontainer(total: 97/100, isi: [BRG042 (53), BRG038 (44)]) Kontainer 2: Kontainer(total: 95/100, isi: [BRG001 (75), BRG018 (20)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG057 (72), BRG015 (28)]) Kontainer 4: Kontainer(total: 99/100, isi: [BRG035 (36), BRG028 (46), BRG031 (17)]) Kontainer 5: Kontainer(total: 95/100, isi: [BRG014 (30), BRG003 (65)]) Kontainer 6: Kontainer(total: 86/100, isi: [BRG041 (51), BRG012 (35)]) Kontainer 7: Kontainer(total: 94/100, isi: [BRG058 (73), BRG030 (21)]) Kontainer 8: Kontainer(total: 94/100, isi: [BRG055 (69), BRG016 (25)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG049 (62), BRG025 (37)]) Kontainer 10: Kontainer(total: 96/100, isi: [BRG011 (38), BRG046 (58)]) Kontainer 11: Kontainer(total: 99/100, isi: [BRG006 (52), BRG033 (32), BRG020 (15)]) Kontainer 12: Kontainer(total: 89/100, isi: [BRG002 (70), BRG021 (19)]) Kontainer 13: Kontainer(total: 93/100, isi: [BRG024 (41), BRG034 (34), BRG019 (18)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG059 (74), BRG032 (26)]) Kontainer 15: Kontainer(total: 93/100, isi: [BRG008 (48), BRG009 (45)]) Kontainer 16: Kontainer(total: 80/100, isi: [BRG060 (80)]) Kontainer 17: Kontainer(total: 97/100, isi: [BRG045 (57), BRG036 (40)]) Kontainer 18: Kontainer(total: 92/100, isi: [BRG047 (59), BRG013 (33)]) Kontainer 19: Kontainer(total: 86/100, isi: [BRG050 (63), BRG027 (23)]) Kontainer 20: Kontainer(total: 98/100, isi: [BRG056 (71), BRG022 (27)]) Kontainer 21: Kontainer(total: 85/100, isi: [BRG043 (54), BRG023 (31)]) Kontainer 22: Kontainer(total: 97/100, isi: [BRG054 (68), BRG026 (29)]) Kontainer 23: Kontainer(total: 99/100, isi: [BRG004 (60), BRG029 (39)]) Kontainer 24: Kontainer(total: 90/100, isi: [BRG037 (43), BRG039 (47)]) Kontainer 25: Kontainer(total: 98/100, isi: [BRG010 (42), BRG044 (56)]) Kontainer 26: Kontainer(total: 86/100, isi: [BRG051 (64), BRG017 (22)]) Kontainer 27: Kontainer(total: 55/100, isi: [BRG005 (55)]) Kontainer 28: Kontainer(total: 99/100, isi: [BRG040 (49), BRG007 (50)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 31: Kontainer(total: 66/100, isi: [BRG052 (66)]) Skor Awal: 625.00</pre>
State Akhir	<pre>--- Random Restart Hill Climbing Selesai --- Durasi Total: 70.7401 detik Total Iterasi (di semua restart): 55 Best Score Ditemukan: 409 Rata-rata iterasi per restart: 7.86 --- STATE AKHIR (HASIL GLOBAL TERBAIK) --- Kontainer 1: Kontainer(total: 94/100, isi: [BRG022 (27), BRG014 (30), BRG031 (17), BRG018 (20)]) Kontainer 2: Kontainer(total: 97/100, isi: [BRG059 (74), BRG027 (23)]) Kontainer 3: Kontainer(total: 97/100, isi: [BRG040 (49), BRG008 (48)]) Kontainer 4: Kontainer(total: 100/100, isi: [BRG004 (60), BRG036 (40)]) Kontainer 5: Kontainer(total: 98/100, isi: [BRG005 (55), BRG037 (43)]) Kontainer 6: Kontainer(total: 96/100, isi: [BRG001 (75), BRG030 (21)]) Kontainer 7: Kontainer(total: 98/100, isi: [BRG053 (67), BRG023 (31)]) Kontainer 8: Kontainer(total: 97/100, isi: [BRG035 (36), BRG012 (35), BRG032 (26)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG029 (39), BRG010 (42), BRG019 (18)]) Kontainer 10: Kontainer(total: 92/100, isi: [BRG025 (37), BRG013 (33), BRG017 (22)]) Kontainer 11: Kontainer(total: 95/100, isi: [BRG011 (38), BRG045 (57)]) Kontainer 12: Kontainer(total: 99/100, isi: [BRG002 (70), BRG026 (29)]) Kontainer 13: Kontainer(total: 92/100, isi: [BRG046 (58), BRG034 (34)]) Kontainer 14: Kontainer(total: 87/100, isi: [BRG028 (46), BRG024 (41)]) Kontainer 15: Kontainer(total: 94/100, isi: [BRG052 (66), BRG015 (28)]) Kontainer 16: Kontainer(total: 98/100, isi: [BRG051 (64), BRG021 (19), BRG020 (15)]) Kontainer 17: Kontainer(total: 99/100, isi: [BRG047 (59), BRG016 (25), BRG020 (15)]) Kontainer 18: Kontainer(total: 100/100, isi: [BRG044 (56), BRG038 (44)]) Kontainer 19: Kontainer(total: 93/100, isi: [BRG048 (61), BRG033 (32)]) Kontainer 20: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Kontainer 21: Kontainer(total: 99/100, isi: [BRG039 (47), BRG006 (52)]) Kontainer 22: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)]) Kontainer 23: Kontainer(total: 100/100, isi: [BRG056 (71), BRG026 (29)]) Kontainer 24: Kontainer(total: 100/100, isi: [BRG050 (63), BRG025 (37)]) Kontainer 25: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)]) Kontainer 26: Kontainer(total: 98/100, isi: [BRG042 (53), BRG009 (45)]) Kontainer 27: Kontainer(total: 100/100, isi: [BRG057 (72), BRG015 (28)]) Kontainer 28: Kontainer(total: 100/100, isi: [BRG049 (62), BRG011 (38)]) Kontainer 29: Kontainer(total: 100/100, isi: [BRG041 (51), BRG040 (49)]) Kontainer 30: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 31: Kontainer(total: 100/100, isi: [BRG043 (54), BRG028 (46)]) Kontainer 32: Kontainer(total: 100/100, isi: [BRG055 (69), BRG023 (31)]) Kontainer 33: Kontainer(total: 99/100, isi: [BRG007 (50), BRG040 (49)]) Total Kontainer: 33 Skor Akhir Terbaik: 409 -----</pre>



Eksperimen 3

State Awal	<pre> --- Restart 1/7 --- State Awal (dari Restart 1): Kontainer 1: Kontainer(total: 92/100, isi: [BRG059 (74), BRG019 (18)]) Kontainer 2: Kontainer(total: 88/100, isi: [BRG035 (36), BRG006 (52)]) Kontainer 3: Kontainer(total: 99/100, isi: [BRG003 (65), BRG021 (19), BRG020 (15)]) Kontainer 4: Kontainer(total: 93/100, isi: [BRG044 (56), BRG025 (37)]) Kontainer 5: Kontainer(total: 98/100, isi: [BRG002 (70), BRG015 (28)]) Kontainer 6: Kontainer(total: 92/100, isi: [BRG039 (47), BRG009 (45)]) Kontainer 7: Kontainer(total: 91/100, isi: [BRG045 (57), BRG034 (34)]) Kontainer 8: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)]) Kontainer 9: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)]) Kontainer 10: Kontainer(total: 98/100, isi: [BRG029 (39), BRG047 (59)]) Kontainer 11: Kontainer(total: 95/100, isi: [BRG013 (33), BRG024 (41), BRG030 (21)]) Kontainer 12: Kontainer(total: 98/100, isi: [BRG050 (63), BRG012 (35)]) Kontainer 13: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG041 (51), BRG040 (49)]) Kontainer 15: Kontainer(total: 94/100, isi: [BRG051 (64), BRG014 (30)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG049 (62), BRG026 (29)]) Kontainer 17: Kontainer(total: 87/100, isi: [BRG038 (44), BRG037 (43)]) Kontainer 18: Kontainer(total: 100/100, isi: [BRG046 (58), BRG010 (42)]) Kontainer 19: Kontainer(total: 99/100, isi: [BRG057 (72), BRG022 (27)]) Kontainer 20: Kontainer(total: 98/100, isi: [BRG007 (50), BRG008 (48)]) Kontainer 21: Kontainer(total: 97/100, isi: [BRG052 (66), BRG023 (31)]) Kontainer 22: Kontainer(total: 98/100, isi: [BRG001 (75), BRG027 (23)]) Kontainer 23: Kontainer(total: 93/100, isi: [BRG053 (67), BRG032 (26)]) Kontainer 24: Kontainer(total: 93/100, isi: [BRG056 (71), BRG017 (22)]) Kontainer 25: Kontainer(total: 86/100, isi: [BRG028 (46), BRG036 (40)]) Kontainer 26: Kontainer(total: 97/100, isi: [BRG005 (55), BRG016 (25), BRG031 (17)]) Kontainer 27: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 28: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 29: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 30: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 31: Kontainer(total: 61/100, isi: [BRG048 (61)]) Skor Awal: 625.00 </pre>
State Akhir	<pre> --- Random Restart Hill Climbing Selesai --- Durasi Total: 62.8243 detik Total Iterasi (di semua restart): 49 Best Score Ditemukan: 395 Rata-rata iterasi per restart: 7.00 --- STATE AKHIR (HASIL GLOBAL TERBAIK) --- Kontainer 1: Kontainer(total: 88/100, isi: [BRG034 (34), BRG035 (36), BRG019 (18)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG049 (62), BRG020 (15), BRG027 (23)]) Kontainer 3: Kontainer(total: 98/100, isi: [BRG043 (54), BRG038 (44)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG041 (51), BRG015 (28), BRG021 (19)]) Kontainer 5: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG046 (58), BRG017 (22), BRG018 (20)]) Kontainer 7: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 8: Kontainer(total: 99/100, isi: [BRG053 (67), BRG031 (17), BRG020 (15)]) Kontainer 9: Kontainer(total: 100/100, isi: [BRG060 (80), BRG018 (20)]) Kontainer 10: Kontainer(total: 96/100, isi: [BRG052 (66), BRG014 (30)]) Kontainer 11: Kontainer(total: 92/100, isi: [BRG039 (47), BRG009 (45)]) Kontainer 12: Kontainer(total: 100/100, isi: [BRG059 (74), BRG032 (26)]) Kontainer 13: Kontainer(total: 94/100, isi: [BRG008 (48), BRG028 (46)]) Kontainer 14: Kontainer(total: 95/100, isi: [BRG045 (57), BRG011 (38)]) Kontainer 15: Kontainer(total: 94/100, isi: [BRG003 (65), BRG026 (29)]) Kontainer 16: Kontainer(total: 99/100, isi: [BRG050 (63), BRG030 (21), BRG020 (15)]) Kontainer 17: Kontainer(total: 96/100, isi: [BRG042 (53), BRG037 (43)]) Kontainer 18: Kontainer(total: 97/100, isi: [BRG002 (70), BRG022 (27)]) Kontainer 19: Kontainer(total: 100/100, isi: [BRG054 (68), BRG033 (32)]) Kontainer 20: Kontainer(total: 99/100, isi: [BRG051 (64), BRG012 (35)]) Kontainer 21: Kontainer(total: 98/100, isi: [BRG047 (59), BRG029 (39)]) Kontainer 22: Kontainer(total: 97/100, isi: [BRG004 (60), BRG025 (37)]) Kontainer 23: Kontainer(total: 100/100, isi: [BRG055 (69), BRG023 (31)]) Kontainer 24: Kontainer(total: 100/100, isi: [BRG056 (71), BRG026 (29)]) Kontainer 25: Kontainer(total: 94/100, isi: [BRG048 (61), BRG013 (33)]) Kontainer 26: Kontainer(total: 93/100, isi: [BRG006 (52), BRG024 (41)]) Kontainer 27: Kontainer(total: 96/100, isi: [BRG044 (56), BRG036 (40)]) Kontainer 28: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 29: Kontainer(total: 100/100, isi: [BRG057 (72), BRG015 (28)]) Kontainer 30: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 31: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Total Kontainer: 31 Skor Akhir Terbaik: 395 </pre>



d. Analisis

Hasil dari Random-Restart merupakan non-deterministik. Algoritma ini selalu mengembalikan hasil yang baik, yaitu 373, 409, dan 395. Hasil ini lebih rendah dari local optimum (400) yang ditemukan oleh Steepest Ascent. Algoritma ini berhasil mendapatkan skor 373 dan mengurangi jumlah kontainer menjadi 30 dari yang awalnya 31. Meskipun solusi ini sangat bagus, konsekuensinya adalah durasi algoritma yang sangat panjang.

2.5.5 Simulated Annealing

Simulated Annealing adalah algoritma yang dapat menerima dan berpindah ke state neighbour yang lebih buruk jika kondisinya mendukung. Untuk bisa menerima state yang lebih buruk, dia harus memiliki probabilitas yang tinggi. Probabilitas tersebut ditentukan melalui rumus $P = e^{-\Delta E/T}$, dengan ΔE adalah seberapa buruk probabilitas baru dan T adalah temperatur sistem.

Di awal, algoritma akan lebih sering menerima state buruk karena temperaturnya masih tinggi. Semakin lama algoritma berjalan, maka temperature akan semakin menurun dan algoritma akan semakin jarang menerima state yang lebih buruk.

a. Source code Simulated Annealing

```
import math
import random
import time
import matplotlib.pyplot as plt
from bin.state import State
from bin.entity.kontainer import Kontainer
from bin.objective_function import objective_function as evaluate

class SimulatedAnnealing:
    def __init__(self, initial_state, kapasitas):
        self.kapasitas = kapasitas
        self.n_iterations = 0
        self.temp_awal = 0.0
        self.alpha = 0.0
        self.current_state = initial_state
        self.current_eval = evaluate(self.current_state, self.kapasitas)
        self.best_state = self.current_state
        self.best_eval = self.current_eval
        self.scores_history = [self.best_eval]
        self.temp_history = []
        self.prob_history = []
        self.current_scores_history = [self.current_eval]

    def get_random_neighbour(self, state):
        new_state = state.clone()
        kontainers = new_state.kontainer_list

        non_empty_kontainers = [k for k in kontainers if k.total_isi() > 0]
        if not non_empty_kontainers:
            return new_state

        move_type = 'move'
        if len(non_empty_kontainers) >= 2:
            move_type = random.choice(['move', 'swap'])

        if move_type == 'move':
            src_k = random.choice(non_empty_kontainers)
            barang = random.choice(src_k.isi)
            src_k.hapus_barang(barang)

            if random.random() < 0.1 or len(kontainers) == 1:
                dest_k = Kontainer(self.kapasitas)
                new_state.kontainer_list.append(dest_k)
            else:
                dest_k = random.choice([k for k in kontainers if k is not src_k])

            if not dest_k.tambah_barang(barang):
                src_k.tambah_barang(barang)
                if dest_k.total_isi() == 0 and dest_k not in state.kontainer_list:
                    new_state.kontainer_list.remove(dest_k)

        elif move_type == 'swap':
```

```

        k1, k2 = random.sample(non_empty_containers, 2)
        b1 = random.choice(k1.isi)
        b2 = random.choice(k2.isi)

        if (k1.total_isi() - b1.ukuran + b2.ukuran <= self.kapasitas) and
(k2.total_isi() - b2.ukuran + b1.ukuran <= self.kapasitas):
            k1.hapus_barang(b1)
            k2.hapus_barang(b2)
            k1.tambah_barang(b2)
            k2.tambah_barang(b1)

    new_state.kontainer_list = [k for k in new_state.kontainer_list if k.total_isi() >
0]

    return new_state

def _plot_scores(self):
    plt.figure(figsize=(12, 7))

    # Plot Best Score
    plt.plot(self.scores_history,
             label="Best Score",
             color="blue",
             linewidth=2)

    # Plot Current Score
    plt.plot(self.current_scores_history,
             label="Current Score",
             color="red",
             alpha=0.5,
             linewidth=1)

    plt.title("Best Score vs Current Score")
    plt.xlabel("Iteration")
    plt.ylabel("Score")
    plt.legend()
    plt.grid(True)
    print("Menampilkan Plot Skor (Best vs Current)...")
    plt.show()

def run(self):
    print("\n--- STATE AWAL ---")
    print(self.current_state)
    print(f"Skor Awal: {self.current_eval}")
    print("-----\n")

    self.temp_awal = float(input("Masukkan Temperatur Awal (misal: 1000): "))
    self.alpha = float(input("Masukkan Cooling Rate/Alpha (misal: 0.99): "))
    self.n_iterations = int(input("Masukkan Jumlah Iterasi (misal: 10000): "))

    print("--- Memulai Simulated Annealing ---")
    start_time = time.time()

```

```

t = self.temp_awal
self.temp_history.append(t)

for i in range(self.n_iterations):
    candidate = self.get_random_neighbour(self.current_state)
    candidate_eval = evaluate(candidate, self.kapasitas)
    delta_e = candidate_eval - self.current_eval

    if delta_e < 0:
        self.current_state, self.current_eval = candidate, candidate_eval

        if candidate_eval < self.best_eval:
            self.best_state, self.best_eval = candidate, candidate_eval
        else:
            if t > 0.000001:
                try:
                    prob = math.exp(-delta_e / t)

                    if random.random() < prob:
                        self.current_state, self.current_eval = candidate,
candidate_eval
                        self.prob_history.append(prob)
                except OverflowError:
                    prob = 0
            t *= self.alpha
            self.scores_history.append(self.best_eval)
            self.current_scores_history.append(self.current_eval)
            self.temp_history.append(t)

    print(f"Iterasi: {i:>5} / {self.n_iterations}, Temp {t:8.3f}, Best
{self.best_eval:8.2f}, Current {self.current_eval:8.2f}")

end_time = time.time()
print("--- Simulated Annealing Selesai!!! ---")
print(f"Durasi: {end_time - start_time:.4f} detik.")
print(f"Best score: {self.best_eval}")

print("\n--- STATE AKHIR (HASIL) ---")
print(self.best_state)
print(f"Total Kontainer: {self.best_state.total_kontainer()}")
print(f"Skor Akhir Terbaik: {self.best_eval}")
print("-----\n")

print("Tutup window plot agar program berjalan lagi.")
self._plot_scores()

return

```

b. Fungsi dan Prosedur

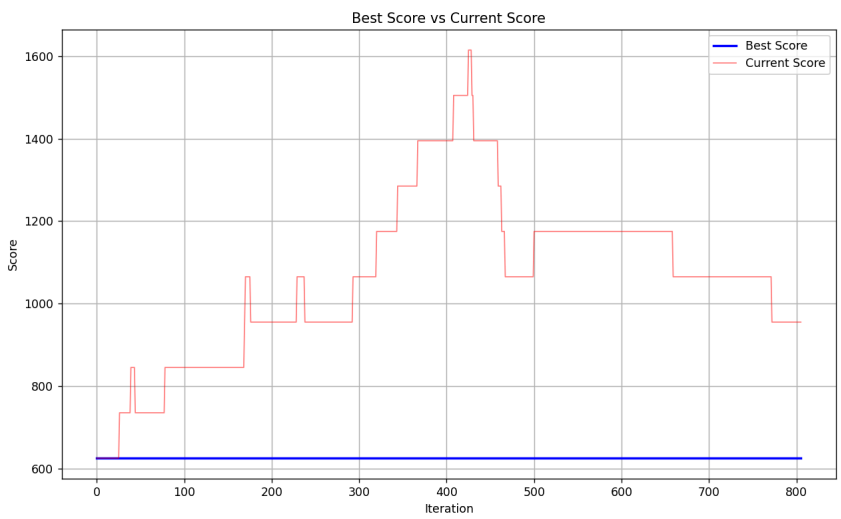
Nama fungsi	get_random_neighbour(self, state)
-------------	-----------------------------------

Deskripsi	Menghasilkan satu tetangga acak.
Output	Fungsi ini secara acak memilih 'move' atau 'swap' yang diaplikasikan untuk membuat state kandidat baru. Fungsi ini juga menangani validasi kapasitas kontainer.
Source code	Bagian bertanda kuning pada source code Simulated Annealing.

Nama fungsi	_plot_scores(self)
Deskripsi	Fungsi untuk membuat visualisasi hasil menggunakan matplotlib, menunjukkan skor terbaik (scores) dan skor saat ini (current score).
Output	Menampilkan window plot.
Source code	Bagian bertanda hijau pada source code Simulated Annealing.

c. Hasil Eksperimen

Eksperimen 1	
State Awal	<pre> --- STATE AWAL --- Kontainer 1: Kontainer(total: 99/100, isi: [BRG049 (62), BRG018 (20), BRG031 (17)]) Kontainer 2: Kontainer(total: 98/100, isi: [BRG060 (80), BRG019 (18)]) Kontainer 3: Kontainer(total: 93/100, isi: [BRG056 (71), BRG017 (22)]) Kontainer 4: Kontainer(total: 97/100, isi: [BRG034 (34), BRG050 (63)]) Kontainer 5: Kontainer(total: 93/100, isi: [BRG039 (47), BRG028 (46)]) Kontainer 6: Kontainer(total: 91/100, isi: [BRG047 (59), BRG033 (32)]) Kontainer 7: Kontainer(total: 87/100, isi: [BRG007 (50), BRG025 (37)]) Kontainer 8: Kontainer(total: 91/100, isi: [BRG057 (72), BRG021 (19)]) Kontainer 9: Kontainer(total: 87/100, isi: [BRG043 (54), BRG013 (33)]) Kontainer 10: Kontainer(total: 100/100, isi: [BRG035 (36), BRG026 (29), BRG012 (35)]) Kontainer 11: Kontainer(total: 92/100, isi: [BRG037 (43), BRG040 (49)]) Kontainer 12: Kontainer(total: 90/100, isi: [BRG041 (51), BRG029 (39)]) Kontainer 13: Kontainer(total: 90/100, isi: [BRG003 (65), BRG016 (25)]) Kontainer 14: Kontainer(total: 89/100, isi: [BRG009 (45), BRG038 (44)]) Kontainer 15: Kontainer(total: 96/100, isi: [BRG002 (70), BRG032 (26)]) Kontainer 16: Kontainer(total: 98/100, isi: [BRG054 (68), BRG014 (30)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG010 (42), BRG046 (58)]) Kontainer 18: Kontainer(total: 92/100, isi: [BRG048 (61), BRG023 (31)]) Kontainer 19: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 20: Kontainer(total: 97/100, isi: [BRG042 (53), BRG030 (21), BRG027 (23)]) Kontainer 21: Kontainer(total: 81/100, isi: [BRG052 (66), BRG020 (15)]) Kontainer 22: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 23: Kontainer(total: 95/100, isi: [BRG005 (55), BRG036 (40)]) Kontainer 24: Kontainer(total: 92/100, isi: [BRG051 (64), BRG015 (28)]) Kontainer 25: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 26: Kontainer(total: 94/100, isi: [BRG053 (67), BRG022 (27)]) Kontainer 27: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 28: Kontainer(total: 97/100, isi: [BRG044 (56), BRG024 (41)]) Kontainer 29: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 30: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 31: Kontainer(total: 75/100, isi: [BRG001 (75)]) Skor Awal: 625 </pre>

State Akhir	<pre> --- Simulated Annealing Selesai!!! --- Durasi: 0.7716 detik. Best score: 625 --- STATE AKHIR (HASIL) --- Kontainer 1: Kontainer(total: 99/100, isi: [BRG049 (62), BRG018 (20), BRG031 (17)]) Kontainer 2: Kontainer(total: 98/100, isi: [BRG060 (80), BRG019 (18)]) Kontainer 3: Kontainer(total: 93/100, isi: [BRG056 (71), BRG017 (22)]) Kontainer 4: Kontainer(total: 97/100, isi: [BRG034 (34), BRG050 (63)]) Kontainer 5: Kontainer(total: 93/100, isi: [BRG039 (47), BRG028 (46)]) Kontainer 6: Kontainer(total: 91/100, isi: [BRG047 (59), BRG033 (32)]) Kontainer 7: Kontainer(total: 87/100, isi: [BRG007 (50), BRG025 (37)]) Kontainer 8: Kontainer(total: 91/100, isi: [BRG057 (72), BRG021 (19)]) Kontainer 9: Kontainer(total: 87/100, isi: [BRG043 (54), BRG013 (33)]) Kontainer 10: Kontainer(total: 100/100, isi: [BRG035 (36), BRG026 (29), BRG012 (35)]) Kontainer 11: Kontainer(total: 92/100, isi: [BRG037 (43), BRG040 (49)]) Kontainer 12: Kontainer(total: 90/100, isi: [BRG041 (51), BRG029 (39)]) Kontainer 13: Kontainer(total: 90/100, isi: [BRG003 (65), BRG016 (25)]) Kontainer 14: Kontainer(total: 89/100, isi: [BRG009 (45), BRG038 (44)]) Kontainer 15: Kontainer(total: 96/100, isi: [BRG002 (70), BRG032 (26)]) Kontainer 16: Kontainer(total: 98/100, isi: [BRG054 (68), BRG014 (30)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG010 (42), BRG046 (58)]) Kontainer 18: Kontainer(total: 92/100, isi: [BRG048 (61), BRG023 (31)]) Kontainer 19: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 20: Kontainer(total: 97/100, isi: [BRG042 (53), BRG030 (21), BRG027 (23)]) Kontainer 21: Kontainer(total: 81/100, isi: [BRG052 (66), BRG020 (15)]) Kontainer 22: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 23: Kontainer(total: 95/100, isi: [BRG005 (55), BRG036 (40)]) Kontainer 24: Kontainer(total: 92/100, isi: [BRG051 (64), BRG015 (28)]) Kontainer 25: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 26: Kontainer(total: 94/100, isi: [BRG053 (67), BRG022 (27)]) Kontainer 27: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 28: Kontainer(total: 97/100, isi: [BRG044 (56), BRG024 (41)]) Kontainer 29: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 30: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 31: Kontainer(total: 75/100, isi: [BRG001 (75)]) Total Kontainer: 31 Skor Akhir Terbaik: 625 </pre> 		
Objective awal	625	Objective akhir	625
Jml kontainer awal	31	Jml kontainer akhir	31
Jml iterasi	805		
Durasi	0.7716 detik		

Eksperimen 2

State Awal

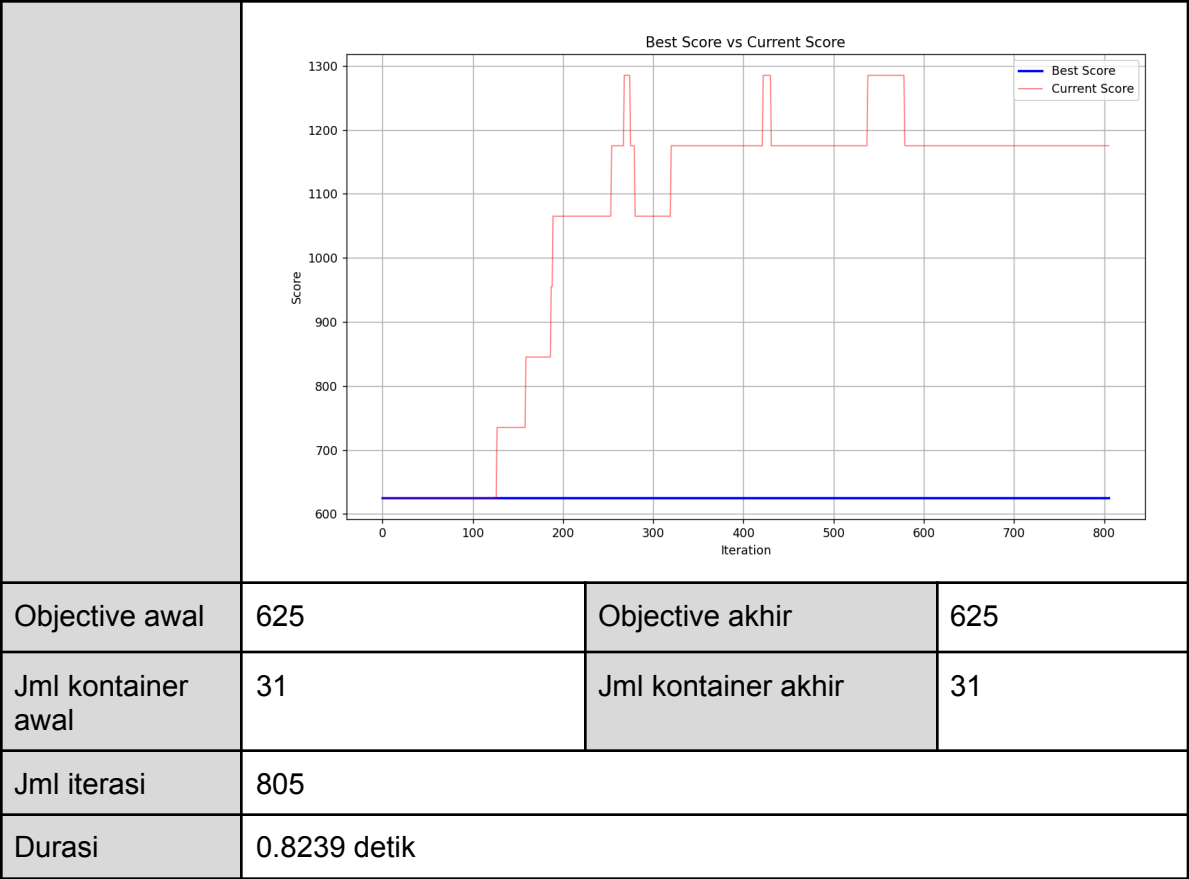
```
--- STATE AWAL ---
Kontainer 1: Kontainer(total: 86/100, isi: [BRG055 (69), BRG031 (17)])
Kontainer 2: Kontainer(total: 99/100, isi: [BRG012 (35), BRG051 (64)])
Kontainer 3: Kontainer(total: 88/100, isi: [BRG020 (15), BRG017 (22), BRG021 (19), BRG033 (32)])
Kontainer 4: Kontainer(total: 85/100, isi: [BRG032 (26), BRG023 (31), BRG015 (28)])
Kontainer 5: Kontainer(total: 95/100, isi: [BRG001 (75), BRG018 (20)])
Kontainer 6: Kontainer(total: 86/100, isi: [BRG007 (50), BRG035 (36)])
Kontainer 7: Kontainer(total: 100/100, isi: [BRG049 (62), BRG011 (38)])
Kontainer 8: Kontainer(total: 94/100, isi: [BRG048 (61), BRG013 (33)])
Kontainer 9: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)])
Kontainer 10: Kontainer(total: 99/100, isi: [BRG046 (58), BRG024 (41)])
Kontainer 11: Kontainer(total: 100/100, isi: [BRG052 (66), BRG034 (34)])
Kontainer 12: Kontainer(total: 97/100, isi: [BRG004 (60), BRG025 (37)])
Kontainer 13: Kontainer(total: 98/100, isi: [BRG060 (80), BRG019 (18)])
Kontainer 14: Kontainer(total: 100/100, isi: [BRG043 (54), BRG028 (46)])
Kontainer 15: Kontainer(total: 95/100, isi: [BRG006 (52), BRG037 (43)])
Kontainer 16: Kontainer(total: 91/100, isi: [BRG054 (68), BRG027 (23)])
Kontainer 17: Kontainer(total: 82/100, isi: [BRG042 (53), BRG026 (29)])
Kontainer 18: Kontainer(total: 93/100, isi: [BRG057 (72), BRG030 (21)])
Kontainer 19: Kontainer(total: 95/100, isi: [BRG002 (70), BRG016 (25)])
Kontainer 20: Kontainer(total: 97/100, isi: [BRG040 (49), BRG008 (48)])
Kontainer 21: Kontainer(total: 95/100, isi: [BRG003 (65), BRG014 (30)])
Kontainer 22: Kontainer(total: 97/100, isi: [BRG045 (57), BRG036 (40)])
Kontainer 23: Kontainer(total: 74/100, isi: [BRG059 (74)])
Kontainer 24: Kontainer(total: 63/100, isi: [BRG050 (63)])
Kontainer 25: Kontainer(total: 67/100, isi: [BRG053 (67)])
Kontainer 26: Kontainer(total: 95/100, isi: [BRG038 (44), BRG041 (51)])
Kontainer 27: Kontainer(total: 71/100, isi: [BRG056 (71)])
Kontainer 28: Kontainer(total: 100/100, isi: [BRG009 (45), BRG005 (55)])
Kontainer 29: Kontainer(total: 95/100, isi: [BRG044 (56), BRG029 (39)])
Kontainer 30: Kontainer(total: 59/100, isi: [BRG047 (59)])
Kontainer 31: Kontainer(total: 89/100, isi: [BRG039 (47), BRG010 (42)])

Skor Awal: 625
-----
```

State Akhir

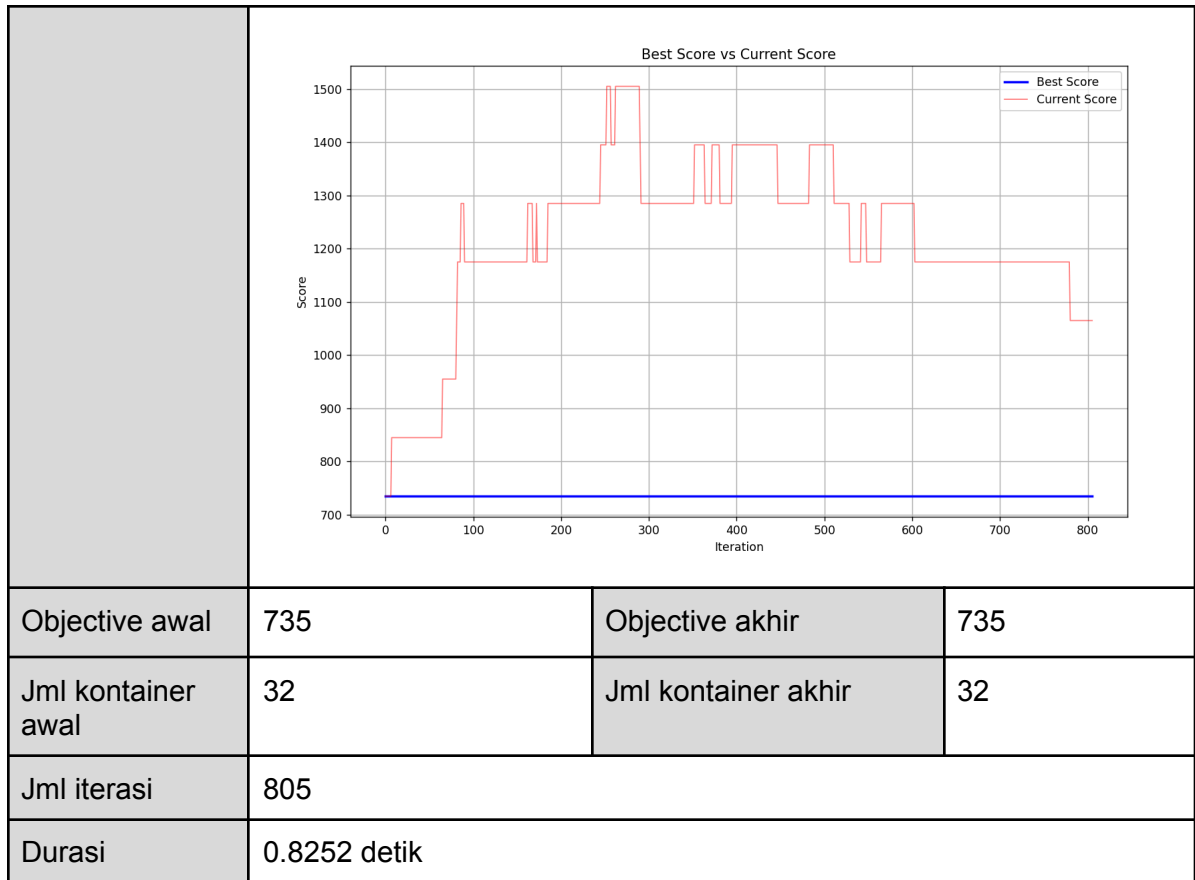
```
--- STATE AKHIR (HASIL) ---
Kontainer 1: Kontainer(total: 86/100, isi: [BRG055 (69), BRG031 (17)])
Kontainer 2: Kontainer(total: 99/100, isi: [BRG012 (35), BRG051 (64)])
Kontainer 3: Kontainer(total: 88/100, isi: [BRG020 (15), BRG017 (22), BRG021 (19), BRG033 (32)])
Kontainer 4: Kontainer(total: 85/100, isi: [BRG032 (26), BRG023 (31), BRG015 (28)])
Kontainer 5: Kontainer(total: 95/100, isi: [BRG001 (75), BRG018 (20)])
Kontainer 6: Kontainer(total: 86/100, isi: [BRG007 (50), BRG035 (36)])
Kontainer 7: Kontainer(total: 100/100, isi: [BRG049 (62), BRG011 (38)])
Kontainer 8: Kontainer(total: 94/100, isi: [BRG048 (61), BRG013 (33)])
Kontainer 9: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)])
Kontainer 10: Kontainer(total: 99/100, isi: [BRG046 (58), BRG024 (41)])
Kontainer 11: Kontainer(total: 100/100, isi: [BRG052 (66), BRG034 (34)])
Kontainer 12: Kontainer(total: 97/100, isi: [BRG004 (60), BRG025 (37)])
Kontainer 13: Kontainer(total: 98/100, isi: [BRG060 (80), BRG019 (18)])
Kontainer 14: Kontainer(total: 100/100, isi: [BRG043 (54), BRG028 (46)])
Kontainer 15: Kontainer(total: 95/100, isi: [BRG006 (52), BRG037 (43)])
Kontainer 16: Kontainer(total: 91/100, isi: [BRG054 (68), BRG027 (23)])
Kontainer 17: Kontainer(total: 82/100, isi: [BRG042 (53), BRG026 (29)])
Kontainer 18: Kontainer(total: 93/100, isi: [BRG057 (72), BRG030 (21)])
Kontainer 19: Kontainer(total: 95/100, isi: [BRG002 (70), BRG016 (25)])
Kontainer 20: Kontainer(total: 97/100, isi: [BRG040 (49), BRG008 (48)])
Kontainer 21: Kontainer(total: 95/100, isi: [BRG003 (65), BRG014 (30)])
Kontainer 22: Kontainer(total: 97/100, isi: [BRG045 (57), BRG036 (40)])
Kontainer 23: Kontainer(total: 74/100, isi: [BRG059 (74)])
Kontainer 24: Kontainer(total: 63/100, isi: [BRG050 (63)])
Kontainer 25: Kontainer(total: 67/100, isi: [BRG053 (67)])
Kontainer 26: Kontainer(total: 95/100, isi: [BRG038 (44), BRG041 (51)])
Kontainer 27: Kontainer(total: 71/100, isi: [BRG056 (71)])
Kontainer 28: Kontainer(total: 100/100, isi: [BRG009 (45), BRG005 (55)])
Kontainer 29: Kontainer(total: 95/100, isi: [BRG044 (56), BRG029 (39)])
Kontainer 30: Kontainer(total: 59/100, isi: [BRG047 (59)])
Kontainer 31: Kontainer(total: 89/100, isi: [BRG039 (47), BRG010 (42)])

Total Kontainer: 31
Skor Akhir Terbaik: 625
-----
```



Eksperimen 3

State Awal	<pre> --- STATE AWAL --- Kontainer 1: Kontainer(total: 89/100, isi: [BRG008 (48), BRG018 (20), BRG030 (21)]) Kontainer 2: Kontainer(total: 86/100, isi: [BRG010 (42), BRG016 (25), BRG021 (19)]) Kontainer 3: Kontainer(total: 87/100, isi: [BRG041 (51), BRG035 (36)]) Kontainer 4: Kontainer(total: 100/100, isi: [BRG044 (56), BRG032 (26), BRG019 (18)]) Kontainer 5: Kontainer(total: 91/100, isi: [BRG059 (74), BRG031 (17)]) Kontainer 6: Kontainer(total: 91/100, isi: [BRG047 (59), BRG033 (32)]) Kontainer 7: Kontainer(total: 95/100, isi: [BRG046 (58), BRG025 (37)]) Kontainer 8: Kontainer(total: 95/100, isi: [BRG051 (64), BRG023 (31)]) Kontainer 9: Kontainer(total: 89/100, isi: [BRG038 (44), BRG009 (45)]) Kontainer 10: Kontainer(total: 97/100, isi: [BRG001 (75), BRG017 (22)]) Kontainer 11: Kontainer(total: 95/100, isi: [BRG052 (66), BRG026 (29)]) Kontainer 12: Kontainer(total: 92/100, isi: [BRG037 (43), BRG040 (49)]) Kontainer 13: Kontainer(total: 91/100, isi: [BRG042 (53), BRG011 (38)]) Kontainer 14: Kontainer(total: 96/100, isi: [BRG005 (55), BRG024 (41)]) Kontainer 15: Kontainer(total: 92/100, isi: [BRG055 (69), BRG027 (23)]) Kontainer 16: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Kontainer 17: Kontainer(total: 99/100, isi: [BRG056 (71), BRG015 (28)]) Kontainer 18: Kontainer(total: 96/100, isi: [BRG049 (62), BRG034 (34)]) Kontainer 19: Kontainer(total: 95/100, isi: [BRG060 (80), BRG020 (15)]) Kontainer 20: Kontainer(total: 89/100, isi: [BRG007 (50), BRG029 (39)]) Kontainer 21: Kontainer(total: 92/100, isi: [BRG036 (40), BRG006 (52)]) Kontainer 22: Kontainer(total: 98/100, isi: [BRG054 (68), BRG014 (30)]) Kontainer 23: Kontainer(total: 98/100, isi: [BRG003 (65), BRG013 (33)]) Kontainer 24: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 25: Kontainer(total: 95/100, isi: [BRG004 (60), BRG012 (35)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 100/100, isi: [BRG043 (54), BRG028 (46)]) Kontainer 28: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 29: Kontainer(total: 70/100, isi: [BRG002 (70)]) Kontainer 30: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 31: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 32: Kontainer(total: 47/100, isi: [BRG039 (47)]) Skor Awal: 735 ----- </pre>
State Akhir	<pre> --- STATE AKHIR (HASIL) --- Kontainer 1: Kontainer(total: 89/100, isi: [BRG008 (48), BRG018 (20), BRG030 (21)]) Kontainer 2: Kontainer(total: 86/100, isi: [BRG010 (42), BRG016 (25), BRG021 (19)]) Kontainer 3: Kontainer(total: 87/100, isi: [BRG041 (51), BRG035 (36)]) Kontainer 4: Kontainer(total: 100/100, isi: [BRG044 (56), BRG032 (26), BRG019 (18)]) Kontainer 5: Kontainer(total: 91/100, isi: [BRG059 (74), BRG031 (17)]) Kontainer 6: Kontainer(total: 91/100, isi: [BRG047 (59), BRG033 (32)]) Kontainer 7: Kontainer(total: 95/100, isi: [BRG046 (58), BRG025 (37)]) Kontainer 8: Kontainer(total: 95/100, isi: [BRG051 (64), BRG023 (31)]) Kontainer 9: Kontainer(total: 89/100, isi: [BRG038 (44), BRG009 (45)]) Kontainer 10: Kontainer(total: 97/100, isi: [BRG001 (75), BRG017 (22)]) Kontainer 11: Kontainer(total: 95/100, isi: [BRG052 (66), BRG026 (29)]) Kontainer 12: Kontainer(total: 92/100, isi: [BRG037 (43), BRG040 (49)]) Kontainer 13: Kontainer(total: 91/100, isi: [BRG042 (53), BRG011 (38)]) Kontainer 14: Kontainer(total: 96/100, isi: [BRG005 (55), BRG024 (41)]) Kontainer 15: Kontainer(total: 92/100, isi: [BRG055 (69), BRG027 (23)]) Kontainer 16: Kontainer(total: 100/100, isi: [BRG058 (73), BRG022 (27)]) Kontainer 17: Kontainer(total: 99/100, isi: [BRG056 (71), BRG015 (28)]) Kontainer 18: Kontainer(total: 96/100, isi: [BRG049 (62), BRG034 (34)]) Kontainer 19: Kontainer(total: 95/100, isi: [BRG060 (80), BRG020 (15)]) Kontainer 20: Kontainer(total: 89/100, isi: [BRG007 (50), BRG029 (39)]) Kontainer 21: Kontainer(total: 92/100, isi: [BRG036 (40), BRG006 (52)]) Kontainer 22: Kontainer(total: 98/100, isi: [BRG054 (68), BRG014 (30)]) Kontainer 23: Kontainer(total: 98/100, isi: [BRG003 (65), BRG013 (33)]) Kontainer 24: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 25: Kontainer(total: 95/100, isi: [BRG004 (60), BRG012 (35)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 100/100, isi: [BRG043 (54), BRG028 (46)]) Kontainer 28: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 29: Kontainer(total: 70/100, isi: [BRG002 (70)]) Kontainer 30: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 31: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 32: Kontainer(total: 47/100, isi: [BRG039 (47)]) Total Kontainer: 32 Skor Akhir Terbaik: 735 ----- </pre>



d. Analisis

Dalam ketiga eksperimen, algoritma Simulated Annealing gagal mencari solusi yang lebih optimal, ditunjukkan kepada objective akhir tidak bisa berganti dari state awal. Hal ini juga dilihat dari plot grafik, yang menunjukkan garis best score tidak berubah, sedangkan garis current score selalu naik turun.

2.5.6 Genetic Algorithm

Genetic Algorithm merupakan metode pencarian yang bekerja dengan meniru mekanisme evolusi biologis, yaitu seleksi alam, reproduksi, dan mutasi. Setiap solusi direpresentasikan sebagai kromosom dalam suatu populasi. Selama proses iteratif, individu dengan nilai fitness terbaik dipilih untuk menghasilkan keturunan melalui operasi crossover dan mutasi. Dengan cara ini, GA melakukan pencarian solusi secara stokastik namun terarah menuju solusi yang lebih optimal.

a. Source code Genetic Algorithm

```
import random
```

```

import time
import matplotlib.pyplot as plt
from bin.objective_function import objective_function
from bin.state import State

class GeneticAlgorithm:
    def __init__(self, initial_state, kapasitas):
        self.initial_state = initial_state
        self.kapasitas = kapasitas

        print("\n=== Input Parameter Genetic Algorithm ===")
        self.population_size = int(input("Masukkan jumlah populasi: "))
        self.max_iterasi = int(input("Masukkan banyak iterasi (generasi): "))
        self.mutation_rate = float(input("Masukkan probabilitas mutasi (contoh 0.1): "))

        # ambil jumlah barang
        self.barang_list = []
        for k in initial_state.kontainer_list:
            for b in k.isi:
                self.barang_list.append(b)
        self.n_barang = len(self.barang_list)

        # banyak kontainer awal = batas max kontainer
        self.max_kontainer = len(initial_state.kontainer_list)

        self.population = []
        self.best_chrom = None
        self.best_value = float("inf")
        self.best_history = []
        self.avg_history = []

    def initialize_population(self):
        # hasil awal pakai representasi kromosom
        init_chrom = []
        mapping = {}
        for idx_k, k in enumerate(self.initial_state.kontainer_list):
            for b in k.isi:
                mapping[b.id] = idx_k

        for b in self.barang_list:
            init_chrom.append(mapping[b.id])

        self.population.append(init_chrom)

        while len(self.population) < self.population_size:
            chrom = [random.randint(0, self.max_kontainer - 1) for _ in range(self.n_barang)]
            self.population.append(chrom)

    def decode(self, chrom):
        from bin.entity.kontainer import Kontainer

        kontainer_real = [Kontainer(self.kapasitas) for _ in range(self.max_kontainer)]
        for idx_barang, id_k in enumerate(chrom):
            kontainer_real[id_k].tambah_barang(self.barang_list[idx_barang])

        # if kontainer kosong = remove
        kontainer_real = [k for k in kontainer_real if len(k.isi) > 0]
        return kontainer_real

    def evaluate(self, chrom):

```

```
kontainer_real = self.decode(chrom)
return objective_function(State(kontainer_real), self.kapasitas)
```

```
def selection(self):
    # Tournament selection
    a = random.choice(self.population)
    b = random.choice(self.population)
    return a if self.evaluate(a) < self.evaluate(b) else b
```

```
def crossover(self, p1, p2):
    point = random.randint(1, self.n_barang - 1)
    c1 = p1[:point] + p2[point:]
    c2 = p2[:point] + p1[point:]
    return c1, c2
```

```
def mutate(self, chrom):
    if random.random() > self.mutation_rate:
        return chrom
    # mutasi: pindahkan 1 barang ke kontainer lain
    idx = random.randint(0, self.n_barang - 1)
    new_k = random.randint(0, self.max_kontainer - 1)
    chrom[idx] = new_k
    return chrom
```

```
def run(self):
    self.initialize_population()

    print("\n=====")
    print("STATE AWAL GENETIC ALGORITHM")
    print("=====")
    print(self.initial_state)
    print(f"Objective awal : {objective_function(self.initial_state, self.kapasitas)}\n")

    start = time.time()

    for gen in range(self.max_iterasi):
        scored = [(chrom, self.evaluate(chrom)) for chrom in self.population]
        scored.sort(key=lambda x: x[1])

        best_val = scored[0][1]
        avg_val = sum(val for _, val in scored) / len(scored)

        self.best_history.append(best_val)
        self.avg_history.append(avg_val)

        if best_val < self.best_value:
            self.best_value = best_val
            self.best_chrom = scored[0][0]

        print(f"Iterasi {gen+1} | Best: {best_val} | Avg: {avg_val}")

        new_pop = []
        while len(new_pop) < self.population_size:
            p1 = self.selection()
            p2 = self.selection()
            c1, c2 = self.crossover(p1, p2)
            c1 = self.mutate(c1)
            c2 = self.mutate(c2)
            new_pop.extend([c1, c2])
```

```

        self.population = new_pop[:self.population_size]

    elapsed = time.time() - start

    best_kontainer = self.decode(self.best_chrom)

    print("\n=====")
    print("STATE AKHIR GENETIC ALGORITHM")
    print("=====")

    for i, k in enumerate(best_kontainer):
        print(f"Kontainer {i+1}: {k}")

    print("\nDetail Eksekusi:")
    print(f"Objective Terbaik      : {self.best_value}")
    print(f"Jumlah Iterasi           : {self.max_iterasi}")
    print(f"Jumlah Populasi          : {self.population_size}")
    print(f"Probabilitas Mutasi       : {self.mutation_rate}")
    print(f"Waktu Eksekusi           : {elapsed:.4f} detik")
    print("=" * 32)

    self.show_plot()

    return best_kontainer, self.best_value, elapsed

def show_plot(self):
    plt.figure(figsize=(10, 6))
    plt.plot(self.best_history, label="Best Objective")
    plt.plot(self.avg_history, label="Avg Objective")
    plt.xlabel("Iterasi")
    plt.ylabel("Objective Value")
    plt.title("Perkembangan Objective Value terhadap Iterasi")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()

```

b. Fungsi dan Prosedur

Nama fungsi	decode(self, chrom)
Deskripsi	Fungsi ini berfungsi untuk menerjemahkan sebuah kromosom (array berisi nomor kontainer untuk setiap barang) menjadi struktur solusi nyata berupa daftar objek kontainer.
Output	List objek kontainer berisi barang hasil decoding.
Source code	Source code berwarna hijau pada genetic algorithm.

Nama fungsi	evaluate(self, chrom)
Deskripsi	Fungsi ini menghitung nilai <i>objective function</i> dari suatu kromosom.
Output	Nilai objective

Source code	Source code berwarna biru pada genetic algorithm.
-------------	--

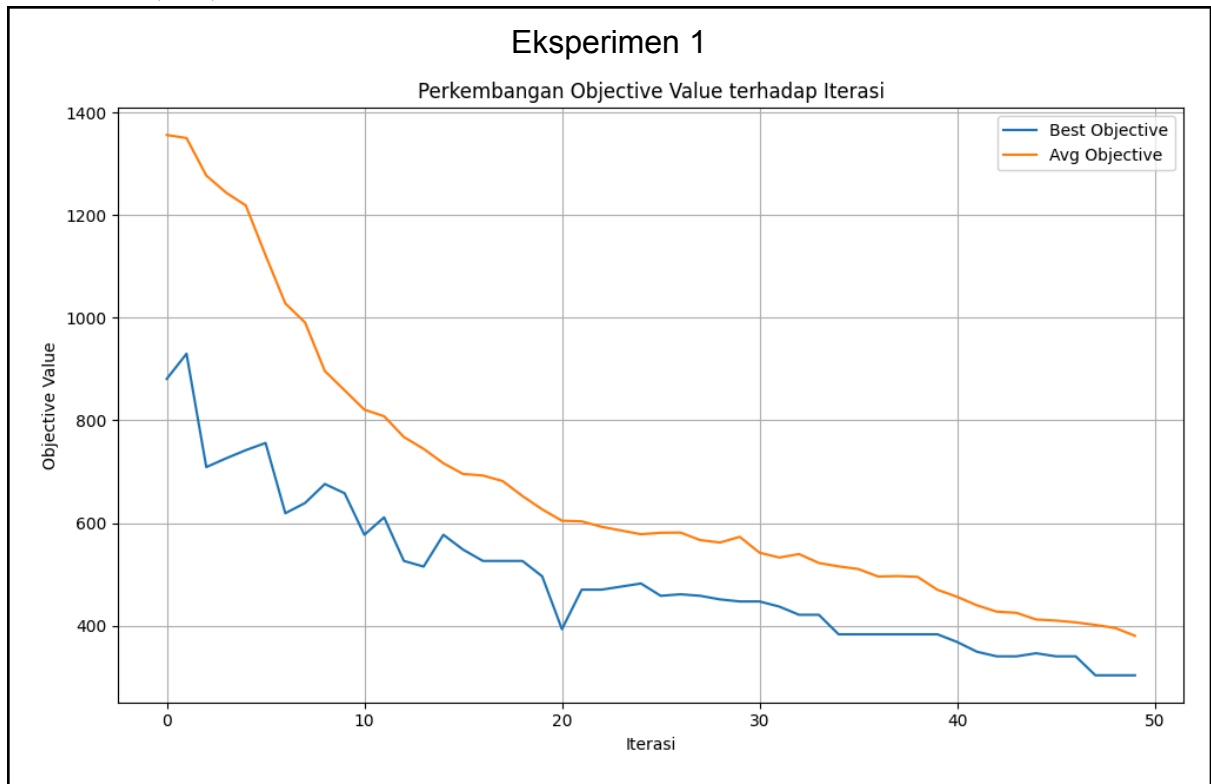
Nama fungsi	selection(self)
Deskripsi	Fungsi ini memilih parent dengan metode Tournament Selection.
Output	Kromosom yang terpilih sebagai parent.
Source code	Source code berwarna pink pada genetic algorithm.

Nama fungsi	crossover(self, p1, p2)
Deskripsi	Melakukan persilangan dua kromosom (p1 dan p2) untuk menghasilkan dua kromosom baru dgn pendekatan one-cut-point crossover
Output	Dua kromosom baru (c1, c2)
Source code	Source code berwarna ungu pada genetic algorithm.

Nama fungsi	mutate(chrom)
Deskripsi	Melakukan mutasi terhadap sebuah kromosom secara acak berdasarkan probabilitas mutasi.
Output	Kromosom baru hasil mutasi.
Source code	Source code berwarna kuning pada genetic algorithm.

- ❖ Populasi sebagai kontrol

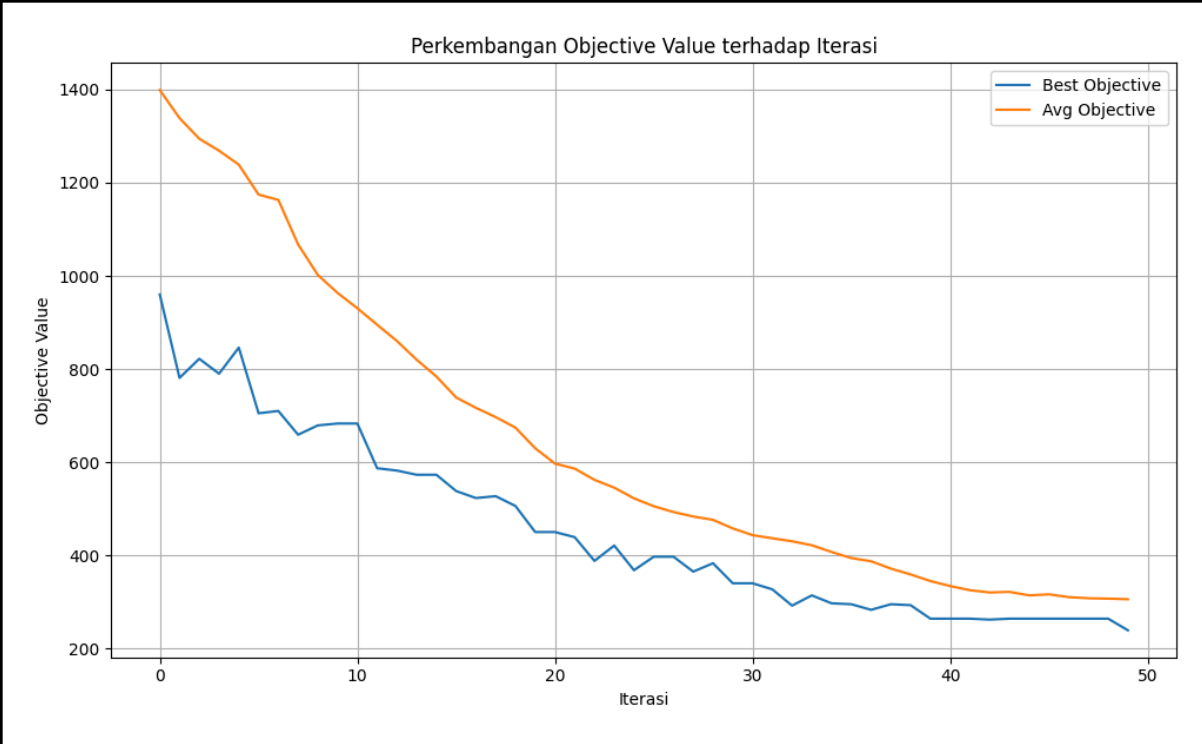
I. 100, 50, 0.3



Populasi	100	Iterasi	50	Probabilitas	0.3
State Awal	<pre> ===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)]) </pre>				



State Awal	<pre>===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>		
State Akhir	<pre>===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 97/100, isi: [BRG015 (28), BRG017 (22), BRG020 (15), BRG033 (32)]) Kontainer 9: Kontainer(total: 90/100, isi: [BRG013 (33), BRG018 (20), BRG019 (18), BRG021 (19)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 96/100, isi: [BRG026 (29), BRG030 (21), BRG028 (46)]) Kontainer 12: Kontainer(total: 100/100, isi: [BRG029 (39), BRG048 (61)]) Kontainer 13: Kontainer(total: 83/100, isi: [BRG031 (17), BRG032 (26), BRG036 (40)]) Kontainer 14: Kontainer(total: 93/100, isi: [BRG035 (36), BRG045 (57)]) Kontainer 15: Kontainer(total: 100/100, isi: [BRG034 (34), BRG052 (66)])</pre>		
Objective awal	1175	Objective Akhir	206
Durasi	0.7090 s		
Eksperimen 3			

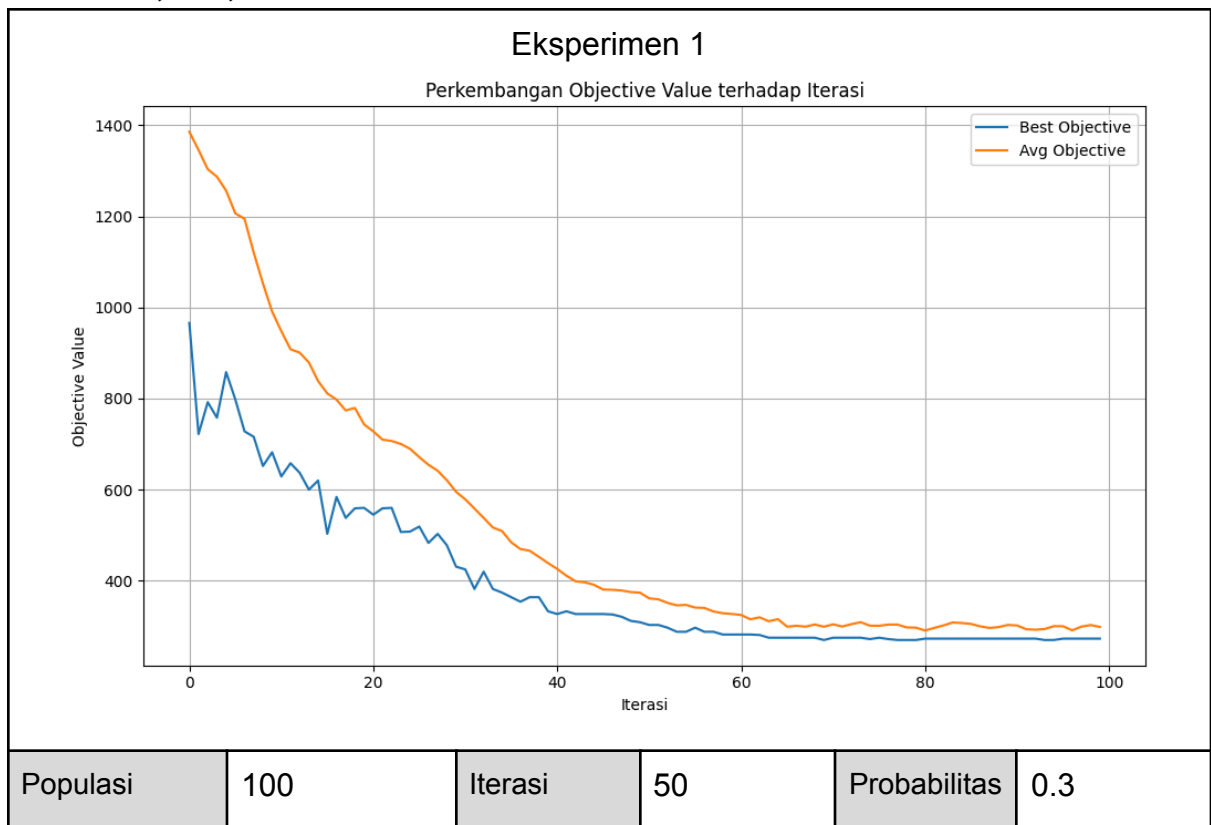


Populasi	100	Iterasi	50	Probabilitas	0.3
----------	-----	---------	----	--------------	-----

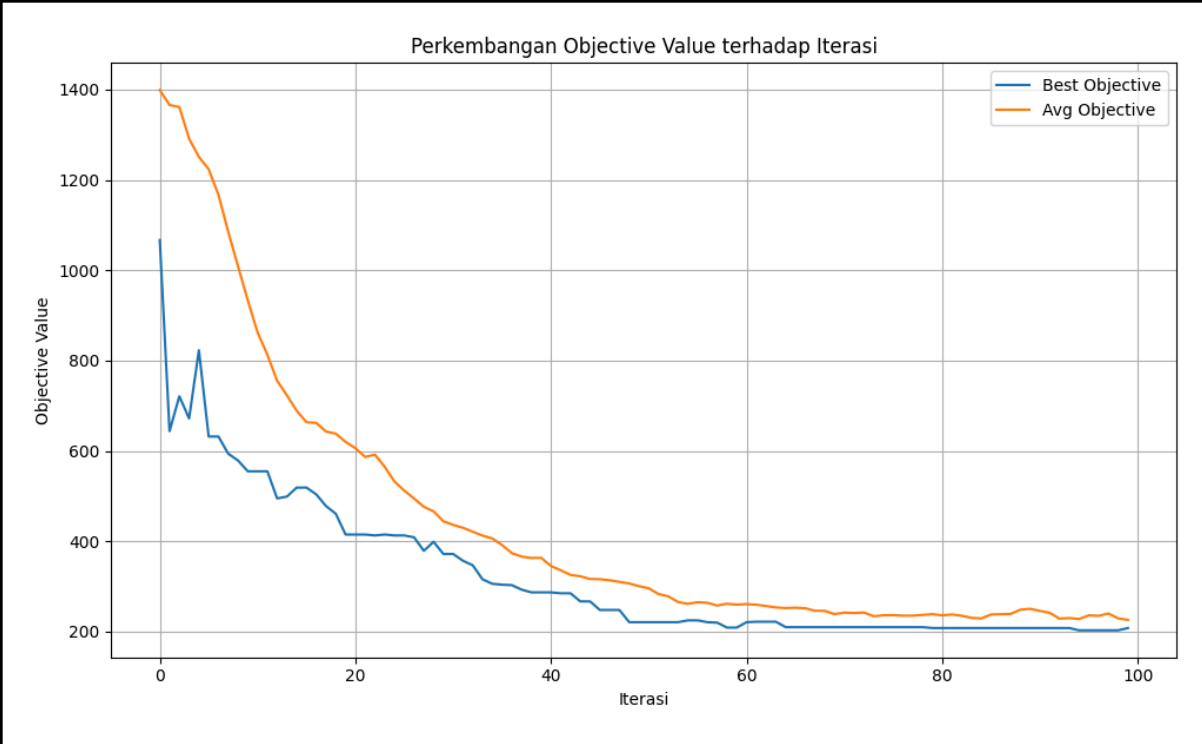
State Awal	<pre>===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>
------------	--

State Akhir	<pre> ===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 96/100, isi: [BRG006 (52), BRG038 (44)]) Kontainer 7: Kontainer(total: 98/100, isi: [BRG008 (48), BRG007 (50)]) Kontainer 8: Kontainer(total: 91/100, isi: [BRG013 (33), BRG017 (22), BRG020 (15), BRG030 (21)]) Kontainer 9: Kontainer(total: 94/100, isi: [BRG018 (20), BRG019 (18), BRG022 (27), BRG026 (29)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 90/100, isi: [BRG025 (37), BRG042 (53)]) Kontainer 12: Kontainer(total: 98/100, isi: [BRG036 (40), BRG046 (58)]) Kontainer 13: Kontainer(total: 99/100, isi: [BRG035 (36), BRG050 (63)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG029 (39), BRG048 (61)]) Kontainer 15: Kontainer(total: 79/100, isi: [BRG031 (17), BRG049 (62)]) Kontainer 16: Kontainer(total: 97/100, isi: [BRG028 (46), BRG041 (51)]) Kontainer 17: Kontainer(total: 96/100, isi: [BRG021 (19), BRG034 (34), BRG037 (43)]) </pre>		
Objective awal	1175	Objective Akhir	239
Durasi	0.6410 s		

II. 100, 100, 0.3

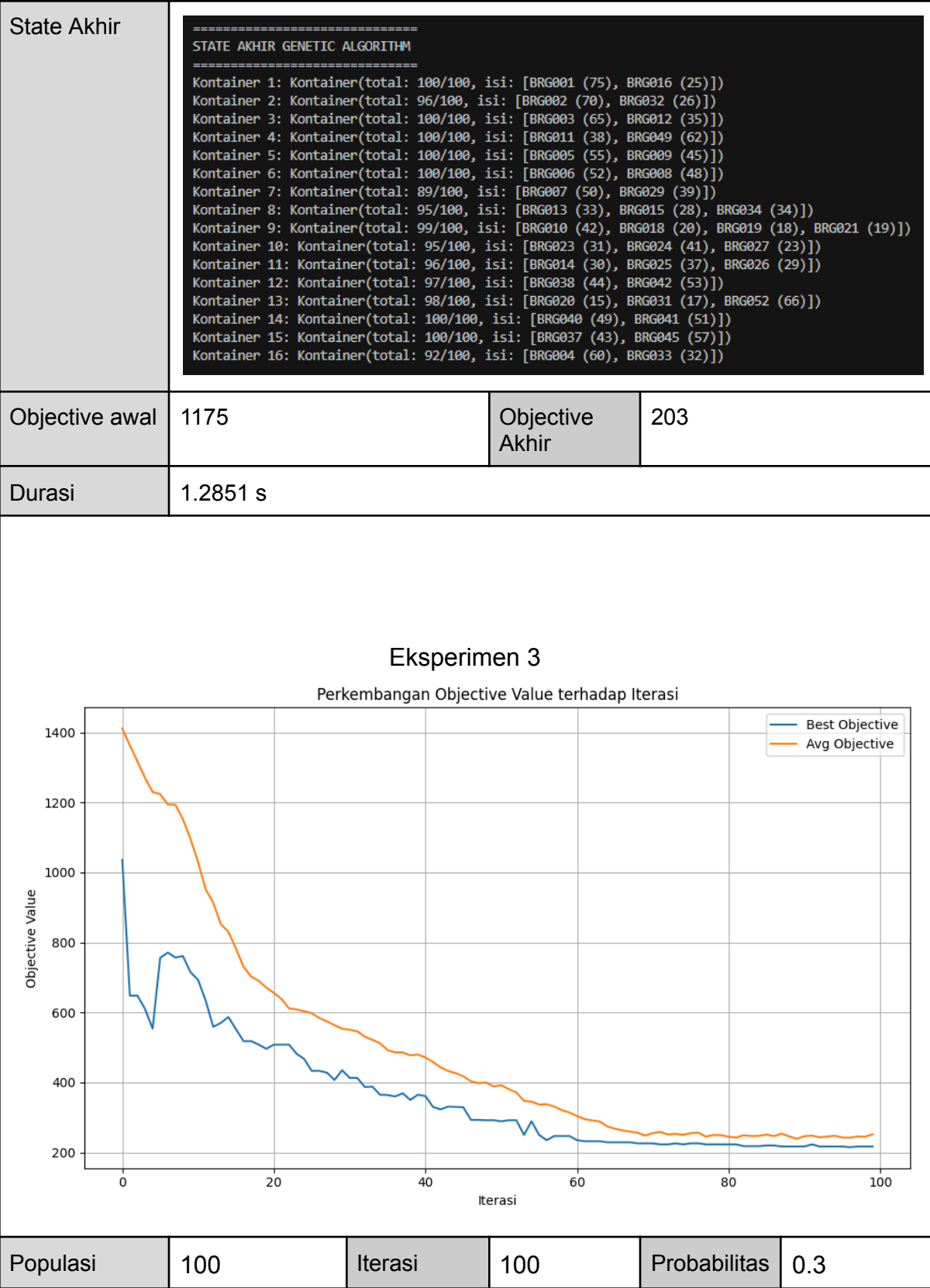


State Awal	<pre>===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>		
State Akhir	<pre>===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 88/100, isi: [BRG022 (27), BRG048 (61)]) Kontainer 9: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 10: Kontainer(total: 94/100, isi: [BRG025 (37), BRG030 (21), BRG035 (36)]) Kontainer 11: Kontainer(total: 100/100, isi: [BRG013 (33), BRG015 (28), BRG029 (39)]) Kontainer 12: Kontainer(total: 90/100, isi: [BRG020 (15), BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 13: Kontainer(total: 93/100, isi: [BRG034 (34), BRG047 (59)]) Kontainer 14: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 15: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 16: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 17: Kontainer(total: 93/100, isi: [BRG021 (19), BRG059 (74)]) Kontainer 18: Kontainer(total: 93/100, isi: [BRG018 (20), BRG058 (73)])</pre>		
Objective awal	1175	Objective Akhir	270
Durasi	1.3295 s		
Eksperimen 2			



Populasi	100	Iterasi	100	Probabilitas	0.3
----------	-----	---------	-----	--------------	-----

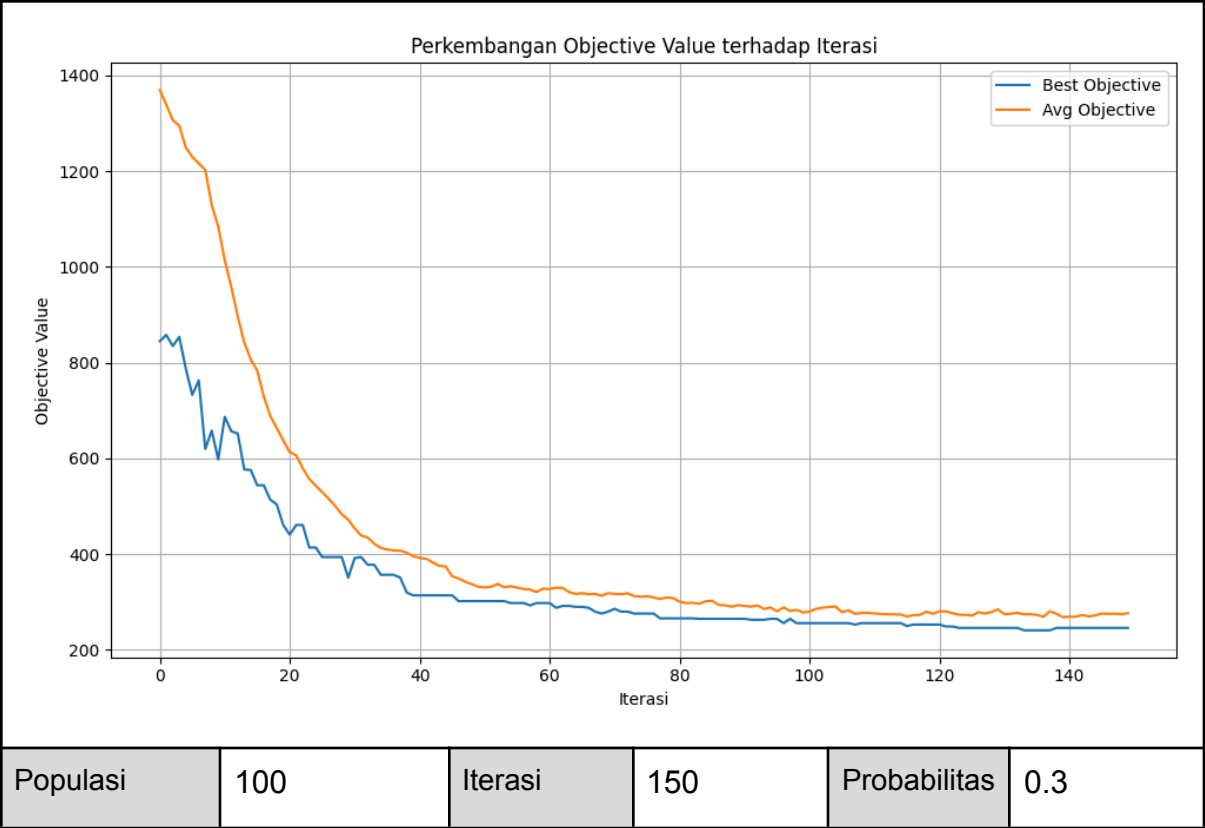
State Awal	<pre>===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>
------------	--



State Awal	<pre> ===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)]) </pre>		
State Akhir	<pre> ===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 99/100, isi: [BRG016 (25), BRG059 (74)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 99/100, isi: [BRG011 (38), BRG048 (61)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 6: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 7: Kontainer(total: 99/100, isi: [BRG010 (42), BRG018 (20), BRG019 (18), BRG021 (19)]) Kontainer 8: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 9: Kontainer(total: 96/100, isi: [BRG001 (75), BRG030 (21)]) Kontainer 10: Kontainer(total: 99/100, isi: [BRG004 (60), BRG029 (39)]) Kontainer 11: Kontainer(total: 88/100, isi: [BRG009 (45), BRG031 (17), BRG032 (26)]) Kontainer 12: Kontainer(total: 93/100, isi: [BRG007 (50), BRG037 (43)]) Kontainer 13: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 14: Kontainer(total: 93/100, isi: [BRG035 (36), BRG045 (57)]) Kontainer 15: Kontainer(total: 95/100, isi: [BRG033 (32), BRG050 (63)]) Kontainer 16: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) </pre>		
Objective awal	1175	Objective Akhir	215
Durasi	1.2957 s		

III. 100, 150, 0.3

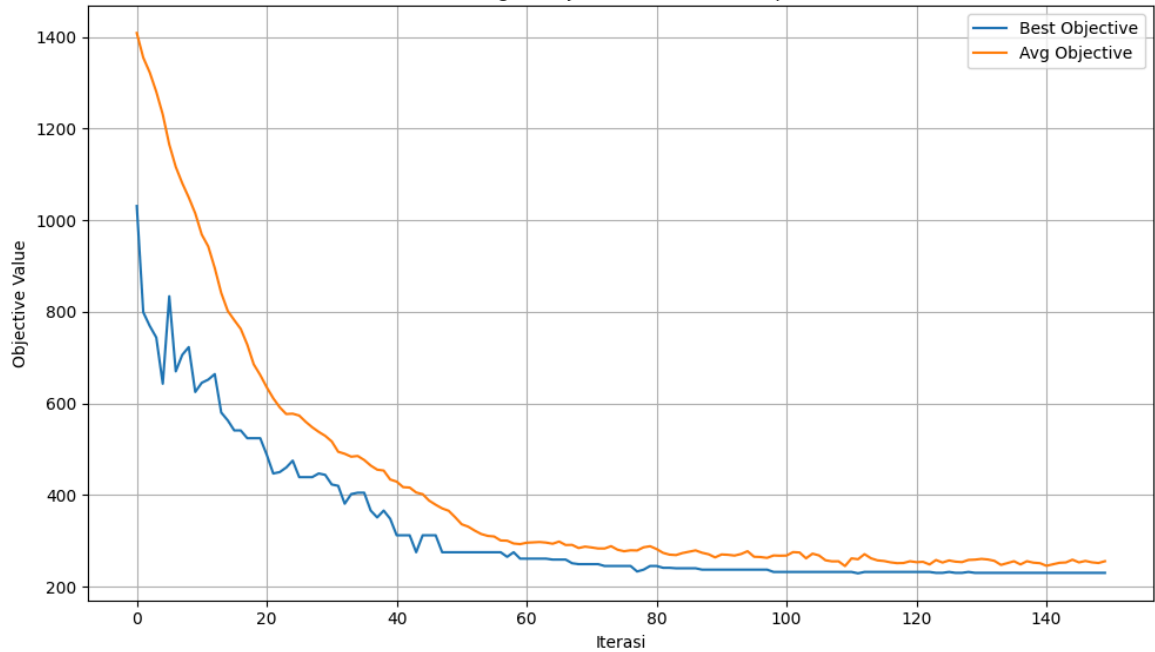
Eksperimen 1



State Awal	<pre>===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>		
State Akhir	<pre>===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 100/100, isi: [BRG022 (27), BRG032 (26), BRG039 (47)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 96/100, isi: [BRG019 (18), BRG024 (41), BRG025 (37)]) Kontainer 10: Kontainer(total: 86/100, isi: [BRG031 (17), BRG055 (69)]) Kontainer 11: Kontainer(total: 97/100, isi: [BRG037 (43), BRG043 (54)]) Kontainer 12: Kontainer(total: 96/100, isi: [BRG036 (40), BRG044 (56)]) Kontainer 13: Kontainer(total: 77/100, isi: [BRG021 (19), BRG046 (58)]) Kontainer 14: Kontainer(total: 99/100, isi: [BRG033 (32), BRG053 (67)]) Kontainer 15: Kontainer(total: 87/100, isi: [BRG030 (21), BRG052 (66)]) Kontainer 16: Kontainer(total: 98/100, isi: [BRG018 (20), BRG034 (34), BRG038 (44)]) Kontainer 17: Kontainer(total: 97/100, isi: [BRG026 (29), BRG054 (68)])</pre>		
Objective awal	1175	Objective Akhir	241
Durasi	2.0637 s		

Eksperimen 2

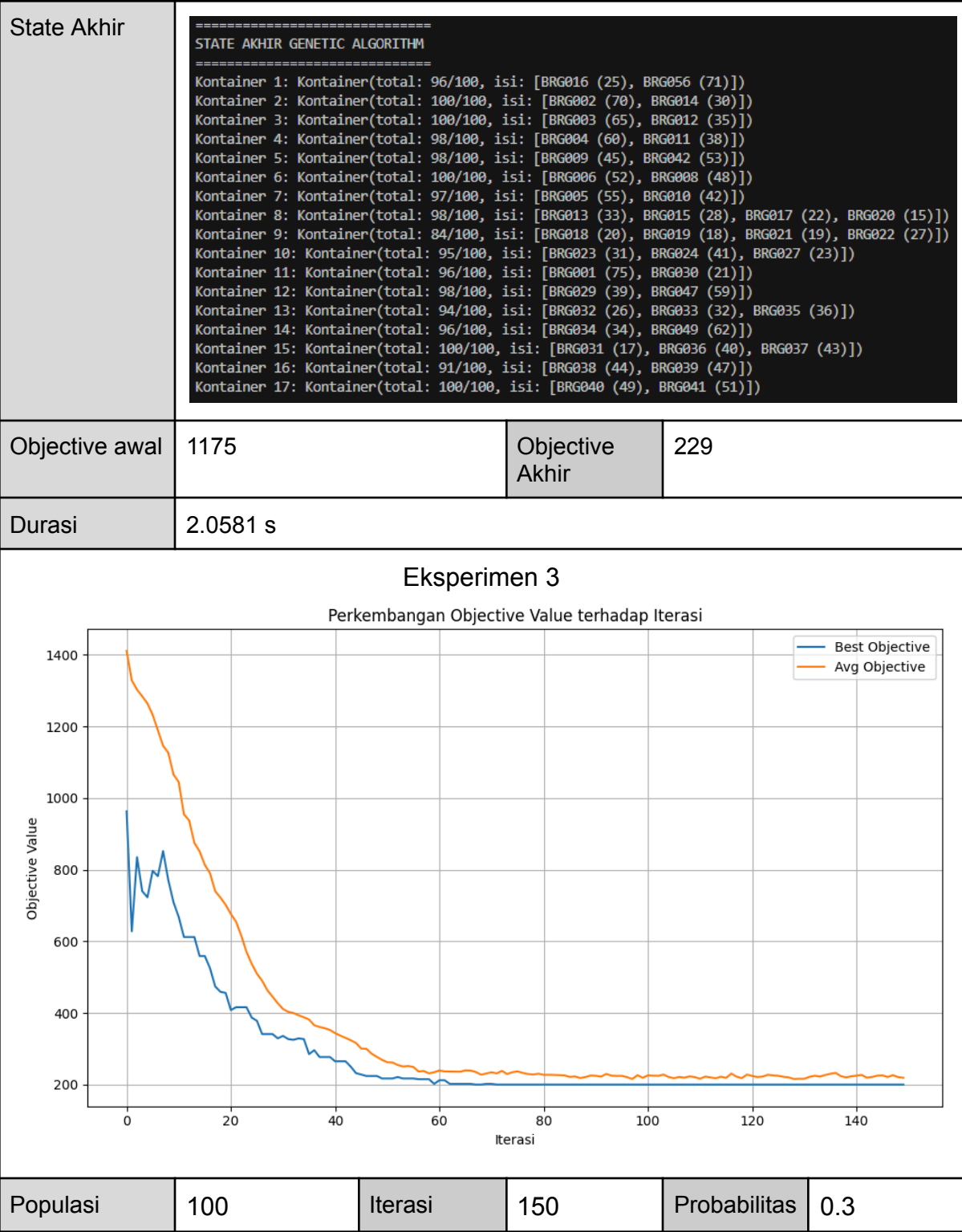
Perkembangan Objective Value terhadap Iterasi



Populasi	100	Iterasi	150	Probabilitas	0.3
----------	-----	---------	-----	--------------	-----

State Awal

```
=====
STATE AWAL GENETIC ALGORITHM
=====
Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)])
Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)])
Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)])
Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)])
Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)])
Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)])
Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)])
Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)])
Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)])
Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)])
Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)])
Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)])
Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)])
Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)])
Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)])
Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)])
Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)])
Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)])
Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)])
Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)])
Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)])
Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)])
Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)])
Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)])
Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)])
Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)])
Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)])
Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)])
Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)])
Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)])
Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)])
Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)])
Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)])
Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)])
Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)])
Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])
```



State Awal	<pre> ===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)]) </pre>		
State Akhir	<pre> ===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 99/100, isi: [BRG007 (50), BRG040 (49)]) Kontainer 8: Kontainer(total: 100/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG031 (17)]) Kontainer 9: Kontainer(total: 99/100, isi: [BRG010 (42), BRG018 (20), BRG019 (18), BRG021 (19)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 94/100, isi: [BRG025 (37), BRG045 (57)]) Kontainer 12: Kontainer(total: 95/100, isi: [BRG035 (36), BRG047 (59)]) Kontainer 13: Kontainer(total: 100/100, isi: [BRG029 (39), BRG048 (61)]) Kontainer 14: Kontainer(total: 96/100, isi: [BRG033 (32), BRG051 (64)]) Kontainer 15: Kontainer(total: 92/100, isi: [BRG030 (21), BRG056 (71)]) Kontainer 16: Kontainer(total: 92/100, isi: [BRG034 (34), BRG046 (58)]) </pre>		
Objective awal	1175	Objective Akhir	200
Durasi	2.0514 s		

Analisis:

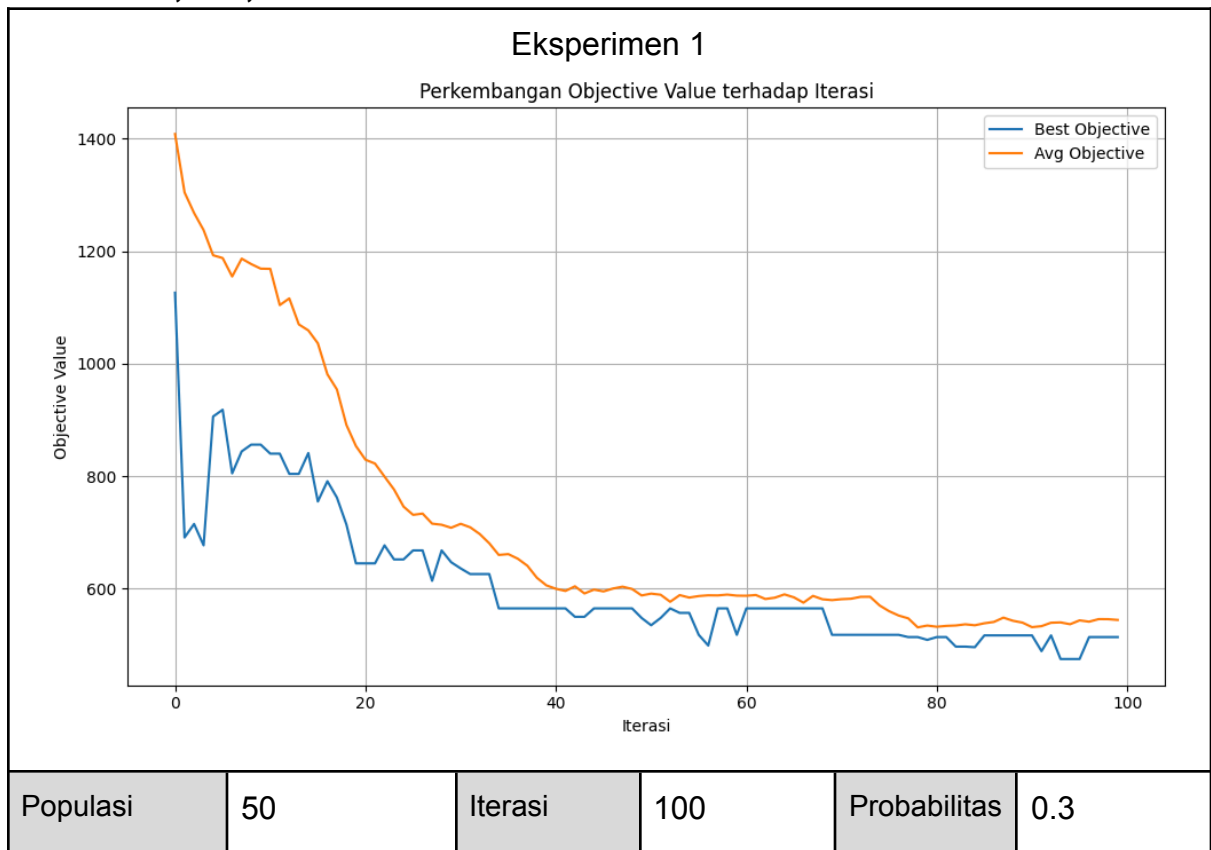
- Konfigurasi 1 (Populasi = 100, Iterasi = 50, Mutasi = 0.3)
 Dengan jumlah iterasi yang relatif kecil (50 generasi), GA sudah menunjukkan penurunan *objective function* yang signifikan dari 1175 menjadi rata-rata 249.3. Durasi waktu masih sangat efisien (<1 detik),

menunjukkan bahwa GA dapat melakukan perbaikan solusi secara cepat pada fase awal evolusi.

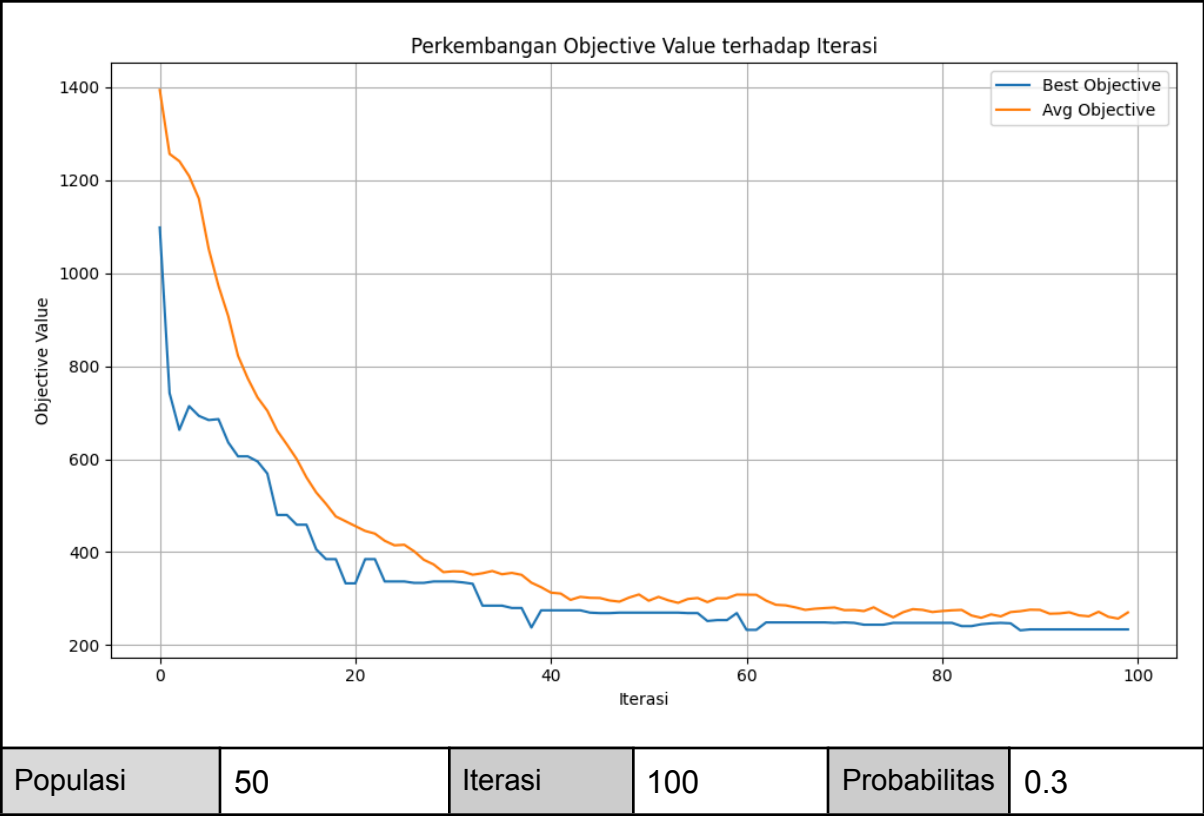
- Konfigurasi 2 (Populasi = 100, Iterasi = 100, Mutasi = 0.3)
Ketika jumlah iterasi ditingkatkan dua kali lipat, nilai *objective function* akhir sedikit membaik dibanding konfigurasi sebelumnya. Namun peningkatan kualitas solusi tidak sebanding dengan kenaikan waktu eksekusi (sekitar dua kali lebih lama). Ini menunjukkan bahwa sebagian besar perbaikan terjadi pada generasi awal, sedangkan generasi akhir cenderung mengalami stagnasi (*convergence*).
- Konfigurasi 3 (Populasi = 100, Iterasi = 150, Mutasi = 0.3)
Penambahan jumlah iterasi hingga 150 menghasilkan sedikit peningkatan akurasi (rata-rata *objective* lebih rendah). Namun peningkatan kualitas ini relatif kecil jika dibandingkan dengan kenaikan waktu eksekusi yang signifikan. Hal ini menunjukkan titik efisiensi algoritma berada di kisaran 100 iterasi.

❖ Iterasi (generasi) sebagai kontrol

I. 50, 100, 0.3



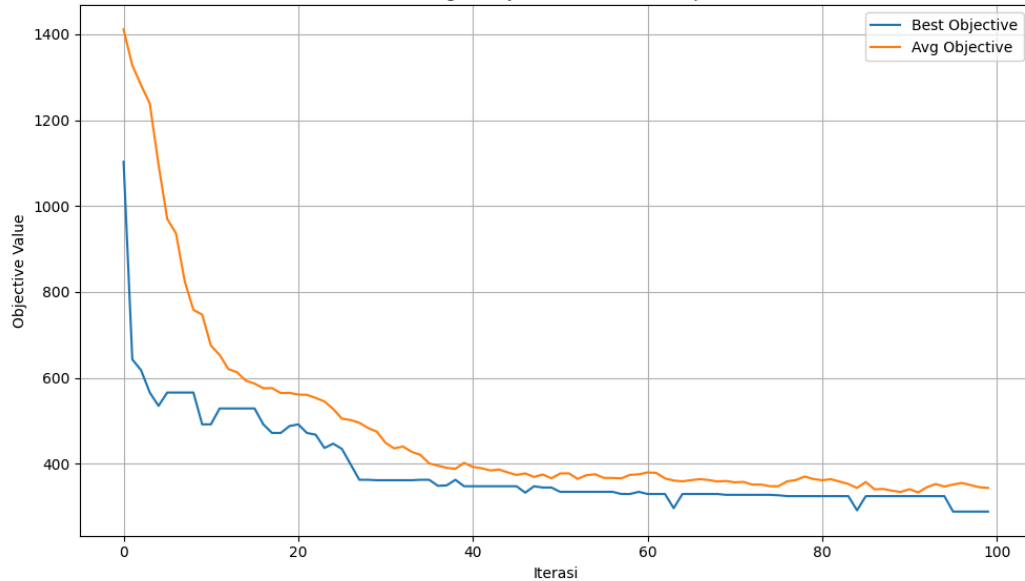
State Awal	<pre>STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>		
State Akhir	<pre>===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 81/100, isi: [BRG025 (37), BRG038 (44)]) Kontainer 2: Kontainer(total: 97/100, isi: [BRG005 (55), BRG010 (42)]) Kontainer 3: Kontainer(total: 75/100, isi: [BRG001 (75)]) Kontainer 4: Kontainer(total: 91/100, isi: [BRG007 (50), BRG019 (18), BRG027 (23)]) Kontainer 5: Kontainer(total: 99/100, isi: [BRG009 (45), BRG043 (54)]) Kontainer 6: Kontainer(total: 99/100, isi: [BRG028 (46), BRG042 (53)]) Kontainer 7: Kontainer(total: 90/100, isi: [BRG037 (43), BRG039 (47)]) Kontainer 8: Kontainer(total: 91/100, isi: [BRG034 (34), BRG045 (57)]) Kontainer 9: Kontainer(total: 86/100, isi: [BRG004 (60), BRG032 (26)]) Kontainer 10: Kontainer(total: 82/100, isi: [BRG013 (33), BRG040 (49)]) Kontainer 11: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 12: Kontainer(total: 88/100, isi: [BRG022 (27), BRG048 (61)]) Kontainer 13: Kontainer(total: 86/100, isi: [BRG003 (65), BRG030 (21)]) Kontainer 14: Kontainer(total: 86/100, isi: [BRG011 (38), BRG008 (48)]) Kontainer 15: Kontainer(total: 95/100, isi: [BRG012 (35), BRG018 (20), BRG036 (40)]) Kontainer 16: Kontainer(total: 67/100, isi: [BRG015 (28), BRG029 (39)]) Kontainer 17: Kontainer(total: 99/100, isi: [BRG035 (36), BRG050 (63)]) Kontainer 18: Kontainer(total: 91/100, isi: [BRG016 (25), BRG052 (66)]) Kontainer 19: Kontainer(total: 79/100, isi: [BRG020 (15), BRG051 (64)]) Kontainer 20: Kontainer(total: 60/100, isi: [BRG023 (31), BRG026 (29)]) Kontainer 21: Kontainer(total: 93/100, isi: [BRG006 (52), BRG017 (22), BRG021 (19)])</pre>		
Objective awal	1175	Objective Akhir	475
Durasi	0.6777 s		
Eksperimen 2			



State Awal	<pre>===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>		
State Akhir	<pre>===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 98/100, isi: [BRG014 (30), BRG054 (68)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 7: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 8: Kontainer(total: 87/100, isi: [BRG008 (48), BRG018 (20), BRG021 (19)]) Kontainer 9: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 10: Kontainer(total: 91/100, isi: [BRG002 (70), BRG030 (21)]) Kontainer 11: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 12: Kontainer(total: 95/100, isi: [BRG006 (52), BRG031 (17), BRG032 (26)]) Kontainer 13: Kontainer(total: 99/100, isi: [BRG026 (29), BRG034 (34), BRG035 (36)]) Kontainer 14: Kontainer(total: 99/100, isi: [BRG036 (40), BRG047 (59)]) Kontainer 15: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 16: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)])</pre>		
Objective awal	1175	Objective Akhir	232
Durasi	0.7277 s		

Eksperimen 3

Perkembangan Objective Value terhadap Iterasi



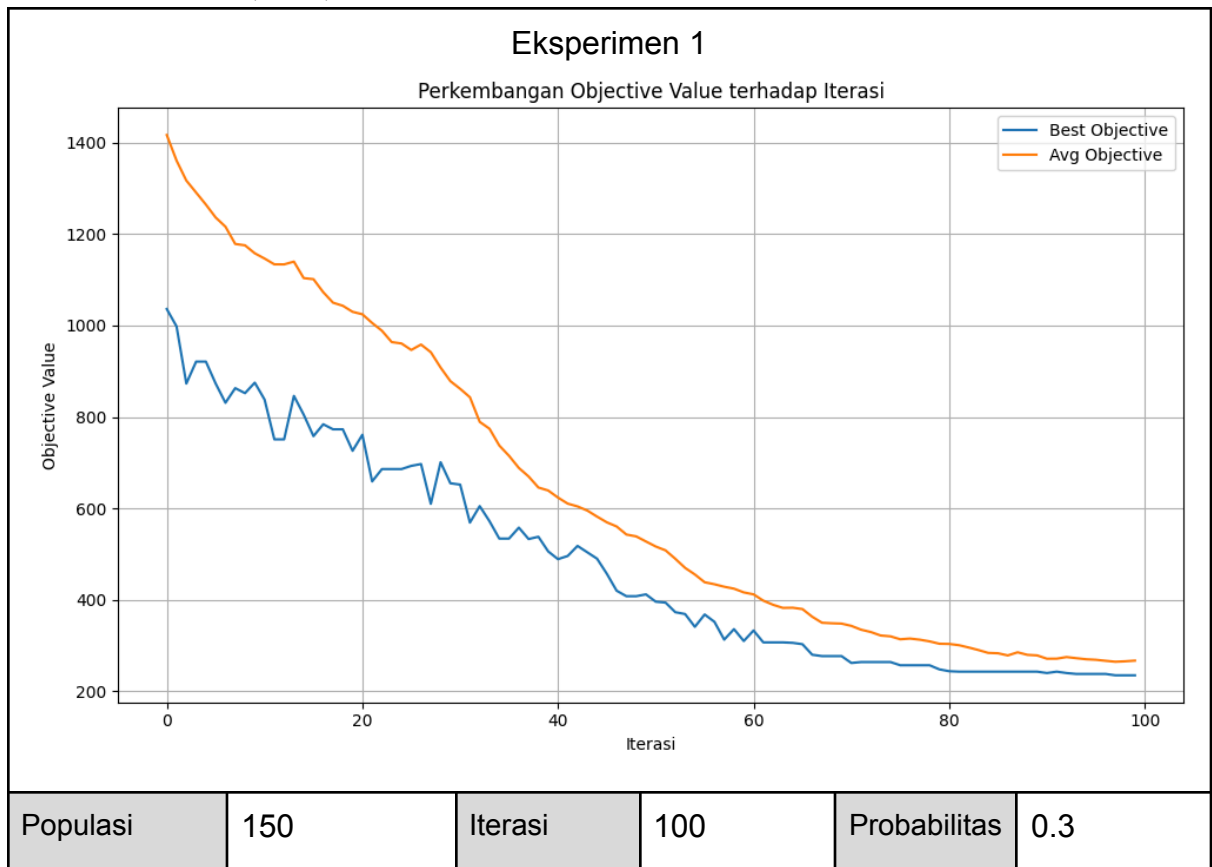
Populasi	50	Iterasi	100	Probabilitas	0.3
----------	----	---------	-----	--------------	-----

State Awal

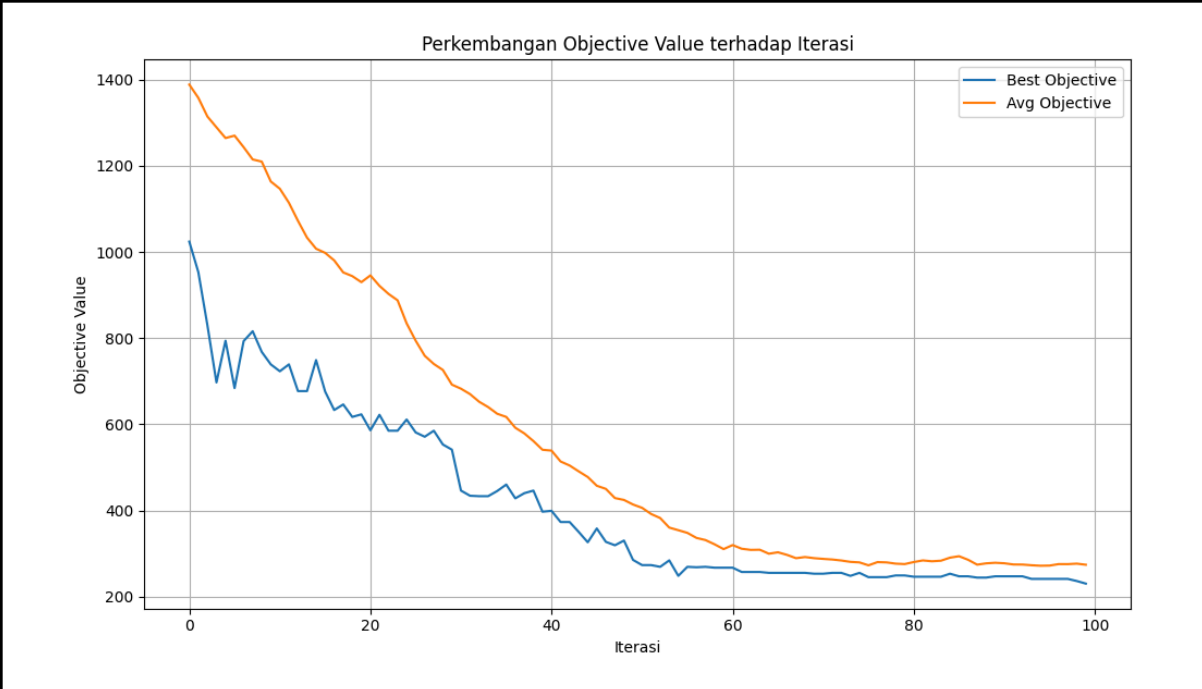
```
=====
STATE AWAL GENETIC ALGORITHM
=====
Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)])
Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)])
Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)])
Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)])
Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)])
Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)])
Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)])
Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)])
Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)])
Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)])
Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)])
Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)])
Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)])
Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)])
Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)])
Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)])
Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)])
Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)])
Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)])
Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)])
Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)])
Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)])
Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)])
Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)])
Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)])
Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)])
Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)])
Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)])
Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)])
Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)])
Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)])
Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)])
Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)])
Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)])
Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)])
Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])
```

State Akhir	<pre> ===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 6: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 7: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 8: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 9: Kontainer(total: 93/100, isi: [BRG023 (31), BRG027 (23), BRG029 (39)]) Kontainer 10: Kontainer(total: 100/100, isi: [BRG025 (37), BRG026 (29), BRG034 (34)]) Kontainer 11: Kontainer(total: 100/100, isi: [BRG028 (46), BRG043 (54)]) Kontainer 12: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 13: Kontainer(total: 96/100, isi: [BRG005 (55), BRG024 (41)]) Kontainer 14: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 15: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 16: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 17: Kontainer(total: 98/100, isi: [BRG009 (45), BRG042 (53)]) Kontainer 18: Kontainer(total: 83/100, isi: [BRG030 (21), BRG049 (62)]) </pre>		
Objective awal	1175	Objective Akhir	289
Durasi	0.6438 s		

II. 150, 100, 0.3



State Awal	<pre>===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>		
State Akhir	<pre>===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 90/100, isi: [BRG006 (52), BRG018 (20), BRG019 (18)]) Kontainer 2: Kontainer(total: 90/100, isi: [BRG029 (39), BRG041 (51)]) Kontainer 3: Kontainer(total: 98/100, isi: [BRG005 (55), BRG037 (43)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG035 (36), BRG049 (62)]) Kontainer 5: Kontainer(total: 98/100, isi: [BRG024 (41), BRG045 (57)]) Kontainer 6: Kontainer(total: 96/100, isi: [BRG007 (50), BRG028 (46)]) Kontainer 7: Kontainer(total: 97/100, isi: [BRG010 (42), BRG013 (33), BRG017 (22)]) Kontainer 8: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 9: Kontainer(total: 96/100, isi: [BRG016 (25), BRG021 (19), BRG027 (23), BRG026 (29)]) Kontainer 10: Kontainer(total: 96/100, isi: [BRG011 (38), BRG046 (58)]) Kontainer 11: Kontainer(total: 93/100, isi: [BRG009 (45), BRG008 (48)]) Kontainer 12: Kontainer(total: 100/100, isi: [BRG012 (35), BRG023 (31), BRG034 (34)]) Kontainer 13: Kontainer(total: 90/100, isi: [BRG022 (27), BRG025 (37), BRG032 (26)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG015 (28), BRG057 (72)]) Kontainer 15: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 16: Kontainer(total: 92/100, isi: [BRG001 (75), BRG031 (17)])</pre>		
Objective awal	1175	Objective Akhir	235
Durasi	2.2503		
Eksperimen 2			

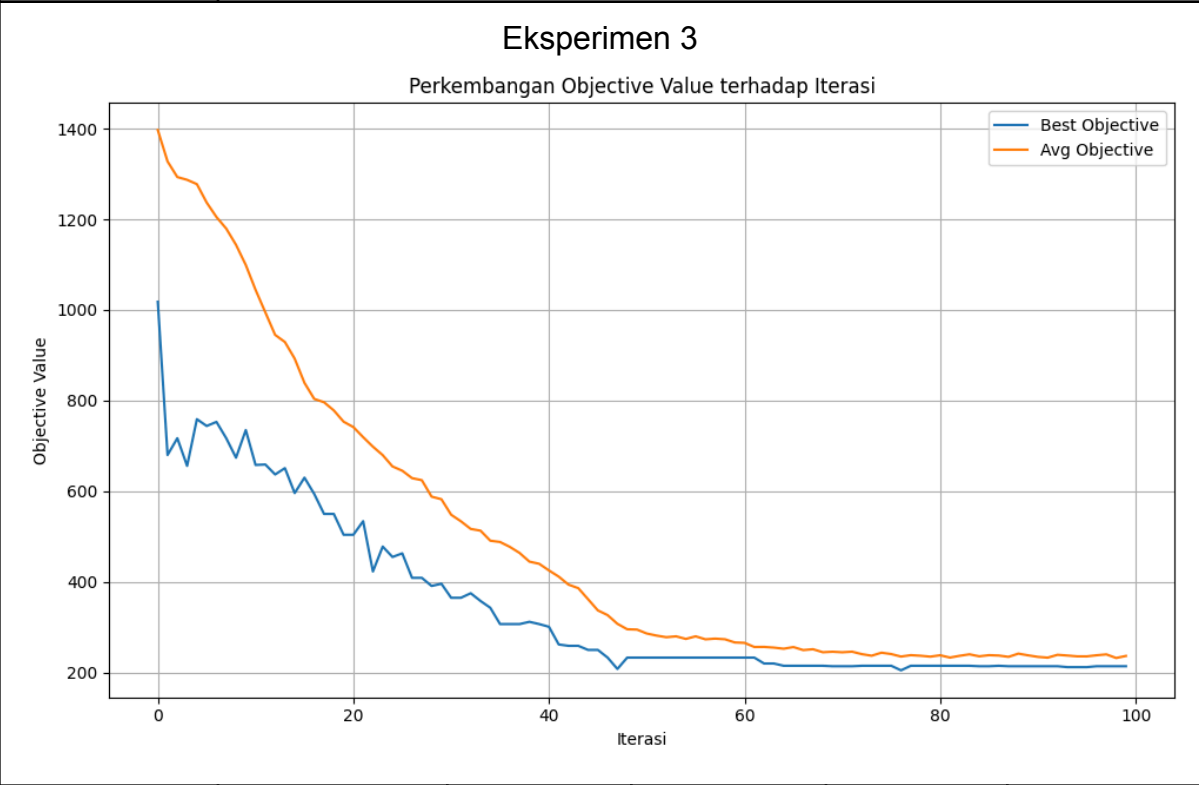


Populasi	150	Iterasi	100	Probabilitas	0.3
----------	-----	---------	-----	--------------	-----

State Awal	<pre>===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)])</pre>
------------	--

State Akhir	<pre> ===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 82/100, isi: [BRG020 (15), BRG053 (67)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 97/100, isi: [BRG015 (28), BRG017 (22), BRG039 (47)]) Kontainer 9: Kontainer(total: 97/100, isi: [BRG013 (33), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 92/100, isi: [BRG027 (23), BRG055 (69)]) Kontainer 11: Kontainer(total: 99/100, isi: [BRG024 (41), BRG025 (37), BRG030 (21)]) Kontainer 12: Kontainer(total: 93/100, isi: [BRG029 (39), BRG043 (54)]) Kontainer 13: Kontainer(total: 100/100, isi: [BRG031 (17), BRG033 (32), BRG041 (51)]) Kontainer 14: Kontainer(total: 99/100, isi: [BRG028 (46), BRG042 (53)]) Kontainer 15: Kontainer(total: 97/100, isi: [BRG036 (40), BRG045 (57)]) Kontainer 16: Kontainer(total: 94/100, isi: [BRG032 (26), BRG054 (68)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) </pre>		
-------------	--	--	--

Objective awal	1175	Objective Akhir	230
Durasi	2.1047 s		



Populasi	150	Iterasi	100	Probabilitas	0.3
----------	-----	---------	-----	--------------	-----

State Awal	<pre> ===== STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)]) </pre>		
State Akhir	<pre> ===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 3: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 4: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 6: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 7: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 8: Kontainer(total: 96/100, isi: [BRG019 (18), BRG021 (19), BRG022 (27), BRG033 (32)]) Kontainer 9: Kontainer(total: 98/100, isi: [BRG023 (31), BRG024 (41), BRG032 (26)]) Kontainer 10: Kontainer(total: 84/100, isi: [BRG031 (17), BRG053 (67)]) Kontainer 11: Kontainer(total: 98/100, isi: [BRG038 (44), BRG043 (54)]) Kontainer 12: Kontainer(total: 99/100, isi: [BRG018 (20), BRG027 (23), BRG044 (56)]) Kontainer 13: Kontainer(total: 96/100, isi: [BRG025 (37), BRG047 (59)]) Kontainer 14: Kontainer(total: 100/100, isi: [BRG037 (43), BRG045 (57)]) Kontainer 15: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 16: Kontainer(total: 96/100, isi: [BRG039 (47), BRG040 (49)]) </pre>		
Objective awal	1175	Objective Akhir	205
Durasi	1.9481 detik		

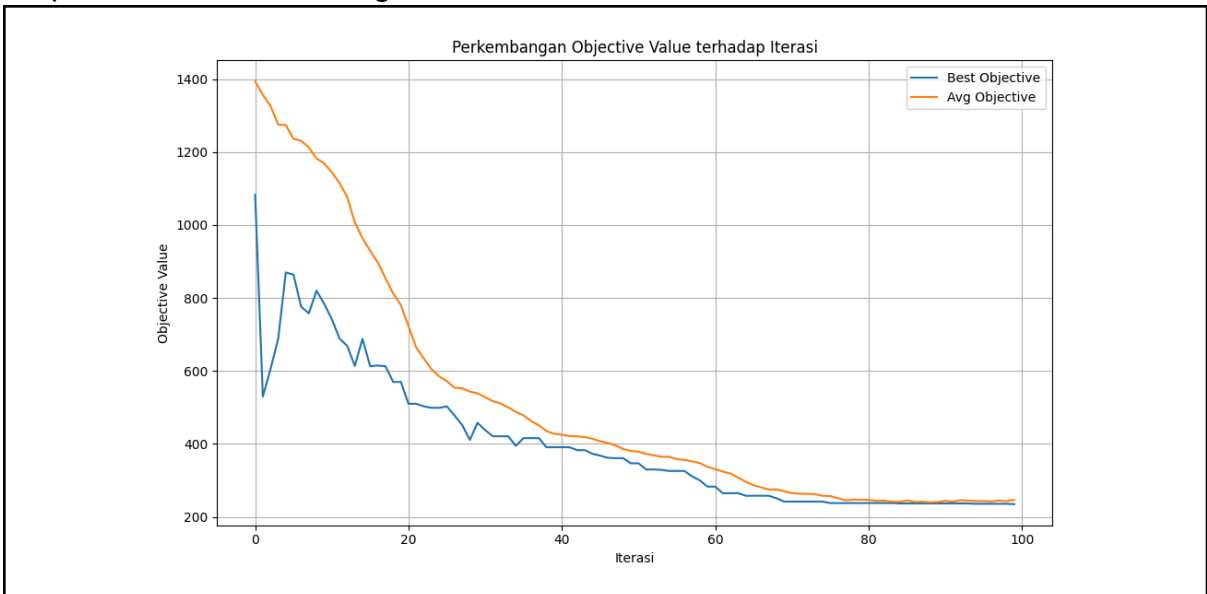
Analisis:

- Konfigurasi 1 (Populasi = 50, Iterasi = 100, Mutasi = 0.3)
Populasi yang lebih kecil menghasilkan hasil yang kurang optimal karena keragaman genetik (*diversity*) lebih rendah. Beberapa individu cepat mendominasi populasi, menyebabkan konvergensi prematur (*premature convergence*). Walau cepat, kualitas solusi rata-rata menurun cukup signifikan dibanding populasi 100 atau lebih.

- Konfigurasi 2 (Populasi = 100, Iterasi = 100, Mutasi = 0.3)
Populasi yang lebih besar (150 individu) meningkatkan kualitas hasil karena ruang solusi yang dijelajahi menjadi lebih luas. Nilai *objective function* akhir membaik secara konsisten dibanding populasi 50, meskipun waktu komputasi meningkat sekitar tiga kali lipat. Hal ini menunjukkan bahwa diversitas populasi sangat berpengaruh terhadap performa GA, terutama pada tahap awal evolusi.

❖ Eksperimen pengaruh probabilitas mutasi

Untuk menguji *mutation rate*, dilakukan pula eksperimen untuk varian probabilitas = 0.1 sebagai berikut.



Populasi	150	Iterasi	100	Probabilitas	0.1
----------	-----	---------	-----	--------------	-----

State Awal	<pre> STATE AWAL GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 98/100, isi: [BRG004 (60), BRG011 (38)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 92/100, isi: [BRG007 (50), BRG010 (42)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (28), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 95/100, isi: [BRG023 (31), BRG024 (41), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 85/100, isi: [BRG028 (46), BRG029 (39)]) Kontainer 13: Kontainer(total: 75/100, isi: [BRG031 (17), BRG032 (26), BRG033 (32)]) Kontainer 14: Kontainer(total: 70/100, isi: [BRG034 (34), BRG035 (36)]) Kontainer 15: Kontainer(total: 83/100, isi: [BRG036 (40), BRG037 (43)]) Kontainer 16: Kontainer(total: 91/100, isi: [BRG038 (44), BRG039 (47)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG040 (49), BRG041 (51)]) Kontainer 18: Kontainer(total: 53/100, isi: [BRG042 (53)]) Kontainer 19: Kontainer(total: 54/100, isi: [BRG043 (54)]) Kontainer 20: Kontainer(total: 56/100, isi: [BRG044 (56)]) Kontainer 21: Kontainer(total: 57/100, isi: [BRG045 (57)]) Kontainer 22: Kontainer(total: 58/100, isi: [BRG046 (58)]) Kontainer 23: Kontainer(total: 59/100, isi: [BRG047 (59)]) Kontainer 24: Kontainer(total: 61/100, isi: [BRG048 (61)]) Kontainer 25: Kontainer(total: 62/100, isi: [BRG049 (62)]) Kontainer 26: Kontainer(total: 63/100, isi: [BRG050 (63)]) Kontainer 27: Kontainer(total: 64/100, isi: [BRG051 (64)]) Kontainer 28: Kontainer(total: 66/100, isi: [BRG052 (66)]) Kontainer 29: Kontainer(total: 67/100, isi: [BRG053 (67)]) Kontainer 30: Kontainer(total: 68/100, isi: [BRG054 (68)]) Kontainer 31: Kontainer(total: 69/100, isi: [BRG055 (69)]) Kontainer 32: Kontainer(total: 71/100, isi: [BRG056 (71)]) Kontainer 33: Kontainer(total: 72/100, isi: [BRG057 (72)]) Kontainer 34: Kontainer(total: 73/100, isi: [BRG058 (73)]) Kontainer 35: Kontainer(total: 74/100, isi: [BRG059 (74)]) Kontainer 36: Kontainer(total: 80/100, isi: [BRG060 (80)]) </pre>
------------	--

State Akhir	<pre> ===== STATE AKHIR GENETIC ALGORITHM ===== Kontainer 1: Kontainer(total: 100/100, isi: [BRG001 (75), BRG016 (25)]) Kontainer 2: Kontainer(total: 100/100, isi: [BRG002 (70), BRG014 (30)]) Kontainer 3: Kontainer(total: 100/100, isi: [BRG003 (65), BRG012 (35)]) Kontainer 4: Kontainer(total: 100/100, isi: [BRG004 (60), BRG036 (40)]) Kontainer 5: Kontainer(total: 100/100, isi: [BRG005 (55), BRG009 (45)]) Kontainer 6: Kontainer(total: 100/100, isi: [BRG006 (52), BRG008 (48)]) Kontainer 7: Kontainer(total: 99/100, isi: [BRG007 (50), BRG040 (49)]) Kontainer 8: Kontainer(total: 98/100, isi: [BRG013 (33), BRG015 (28), BRG017 (22), BRG020 (15)]) Kontainer 9: Kontainer(total: 84/100, isi: [BRG018 (20), BRG019 (18), BRG021 (19), BRG022 (27)]) Kontainer 10: Kontainer(total: 96/100, isi: [BRG010 (42), BRG023 (31), BRG027 (23)]) Kontainer 11: Kontainer(total: 87/100, isi: [BRG025 (37), BRG026 (29), BRG030 (21)]) Kontainer 12: Kontainer(total: 100/100, isi: [BRG029 (39), BRG048 (61)]) Kontainer 13: Kontainer(total: 89/100, isi: [BRG028 (46), BRG031 (17), BRG032 (26)]) Kontainer 14: Kontainer(total: 88/100, isi: [BRG034 (34), BRG043 (54)]) Kontainer 15: Kontainer(total: 99/100, isi: [BRG037 (43), BRG044 (56)]) Kontainer 16: Kontainer(total: 95/100, isi: [BRG038 (44), BRG041 (51)]) Kontainer 17: Kontainer(total: 100/100, isi: [BRG033 (32), BRG054 (68)]) </pre>		
Objective awal	1175	Objective Akhir	235
Durasi	2.2444 s		

Analisis Pengaruh Mutation Rate:

Dengan *mutation rate* yang lebih rendah (0.1), hasil akhir relatif sama dengan mutasi 0.3, tetapi waktu eksekusi sedikit lebih singkat. Ini menunjukkan bahwa tingkat mutasi sedang (0.3) sudah cukup untuk menjaga keragaman tanpa memperlambat konvergensi secara signifikan. *Mutation rate* yang terlalu rendah berpotensi menyebabkan populasi yang stagnan, sedangkan nilai terlalu tinggi akan membuat pencarian terlalu acak.

Bab 3

Kesimpulan dan Saran

3.1 Kesimpulan

- Steepest Ascent Hill Climbing terbukti sangat stabil dan deterministik, namun secara konsisten terjebak dalam local optimum yang sama di semua pengujian. Sedangkan, algoritma stokastik seperti Stochastic, Sideways Move, dan Random-Restart menunjukkan bahwa mereka dapat mengeksplorasi *search space* yang lebih luas, memungkinkan mereka untuk menemukan solusi yang lebih baik.
- Stochastic dan Sideways Move Hill Climbing berhasil menemukan solusi yang lebih baik dari state awalnya, namun performanya sangat bervariasi dan sering terjebak di local optimum yang berbeda-beda. Sedangkan Random-Restart Hill Climbing terbukti paling efektif, ditunjukkan dengan dicapainya skor 373 dan jumlah kontainer menjadi hanya 30.
- Simulated Annealing dalam eksperimen ini kurang bisa untuk mengoptimalkan solusi sedangkan Genetic Algorithm dapat memberikan hasil terbaik. Algoritma ini dapat menemukan solusi dengan objective score yang paling rendah dan melakukannya dalam waktu yang sangat pendek.
- Genetic Algorithm menjadi algoritma yang paling optimal karena dapat mendapatkan objective score yang sangat rendah dengan waktu yang cepat, didepan Random-Restart yang dapat memberikan solusi yang baik, namun memakan waktu yang lama.

3.2 Saran

Berdasarkan hasil eksperimen ini, terdapat beberapa saran untuk di masa depannya, yaitu:

- Kegagalan algoritma Simulated Annealing perlu diberikan perhatian ekstra. Kegagalan algoritma ini bisa jadi ada dari kesalahan kode atau *parameter tuning* yang kurang sesuai.
- Algoritma Genetic Algorithm masih bisa dieksplorasi lagi meskipun kinerjanya baik.
- Bisa dibuat adanya tambahan mengenai *Item Constraints*. Jadi, semua algoritma dapat diberikan constraint agar hasilnya lebih variatif lagi.

Pembagian Tugas

18223122 Anggita Najmi Layali	
Source Code	• Membuat struktur direktori dan struktur entity, objective function, dan <i>state initializer</i> (beserta heuristic) • Membuat main • Membuat algoritma Steepest Ascent & Stochastic Hill Climbing • Membuat algoritma Genetic
Dokumentasi	• Membuat pendahuluan • Membuat pembahasan objective function dan heuristic • Membuat pembahasan Steepest Ascent, Stochastic, dan Genetic Algorithm

18223109 Fadhil Rifqi Rabbani Pane	
Source Code	• Membuat algoritma Stochastic Hill Climbing, Hill Climbing with Sideway Move, dan Random-Restart • Membuat algoritma Simulated Annealing
Dokumentasi	• Merapihkan layout dokumen • Membuat pembahasan Hill Climbing with Sideway Move, Random-Restart, dan Simulated Annealing • Membuat kesimpulan dan saran

Referensi

- Panggabean, T. N. (2016). *Analisis tingkat optimasi algoritma genetika dalam hukum ketetapan Hardy-Weinberg pada Bin Packing Problem*. CESS (Journal of Computer Engineering, System and Science), 1(2), 12–18. <https://doi.org/10.24114/cess.v1i2.4062>
jurnal.unimed.ac.id
- GeeksforGeeks. (2025, August 1). *Introduction: Hill climbing (Artificial Intelligence)*. Retrieved from <https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/> [GeeksforGeeks](https://www.geeksforgeeks.org/)
- GeeksforGeeks. (2024, March 8). *Genetic Algorithms*. Retrieved from <https://www.geeksforgeeks.org/dsa/genetic-algorithms/> [GeeksforGeeks](https://www.geeksforgeeks.org/)
- 4522210154 (Uploader). (n.d.). *Bin Packing Problem* [PDF]. Scribd. Retrieved from <https://www.scribd.com/document/697082069/Bin-packing-problem?v=0.724>
- GeeksforGeeks. (2024, June 1). *Implement simulated annealing in Python*. [GeeksforGeeks. https://www.geeksforgeeks.org/dsa/implement-simulated-annealing-in-python/](https://www.geeksforgeeks.org/dsa/implement-simulated-annealing-in-python/)
- Brownlee, J. (2021, October 11). *Simulated annealing from scratch in Python*. MachineLearningMastery.com. <https://machinelearningmastery.com/simulated-annealing-from-scratch-in-python/>